

**PROGRAM STUDI SARJANA SISTEM INFORMASI
PENELITIAN TUGAS AKHIR**



***Performance Analysis of Random Forest and XGBoost for Phishing Website
Detection with Integrated Rule-Based Filtering***

OLEH:

12S22005	Nikita Simanjuntak
12S22022	Grace Arintya Siahaan
12S22026	Ruben Martogi Jeremia Sianipar

**PROGRAM STUDI SARJANA SISTEM INFORMASI
FAKULTAS INFORMATIKA DAN TEKNIK ELEKTRO
INSTITUT TEKNOLOGI DEL
LAGUBOTI
2025**

ABSTRAK

Perkembangan teknologi digital yang pesat diikuti dengan meningkatnya ancaman keamanan siber, salah satunya adalah serangan phishing yang bertujuan mencuri informasi sensitif pengguna melalui situs web palsu. Tingginya jumlah serangan phishing menunjukkan perlunya sistem deteksi yang akurat, adaptif, dan efisien. Penelitian ini bertujuan untuk menganalisis kinerja algoritma Random Forest dan XGBoost baik sebagai model tunggal maupun model gabungan (*stacking*) dalam mendeteksi website phishing, serta mengevaluasi pengaruh integrasi rule-based filtering sebagai lapisan awal dalam sistem.

Metode yang digunakan meliputi proses data *preparation*, *preprocessing*, dan *feature extraction* dari dataset phishing yang diperoleh dari Kaggle, yang terdiri dari fitur berbasis URL, konten web, dan domain. Model dikembangkan menggunakan pendekatan *stacking ensemble* dengan Random Forest dan XGBoost sebagai base learners, serta Logistic Regression sebagai *meta-learner*. Selain itu, dilakukan optimasi *hyperparameter* menggunakan Genetic Algorithm untuk meningkatkan performa model. Rule-based filtering diterapkan sebagai tahap awal untuk menyaring pola *phishing* sederhana sebelum diproses oleh model *machine learning*.

Hasil penelitian menunjukkan bahwa model *ensemble stacking* memberikan kinerja yang lebih baik dibandingkan model tunggal dalam hal akurasi, presisi, recall, dan F1-score. Integrasi rule-based filtering terbukti mampu meningkatkan efisiensi pemrosesan serta mengurangi kesalahan klasifikasi, baik *false positive* maupun *false negative*. Dengan demikian, kombinasi pendekatan rule-based dan machine learning menghasilkan sistem deteksi *phishing* yang lebih stabil, adaptif, dan efektif dalam menghadapi pola serangan yang terus berkembang.

Nama : 1. Nikita Simanjuntak
2. Grace Arintya Siahaan
3. Ruben Martogi Jeremia Sianipar

Program Studi : Sarjana Sistem Informasi
Judul : Analisis Kinerja Random Forest dan XGBoost untuk
Deteksi Situs Web Phishing dengan Integrasi Rule-Based
Filtering

Kata kunci: *phishing, Machine learning, Random Forest, XGBoost, rule-based
filtering*

ABSTRACT

The rapid growth of digital technology has increased cybersecurity threats, especially phishing attacks. Phishing is a type of attack that tries to steal sensitive user information through fake websites. Because of the high number of phishing cases, there is a need for a detection system that is accurate, adaptive, and efficient. This study aims to analyze the performance of Random Forest and XGBoost, both as single models and as a combined model (ensemble), in detecting phishing websites. This study also evaluates the effect of using rule-based filtering as an initial step in the system.

The method includes data preparation, preprocessing, and feature extraction from a phishing dataset obtained from Kaggle. The dataset contains features from URLs, website content, and domain information. The model is built using a stacking ensemble method, where Random Forest and XGBoost are used as base models, and Logistic Regression is used as the final model (meta-learner). Hyperparameter tuning is also applied using a Genetic Algorithm to improve model performance. Rule-based filtering is used at the beginning to detect simple phishing patterns before using machine learning.

The results show that the ensemble model gives better performance than single models in terms of accuracy, precision, recall, and F1-score. The use of rule-based filtering also helps improve efficiency and reduce errors, such as false positives and false negatives. Therefore, combining rule-based methods and machine learning can produce a more stable, adaptive, and effective phishing detection system.

*Names : 1. Nikita Simanjuntak
2. Grace Arintya Siahaan
3. Ruben Martogi Jeremia Sianipar*
Study Program : Bachelor of Information Systems

*Title : Performance Analysis of Random Forest and XGBoost for
Phishing Website Detection with Integrated Rule-Based
Filtering*

Key word: *phishing, Machine learning, Random Forest, XGBoost, rule-based
filtering*

DAFTAR ISI

ABSTRAK	I
ABSTRACT	III
DAFTAR ISI	V
DAFTAR GAMBAR.....	XI
DAFTAR TABEL.....	XII
DAFTAR POTONGAN KODE.....	XIV
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Pertanyaan Penelitian	6
1.3 Tujuan Penelitian	7
1.4 Ruang Lingkup.....	7
1.5 Sistematika penyajian	7
BAB II LANDASAN TEORI	10
2.1 Phishing.....	10
2.2 Deteksi Phishing pada Website.....	12
2.3 Feature Extraction	13
2.4 Machine learning.....	15
2.5 Random Forest	16
2.5.1 Description	16
2.5.2 Hyperparameters of Random Forests.....	17
2.5.2.1 RF Hyperparameter num.trees	17
2.5.2.2 RF Hyperparameter mtry	18
2.5.2.3 RF Hyperparameter Sample.fraction	19
2.5.2.4 RF Hyperparameter Replace (bootstrap).....	20
2.5.2.5 RF Hyperparameter Respect.unordered.factors	21

2.6 XGBoost.....	23
2.6.1 Description	23
2.6.2 Hyperparameter of XGBoost	24
2.6.2.1 XGBoost Hyperparameter nrounds.....	24
2.6.2.2 XGBoost Hyperparameter eta	25
2.6.2.3 XGBoost Hyperparameter lambda	27
2.6.2.4 XGBoost Hyperparameter alpha	27
2.6.2.5 XGBoost Hyperparameter subsample.....	28
2.6.2.4 XGBoost Hyperparameter colsample_bytree	29
2.6.2.7 XGBoost Hyperparameter gamma	30
2.6.2.8 XGBoost Hyperparameter maxdepth	31
2.6.2.9 XGBoost Hyperparameter min_child_weight.....	31
2.7 Rule-Based Filtering	32
2.7.1 Struktur URL.....	36
2.8 Evaluasi	38
2.9 Penelitian Terdahulu	41
BAB III ANALISIS.....	43
3.1 Analisis Dataset.....	43
3.1.1 Data Preparation.....	43
3.1.1.1 Pengumpulan Data	43
3.1.1.2 Atribut Dataset	44
3.1.2 Data Preprocessing.....	50
3.1.2.1 Data Selection	50
3.1.2.2 Data Transformation	59
3.1.2.3 Data Cleaning.....	59
3.2 Analisis Machine Learning	60

3.2.1 Analisis Random Forest	60
3.2.2 Analisis XGBoost	63
3.2.3 Analisis Stacking Ensemble menggunakan Logistic Regression.....	66
3.3 Analisis Rule based Filtering	68
3.3.1 Rule 1: High-Confidence Rule.....	73
3.3.2 Rule 2 : Threshold-Based Rule	74
3.4 Analisis Dataset Eksperimen	76
3.4.1 Analisis <i>Parsing</i>	76
3.4.2 Analisis Feature Extraction	77
3.4.2.1 URL-Based (37 Fitur)	77
3.4.2.2 Non-URL (44 Fitur)	78
3.5 Analisis Eksperimen	79
3.5.1 Eksperimen Random Forest	79
3.5.2 Eksperimen XGBoost	80
3.5.3 Eksperimen Stacking.....	80
3.5.4 Eksperimen Rule based Filtering dengan Stacking.....	82
BAB IV DESAIN.....	85
4.1 Metode Penelitian	85
4.2 Rancangan Sistem	87
4.2.1 Desain Umum Sistem.....	87
4.3 Desain Analisis Dataset	91
4.3.1 Desain Data Preparation.....	92
4.3.2 Desain Data Preprocessing.....	94
4.3.2.1 Desain Data Selection	95
4.3.2.2 Desain Transformation Data	96
4.3.2.3 Desain Data Cleaning.....	98

4.4 Rancangan Arsitektur Machine Learning	98
4.4.1 Rancangan Arsitektur Random Forest	99
4.4.2 Rancangan Arsitektur XGBoost.....	100
4.5 Rancangan Arsitektur Rule based Filtering	102
4.6 Rancangan Arsitektur Eksperimen.....	103
4.7 Mockup Desain Sistem	104
4.7.1 Mockup Sistem Ketika URL Benign	105
4.7.2 Mockup Sistem Saat Proses Analisis	106
4.7.3 Mockup Sistem Ketika URL Phishing.....	106
BAB V IMPLEMENTASI	108
5.1 Lingkungan Implementasi.....	108
5.1.1 Perangkat Keras	108
5.1.2 Perangkat Lunak	108
5.2 Batasan Implementasi	109
5.3 Implementasi Analisis Dataset.....	109
5.3.1 Implementasi <i>Data Preparation</i>	109
5.3.1.1 Pembacaan Dataset.....	109
5.3.1.2 Jumlah baris dan kolom	110
5.3.1.3 Missing Values.....	110
5.3.1.4 URL Duplikat.....	111
5.3.1.5 Distribusi Label Status	111
5.3.1.6 Korelasi Antar Fitur	112
5.3.2 Implementasi Data Preprocessing	112
5.3.2.1 Data Selection	112
5.3.2.2 Data Transformation	113
5.3.2.3 Data Cleaning.....	114

5.4 Implementasi Data Split.....	114
5.5 Implementasi Model	115
5.5.1 Implementasi Persiapan Data dan Fitur	115
5.5.2 Implementasi Training Model Random Forest	117
5.5.3 Implementasi Training Model XGBoost.....	118
5.5.4 Implementasi Training Stacking Ensemble menggunakan Logistic Regression	119
5.5.5 Implementasi Evaluasi Model Final	120
5.5.6 Penyimpanan Model dan Metadata	121
5.6 Implementasi Rule based Filtering	122
5.7 Implementasi Evaluasi	124
5.8 Implementasi Feature Extraction	125
5.8.1 Normalisasi URL dan utilitas awal	126
5.8.2 Ekstraksi feature URL.....	127
5.8.3 Ekstraksi fitur konten web	128
5.8.4 Penggabungan fitur dan pembentukan vektor model.....	129
5.9 Implementasi Eksperimen.....	130
5.9.1 Implementasi Eksperimen Random Forest	130
5.9.2 Implementasi Eksperimen XGBoost.....	131
5.9.3 Implementasi Eksperimen Stacking.....	132
5.9.4 Implementasi Eksperimen Rule based Filtering dengan Stacking.....	133
BAB VI HASIL DAN PEMBAHASAN.....	137
6.1 Hasil	137
6.1.1 Hasil Analisis Dataset	137
6.1.2 Hasil Analisis Data Preprocessing	138
6.1.2.1 Hasil Data Selection.....	138

6.1.2.2 Hasil Data Transformasi	139
6.1.2.3 Hasil Data Cleaning	140
6.1.3 Hasil Model Machine Learning.....	140
6.1.3.1 Hasil Random Forest.....	140
6.1.3.2 Hasil XGBoost	143
6.1.3.3 Hasil Stacking	145
6.1.4 Hasil Rule Based Filtering	148
6.1.5 Hasil Perbandingan Kinerja Model.....	152
6.1.5 Hasil Eksperimen	155
6.1.5.1 Hasil Eksperimen Random Forest.....	155
6.1.5.2 Hasil Eksperimen XGBoost	158
6.1.5.3 Hasil Eksperimen Stacking Ensemble	160
6.1.5.4 Hasil Eksperimen Rule based Filtering dengan Stacking	162
6.2 Pembahasan.....	164
6.2.1 Pembahasan Eksperimen.....	164
6.2.1.1 Pembahasan Kinerja Model Tunggal Random Forest.	164
6.2.1.2 Pembahasan Kinerja Model Tunggal XGBoost.....	167
6.2.1.3 Pembahasan Kinerja Model Stacking Ensemble	169
6.2.1.4 Pembahasan Integrasi Rule-Based Filtering dengan Stacking....	172
6.2.2 Pembahasan Efisiensi dan Konsistensi Kinerja Sistem	175
6.2.2.1 Pembahasan Efisiensi Pemrosesan Rule-Based Filtering	175
6.2.2.2 Analisis Latency Prediksi per URL dan Konsistensi Kinerja	177
DAFTAR PUSTAKA	179

DAFTAR GAMBAR

Gambar 2. 1 Jenis <i>Machine Learning</i>	15
Gambar 2. 2 Struktur URL	37
Gambar 4. 1 Tahapan Penelitian	85
Gambar 4. 2 Desain Umum Sistem.....	89
Gambar 4. 3 Desain Analisis Dataset.....	92
Gambar 4. 4 Desain <i>Data Preparation</i>	93
Gambar 4. 5 Desain <i>Data Preprocessing</i>	95
Gambar 4. 6 Desain <i>Data Selection</i>	96
Gambar 4. 7 Desain <i>Transformation Data</i>	97
Gambar 4. 8 Desain <i>Data Cleaning</i>	98
Gambar 4. 9 Rancangan Arsitektur <i>Random Forest</i>	99
Gambar 4. 10 Rancangan Arsitektur <i>XGBoost</i>	101
Gambar 4. 11 Rancangan Arsitektur <i>Rule based Filtering</i>	103
Gambar 4. 12 Desain Eksperimen.....	104
Gambar 4. 13 <i>Mockup</i> Desain Sistem ketika Benign.....	105
Gambar 4. 14 <i>Mockup</i> Desain Sistem saat Proses	106
Gambar 4. 15 <i>Mockup</i> Desain Sistem ketika Phishing	107
Gambar 6. 1 Distribusi Kelas	138

DAFTAR TABEL

Tabel 2. 1 Tren dan Jenis Serangan <i>Phishing</i>	11
Tabel 2. 2 Contoh Proses Ekstraksi Fitur dari URL <i>Phishing</i>	14
Tabel 2. 3 <i>Characteristics of Rule-based Filtering</i>	32
Tabel 2. 4 Contoh <i>Rule-Based</i> dalam Deteksi <i>Phishing</i>	35
Tabel 2. 5 Komponen URL dan Data Fitur.....	37
Tabel 2. 6 <i>Confusion Matrix</i>	38
Tabel 2. 7 Penelitian Terdahulu	41
Tabel 3. 1 Karakteristik <i>Dataset Phishing</i>	43
Tabel 3. 2 Atribut pada Dataset dataset_phishing.....	44
Tabel 3. 3 <i>Feature Selection</i>	50
Tabel 3. 4 Analisis Pemilihan <i>Hyperparameter Random Forest</i>	61
Tabel 3. 5 Analisis Pemilihan <i>Hyperparameter XGBoost</i>	64
Tabel 3. 6 Metode Umum <i>Ensamble Learning</i>	66
Tabel 3. 7 <i>Rule base Filtering</i>	69
Tabel 3. 8 Kategori Fitur.....	74
Tabel 3. 9 Nilai Skor Risiko pada <i>Rule-based</i>	74
Tabel 3. 10 URL Based.....	77
Tabel 3. 11 Fitur Berbasis Web Content / HTML-DOM.....	78
Tabel 3. 12 Fitur Berbasis Domain, WHOIS, dan Reputasi	79
Tabel 3. 13 Fitur URL/Host Turunan Lanjutan	79
Tabel 5. 1 Perangkat keras (Hardware).....	108
Tabel 5. 2 Perangkat Lunak (Software)	108
Tabel 6. 1 Hasil Analisis Dataset	137
Tabel 6. 2 Hasil implementasi data selection.....	139
Tabel 6. 3 Hasil Data Transformasi	139
Tabel 6. 4 Hasil Data Cleaning	140
Tabel 6. 5 <i>Hyperparameter Random Forest</i>	141
Tabel 6. 6 <i>Confusion Matrix Random Forest</i>	141
Tabel 6. 7 Nilai Evaluasi Random Forest	142
Tabel 6. 8 <i>Hyperparameter XGBoost</i>	143

Tabel 6. 9 Confusion Matrix XGBoost	144
Tabel 6. 10 Nilai Evaluasi XGBoost.....	144
Tabel 6. 11 Konfigurasi Stacking Ensemble.....	146
Tabel 6. 12 Confusion Matrix Stacking Ensemble	146
Tabel 6. 13 Nilai Evaluasi Stacking Ensemble	147
Tabel 6. 14 Konfigurasi Sistem Hybrid	149
Tabel 6. 15 Confusion Matrix Hybrid.....	150
Tabel 6. 16 Nilai Evaluasi Hybrid	150
Tabel 6. 17 Hasil Perbandingan Kinerja Model.....	154
Tabel 6. 18 Hasil Eksperimen Random Forest	156
Tabel 6. 19 Hasil Eksperimen XGBoost	158
Tabel 6. 20 Hasil Eksperimen Stacking Ensemble	160
Tabel 6. 21 Hasil Eksperimen Rule based Filtering dengan Stacking	162
Tabel 6. 22 Kinerja Model Tunggal Random Forest	165
Tabel 6. 23 Kinerja Model Tunggal XGBoost.....	168
Tabel 6. 24 Kinerja Model Stacking Ensemble	170
Tabel 6. 25 Integrasi Rule-Based Filtering dengan Stacking.....	173
Tabel 6. 26 Perbandingan Efisiensi Pemrosesan Model	176
Tabel 6. 27 Latency Prediksi per URL.....	177

DAFTAR POTONGAN KODE

Kode 5. 1 Membaca dataset	109
Kode 5. 2 Menampilkan jumlah baris dan kolom	110
Kode 5. 3 Implementasi Missing Values	110
Kode 5. 4 Implementasi URL Data Duplikat.....	111
Kode 5. 5 Implementasi Analisis Distribusi Kelas Target.....	111
Kode 5. 6 Implementasi korelasi antar fitur.....	112
Kode 5. 7 Implementasi data selection	113
Kode 5. 8 Implementasi <i>data transformation</i>	113
Kode 5. 9 Implementasi korelasi antar fitur.....	114
Kode 5. 10 Implementasi data split.....	115
Kode 5. 11 Implementasi load dan prepare data	116
Kode 5. 12 Tuning Random Forest dengan Genetic Algorithm	117
Kode 5. 13 Tuning XGBoost dengan Genetic Algorithm.....	119
Kode 5. 14 Implementasi Stacking dengan Logistic Regression.....	119
Kode 5. 15 Implementasi Evaluasi Model Final.....	121
Kode 5. 16 Penyimpanan Model dan Metadata	122
Kode 5. 17 Implementasi <i>Rule based Filtering</i>	123
Kode 5. 18 Implementasi Evaluasi	125
Kode 5. 19 Normalisasi URL dan utilitas awal	126
Kode 5. 20 Ekstraksi feature URL	127
Kode 5. 21 Ekstraksi fitur konten web.....	129
Kode 5. 22 Penggabungan fitur dan pembentukan vektor model	129
Kode 5. 23 Implementasi Random Forest.....	131
Kode 5. 24 Implementasi XGBoost	132
Kode 5. 25 Implementasi Stacking	133
Kode 5. 26 Implementasi Rule based Filtering dengan Stacking	135

BAB I

PENDAHULUAN

Bab pendahuluan berisi penjelasan terkait latar belakang pemilihan topik tugas akhir, tujuan yang ingin dicapai dalam tugas akhir, ruang lingkup penelitian, tahapan yang dilakukan dalam penelitian, dan sistematika penyajian laporan tugas akhir.

1.1 Latar Belakang

Dalam perkembangan dunia digital saat ini, aktivitas pengguna internet semakin meningkat [1]. Namun, peningkatan ini juga diikuti oleh munculnya berbagai ancaman terhadap keamanan informasi. Salah satu ancaman yang paling sering terjadi adalah serangan *phishing* [2].

Phishing merupakan bentuk kejahatan siber yang memanfaatkan teknik manipulasi untuk menipu korban agar mengakses situs web palsu dan menyerahkan informasi penting, seperti kredensial akun, data keuangan, atau informasi pribadi lainnya. [3] Serangan ini dapat menimbulkan kerugian yang signifikan dan mencakup berbagai aspek, antara lain pencurian dana dari rekening atau kartu kredit, penyalahgunaan data pribadi korban untuk tindak penipuan, serta kerusakan reputasi individu maupun perusahaan. [4] Selain itu phishing yang dilakukan dengan menyamar sebagai entitas terpercaya dalam komunikasi elektronik mengganggu stabilitas sistem keuangan dan menurunkan kepercayaan publik terhadap layanan digital. Besarnya dampak yang ditimbulkan menggambarkan bahwa aktivitas *phishing* terus berkembang dan mengalami peningkatan dari waktu ke waktu.

Menurut laporan *Anti Phishing Working Group (APWG)* yang mencatat aktivitas serangan selama Januari hingga Maret 2025, tercatat sebanyak 1.003.924 serangan phishing yang terjadi di seluruh dunia. Jumlah ini merupakan yang tertinggi sejak akhir tahun 2023 dan menunjukkan tren peningkatan yang bertahap dari 877.536 awal 2024 naik menjadi 932.923 pada pertengahan tahun, dan 989.123 akhir 2024 [5]. Peningkatan ini menggambarkan bahwa phishing masih menjadi ancaman

serius di ranah keamanan siber. APWG juga melaporkan bahwa sektor layanan keuangan, pembayaran daring, dan perbankan menjadi target utama dengan total 30,9% dari seluruh serangan yang terjadi [5].

Seiring dengan meningkatnya ancaman *phishing* menunjukkan perlunya solusi yang efektif dan adaptif untuk mendeteksi serta mengidentifikasi situs berbahaya secara akurat. Oleh karena itu, penting untuk mengembangkan algoritma komputasi yang mampu mendeteksi dan mengidentifikasi situs web *phishing* dengan tingkat akurasi dan efisiensi yang tinggi [6][7]. Salah satu pendekatan yang banyak digunakan dalam pengembangan sistem tersebut adalah penerapan *Machine learning*, yaitu teknik yang memungkinkan sistem untuk belajar secara otomatis dari data atau contoh yang diberikan [7]. Melalui pendekatan ini, *Machine learning* dapat membantu mendeteksi adanya kejanggalan atau aktivitas mencurigakan secara lebih efektif dan adaptif terhadap pola serangan yang terus berkembang [8].

Beberapa penelitian telah membahas efektivitas *Machine learning* dalam mendeteksi situs web *phishing*. Penelitian yang membandingkan beberapa algoritma termasuk *Gradient Boosting Classifier*, *Random Forest*, *Decision Tree*, SVC, dan *K-Neighbors Classifier* menunjukkan bahwa dua algoritma pertama secara konsisten memberikan hasil akurasi tertinggi [9]. Namun, penelitian tersebut masih sebatas perbandingan tanpa mencoba menggabungkan model untuk hasil yang lebih stabil. Penelitian lain yang menggunakan algoritma *Decision Tree*, *Random Forest*, *Multilayer Perceptron*, *XGBoost*, dan *Support Vector Machines* (SVM) juga menemukan bahwa *Random Forest* dan *XGBoost* memiliki kinerja paling baik dalam mengenali situs *phishing* [10]. Penelitian ini belum mengatasi masalah *overfitting* dan belum melakukan evaluasi *meta-learning* untuk mengoptimalkan hasil prediksi.

Selanjutnya, penelitian terdahulu yang membandingkan algoritma *Random Forest*, *K-Nearest Neighbour* (KNN), dan *Artificial Neural Network* (ANN) di mana *Random Forest* menunjukkan akurasi tertinggi dalam mendeteksi situs *phishing* [11]. Penelitian ini modelnya cenderung menghasilkan keputusan yang kurang

adaptif terhadap pola serangan baru. Hasil lain diperoleh dari penelitian yang membandingkan *XGBoost*, *Random Forest*, dan *Decision Tree*, dengan hasil bahwa *XGBoost* yang telah dioptimalkan menggunakan *hyperparameter tuning* memberikan akurasi tertinggi [7]. Meski akurat, pendekatan tersebut memerlukan waktu pelatihan yang lebih lama dan penggabungan modelnya masih menggunakan metode sederhana seperti rata-rata, sehingga belum optimal dalam menangani variasi data yang kompleks.

Namun, seiring dengan evolusi teknik serangan *phishing*, tantangan baru yang menuntut sistem deteksi dengan stabilitas dan adaptabilitas yang lebih tinggi agar mampu menghadapi berbagai pola serangan yang terus berubah. Berbagai metode baru diusulkan untuk menangani keragaman jenis serangan yang semakin kompleks [12]. Hasil positif dari penelitian sebelumnya menunjukkan bahwa dibutuhkan pendekatan yang lebih kuat dan adaptif untuk mengatasi ancaman *phishing* yang terus berkembang. Oleh karena itu, penelitian ini mengusulkan penggunaan kombinasi dua algoritma *Machine learning* yang terbukti efektif [13]. *Random Forest* dan *XGBoost*, keduanya memiliki keunggulan dalam kemampuan beradaptasi terhadap data yang bervariasi, ketahanan terhadap *overfitting*, serta efisiensi dalam pemrosesan data berukuran besar. Penggunaan gabungan *Random Forest* dan *XGBoost* bertujuan untuk memberikan akurasi yang lebih tinggi, lebih stabil, dan lebih adaptif terhadap perubahan pola serangan *phishing* yang terus berkembang [14].

Dalam kombinasi *Random Forest* dan *XGBoost* terbukti efektif dalam mendeteksi situs *phishing* karena keduanya merepresentasikan dua pendekatan *ensemble* yang saling melengkapi [14]. *XGBoost* adalah algoritma *boosting* yang efisien karena mampu menjalankan proses secara paralelisasi serta menggunakan *Decision Tree* mempercepat perhitungan yang kompleks. yang membantu mencegah *overfitting*, terutama ketika digunakan pada dataset. Keunggulan lain dari *XGBoost* adalah kemampuannya dalam regularisasi, besar dengan banyak atribut yang tidak relevan atau mengandung *noise* [13][15]. Sementara itu, *Random Forest* merupakan algoritma *bagging* yang membangun beberapa *Decision Tree* untuk menghasilkan

prediksi gabungan yang lebih stabil. Keunggulan *Random Forest* ada pada kemampuannya mengurangi *varians*, terhadap *overfitting*, dan mampu melakukan generalisasi dengan baik, bahkan pada dataset besar dengan banyak fitur. Selain itu, *Random Forest* juga mudah dipahami karena menggunakan indeks *gini* atau *entropy*, serta fleksibel dalam menangani data yang tidak lengkap tanpa perlu dilakukan imputasi [16]. Kombinasi kedua algoritma ini dipilih untuk mengoptimalkan efektivitas pendekatan *bagging* dan *boosting* dalam konteks yang lebih adaptif. Kedua algoritma tersebut sama-sama berbasis *decision tree*, sehingga memiliki fondasi pemodelan yang serupa namun dengan karakteristik pembelajaran yang saling melengkapi. Keduanya memiliki dasar yang sama sebagai model berbasis *decision tree*, sehingga dapat dikombinasikan dengan baik dan saling memperkuat kemampuan satu sama lain. Penelitian ini bertujuan untuk mengembangkan pendekatan sebelumnya dengan menambahkan *rule-based filtering*. Komponen ini diharapkan mampu meningkatkan kemampuan sistem dalam mengenali pola serangan *phishing* yang terus berubah, sekaligus mempertahankan tingkat akurasi dan stabilitas model.

Pendekatan *ensemble stacking* diterapkan untuk menggabungkan dua algoritma *Machine learning* yaitu *Random Forest* dan *XGBoost*. *Stacking* merupakan salah satu teknik *ensemble learning* yang menggabungkan dua atau lebih model *Machine learning* [17] untuk meningkatkan performa prediksi. Proses *stacking* dilakukan dengan melatih beberapa model *base learners* secara paralel [17]. Metode ini meningkatkan performa prediksi dengan cara menggabungkan hasil dari beberapa *base learner* yang memiliki karakteristik berbeda, kemudian mempelajari hubungan antarhasil prediksi tersebut melalui *meta-learner* [18]. Kedua algoritma digunakan sebagai bagian dari model *ensemble stacking* yang dioptimasi menggunakan *Genetic Algorithm* untuk menyesuaikan parameter terbaik dari masing-masing model [19]. Pendekatan *ensemble* seperti *stacking* mampu mengatasi keterbatasan model tunggal dengan menggabungkan keunggulan berbagai algoritma.

Secara umum, proses *stacking* diawali dengan melatih kedua model, yaitu *Random Forest* dan *XGBoost*, sebagai *base learners* menggunakan dataset yang memuat fitur-fitur seperti URL, domain, dan konten situs. Setiap model menghasilkan probabilitas yang menggambarkan kemungkinan situs termasuk dalam kategori *phishing* atau *benign*, dengan nilai antara 0 hingga 1. Hasil prediksi dari kedua algoritma tersebut kemudian dikombinasikan dan dijadikan masukan bagi *meta learner* [20], yang bertugas mempelajari pola dari hasil prediksi tersebut untuk menghasilkan keputusan akhir yang lebih akurat dan stabil.

Dalam penelitian ini, Logistic Regression digunakan sebagai *meta learner* karena model ini memiliki sifat yang ringan, stabil, dan interpretatif. Logistic Regression mampu menggabungkan *output probabilistik* dari kedua *base learner* secara efisien tanpa menimbulkan *overfitting* [18], serta menghasilkan bobot yang dapat menunjukkan seberapa besar kontribusi masing-masing model terhadap prediksi akhir. Selain itu, Logistic Regression juga merupakan pilihan yang umum dan efektif dalam *stacking ensemble* karena kemampuannya membentuk kombinasi linear dari hasil prediksi model dasar, sehingga dapat mempelajari pola kesalahan keduanya dan menghasilkan keputusan yang lebih seimbang.

Kontribusi utama penelitian ini terletak pada penerapan *rule based filtering* dengan model gabungan *Random Forest* dan *XGBoost*. *Rule-based filtering berfungsi* sebagai lapisan awal yang menyaring hasil deteksi menggunakan aturan logis sederhana (misalnya panjang URL, simbol mencurigakan, atau umur domain), tetapi juga sebagai analisis pola serangan baru [21]. Fungsi utamanya adalah sebagai filter awal untuk mengidentifikasi pola *phishing* klasik yang jelas, sehingga hanya kasus ambigu yang diproses lebih lanjut oleh algoritma *Machine learning*. Dengan demikian, *rule-based filtering* tidak hanya meningkatkan efisiensi komputasi, tetapi juga membantu mengurangi kesalahan deteksi, yaitu situs aman ditandai *phishing* (*False Positive*) maupun situs *phishing* ditandai sebagai aman (*False Negative*) [22]. Selain itu, *rule-based filtering* berperan penting dalam menyiapkan data yang lebih bersih dan terfokus bagi model *Machine learning* yang digunakan.

Dengan menggabungkan *rule-based filtering* yang berperan sebagai lapisan penyaring awal, *XGBoost* yang berfungsi mengurangi bias, dan *Random Forest* yang mengurangi varians, sistem deteksi *phishing* yang dihasilkan mampu memanfaatkan kelebihan masing-masing pendekatan [22]. Kombinasi ini memungkinkan sistem lebih cerdas untuk bekerja secara efisien, responsif, menjaga kestabilan kinerja, serta lebih adaptif terhadap perubahan pola serangan *phishing* yang terus berkembang. Data yang tidak terklasifikasi dengan baik oleh aturan awal akan dikaji kembali melalui hasil inferensi model *Random Forest* dan *XGBoost* untuk menemukan pola baru yang berpotensi menjadi fitur atau aturan tambahan. Pendekatan dua arah ini memastikan bahwa tidak ada data *phishing* yang hilang dari proses pelatihan, karena setiap data yang tidak lolos filter akan tetap dianalisis ulang melalui tahap pembelajaran mesin

Dengan demikian, model *ensemble Random Forest–XGBoost* yang diperkaya dengan *rule-based filtering* mampu memberikan solusi yang lebih komprehensif dan andal dibandingkan penggunaan satu model saja [12]. Penelitian ini diharapkan dapat berkontribusi dalam pengembangan sistem deteksi *phishing* yang cerdas, dan berkelanjutan, sehingga mampu memberikan perlindungan yang lebih efektif terhadap ancaman keamanan siber di masa mendatang.

1.2 Pertanyaan Penelitian

Pertanyaan penelitian yang akan dilakukan dalam pengerjaan tugas akhir ini adalah sebagai berikut.

1. Bagaimana kinerja algoritma *Random Forest* dan *XGBoost* sebagai model tunggal serta sebagai model gabungan (*ensemble*) dalam mendeteksi website *phishing* berdasarkan evaluasi performa model?
2. Bagaimana pengaruh penerapan *rule-based filtering* sebagai lapisan awal terhadap efisiensi pemrosesan dan konsistensi kinerja prediksi sistem deteksi *phishing* berbasis model gabungan *Random Forest* dan *XGBoost*?

1.3 Tujuan Penelitian

Tujuan penelitian dalam tugas akhir ini adalah sebagai berikut.

1. Mengevaluasi dan membandingkan kinerja algoritma *Random Forest* dan *XGBoost* sebagai model tunggal serta sebagai model gabungan (*ensemble*) dalam mendeteksi website *phishing* baru yang lebih kompleks berdasarkan metrik *accuracy*, *precision*, *recall*, *F1-score*, dan *AUC*.
2. Menganalisis dan mengevaluasi pengaruh penerapan *rule-based filtering* sebagai lapisan awal terhadap efisiensi pemrosesan dan konsistensi kinerja prediksi sistem deteksi *phishing* berbasis *stacking* antara model *Random Forest* dan *XGBoost*.

1.4 Ruang Lingkup

1. Dataset yang digunakan merupakan kumpulan data *phishing* dan *benign website* yang diperoleh dari repositori publik *Kaggle*. Dataset ini berisi URL beserta fitur-fitur relevan yang dapat digunakan dalam proses klasifikasi.
2. Implementasi sistem ini mencakup pembuatan *dashboard* interaktif yang memungkinkan pengguna untuk melakukan input URL dan mendapatkan hasil klasifikasi *phishing* atau *benign*. *Dashboard* ini tidak hanya menampilkan hasil klasifikasi dalam bentuk probabilitas untuk masing-masing kategori (*phishing* atau *benign*), tetapi juga memberikan informasi mengenai fitur-fitur utama yang paling berpengaruh dalam klasifikasi tersebut.

1.5 Sistematika penyajian

Dokumen tugas akhir ini disusun dengan sistematika sebagai berikut untuk memberikan gambaran yang jelas dan terstruktur mengenai penelitian yang dilakukan:

1. Bab 1 Pendahuluan

Bab ini berisi uraian mengenai dasar dilakukannya penelitian. Bagian ini meliputi:

1. Latar Belakang, yang menjelaskan fenomena meningkatnya serangan *phishing*, kelemahan deteksi menggunakan satu algoritma, serta urgensi penggunaan kombinasi metode *Machine learning*.
2. Rumusan Masalah, yang memuat pertanyaan penelitian yang ingin dijawab.
3. Tujuan Penelitian, yang menjelaskan target yang ingin dicapai melalui penelitian ini.
4. Ruang Lingkup Penelitian, yang menjelaskan batas cakupan penelitian, meliputi penggunaan dataset publik, penerapan algoritma *Random Forest* dan *XGBoost*, serta integrasi dengan *rule-based filtering*
5. Batasan Penelitian, yang memperjelas keterbatasan penelitian, baik dari sisi algoritma, data, maupun implementasi sistem.
6. Sistematika Penyajian, yaitu gambaran umum isi dokumen tugas akhir.

2. Bab 2 Landasan Teori

Bab ini membahas teori-teori dan konsep yang menjadi dasar penelitian, serta penelitian terdahulu yang relevan. Bagian ini meliputi:

1. *Phishing*, yang menjelaskan definisi, karakteristik, teknik serangan, dan dampak yang ditimbulkan.
2. *Machine learning* untuk Deteksi *Phishing*, yang membahas konsep *supervised learning*, klasifikasi biner, dan relevansinya dalam mendeteksi *phishing*.
3. Algoritma *Random Forest*, yang membahas prinsip kerja, konsep *bagging*, keunggulan, dan keterbatasannya.
4. Algoritma *XGBoost*, yang membahas prinsip kerja, konsep boosting, keunggulan, dan keterbatasannya.
5. *Rule-Based Filtering*, yang membahas definisi, contoh aturan sederhana (misalnya panjang URL, simbol mencurigakan, umur domain), serta peranannya dalam menyaring hasil deteksi agar lebih efisien.

6. Evaluasi, yang akan membahas teori dan metrik yang digunakan untuk menilai kinerja model deteksi *phishing*. Evaluasi dilakukan untuk mengukur seberapa baik model mampu membedakan antara situs *phishing* dan situs yang *benign*.
 7. Penelitian Terdahulu, yang menyajikan ringkasan penelitian-penelitian relevan dalam bentuk tabel, mencakup judul, tahun, hasil, kesimpulan, serta kekurangan penelitian terdahulu sebagai dasar penelitian ini.
3. Bab 3 Metode Penelitian

Bab ini menjelaskan pendekatan dan tahapan penelitian yang dilakukan. Bagian ini meliputi:

1. Tahapan Penelitian, yang dipaparkan dalam bentuk *flowchart* serta penjelasan setiap tahap, mulai dari studi literatur, pengumpulan data, pra proses data, penerapan *rule-based filtering*, pembangunan model dengan *Random Forest* dan *XGBoost*, evaluasi model, hingga implementasi sistem pengecekan sederhana.
2. Teknik Pengumpulan Data, yang menjelaskan sumber dataset (misalnya *Kaggle*, *UCI*, *PhishTank*, atau *OpenPhish*) serta cara validasi dan persiapan data.
3. Proses Pengolahan Data, yang mencakup pembersihan data, ekstraksi fitur URL dan domain.
4. Metode Analisis, yang menjelaskan penggunaan algoritma *Random Forest*, *XGBoost*, kombinasi keduanya, serta integrasi *rule-based filtering*, disertai metrik evaluasi seperti *accuracy*, *precision*, *recall*, *F1-score*, dan *AUC*.

BAB II

LANDASAN TEORI

Bab ini berisi pemaparan konsep dasar dan teori yang mendukung penelitian, serta uraian mengenai hasil penelitian terdahulu yang relevan. Landasan teori ini menjadi dasar konseptual bagi pengembangan sistem deteksi *phishing website* menggunakan kombinasi algoritma *Random Forest*, *XGBoost*, dan *rule-based filtering*.

2.1 Phishing

Phishing adalah salah satu bentuk kejahatan siber yang dilakukan dengan cara memanipulasi korban agar memberikan informasi pribadi, seperti *username*, *password*, data kartu kredit, atau kredensial akun lainnya [23]. Serangan *phishing* biasanya dilakukan melalui email, pesan instan, media sosial, maupun *website* tiruan yang menyerupai situs resmi [1].

Beberapa jenis *phishing* yang paling umum antara lain email *phishing*, *website phishing*, *spear phishing*, *whaling* dan *smishing* [4]. Email *phishing* melibatkan pengiriman email palsu yang mengelabui korban untuk memberikan informasi pribadi atau mengunduh *malware* [23]. *Website phishing* melibatkan pembuatan situs web palsu yang mirip dengan situs web asli untuk menipu pengguna agar memasukkan data sensitif mereka seperti kata sandi, nomor kartu kredit, atau informasi pribadi lainnya [23]. *Spear phishing* adalah serangan yang ditargetkan pada individu atau organisasi tertentu menggunakan informasi yang dipersonalisasi untuk meningkatkan efektivitas serangan [4]. *Whaling* adalah jenis *spear phishing* yang menargetkan individu berprofil tinggi seperti eksekutif perusahaan atau pejabat pemerintah. *Smishing*, atau SMS *phishing*, adalah jenis *phishing* yang menggunakan pesan teks untuk menipu korban [4].

Website phishing sering kali memiliki karakteristik yang mencurigakan, seperti penggunaan domain yang sangat mirip dengan domain asli, dengan mengganti atau menambahkan satu karakter dalam alamat URL (misalnya, mengganti "g" dengan

"q") [23][1]. Selain itu, tampilan *website phishing* biasanya disalin dengan sangat detail, termasuk logo, warna, dan tata letak, untuk menipu korban agar percaya bahwa mereka berada di situs resmi [4][23]. Meskipun menggunakan *HTTPS*, *website phishing* sering memanfaatkan sertifikat SSL yang tidak sah atau palsu, yang dapat mengecoh pengguna dengan memberi kesan bahwa situs tersebut aman padahal sebenarnya tidak [4][1].

Kerugian yang ditimbulkan akibat *phishing* dapat berupa kerugian finansial, pencurian identitas, hingga kerusakan reputasi perusahaan maupun individu. Korban *phishing* bisa kehilangan uang secara langsung, terutama jika mereka memberikan informasi kartu kredit atau melakukan transfer uang ke penyerang [23]. Selain itu, data yang dicuri, seperti NIK atau informasi bank, dapat digunakan untuk mencuri identitas korban dan melakukan penipuan lebih lanjut [4]. Serangan *phishing* terhadap perusahaan atau lembaga juga dapat merusak reputasi mereka, mengurangi kepercayaan pelanggan, dan menciptakan keraguan tentang keamanan data [1]. Menurut penelitian [1], serangan *phishing* semakin marak terjadi di platform media sosial karena tingginya interaksi pengguna, sehingga memperbesar peluang penyerang untuk menyebarkan tautan berbahaya [1]. Hal ini menunjukkan betapa pentingnya kewaspadaan pengguna dalam berinteraksi di media sosial untuk mencegah serangan *phishing*. Pada Tabel 2. 1 menunjukkan tren dan jenis serangan *phishing* terbaru.

Tabel 2. 1 Tren dan Jenis Serangan *Phishing*

No	Jenis	Target	Penjelasan
1.	<i>Spear Phishing</i>	Pejabat, perusahaan tertentu, atau individu.	Serangan sangat personal atau kelompok tertentu, menyesuaikan komunikasi dengan target, sering menggunakan data media sosial untuk membobol basis data rahasia.
2.	<i>Whaling</i>	Direktur perusahaan atau eksekutif tingkat tinggi.	Menyamar sebagai staf atau mengirim pengumuman internal palsu via email/medsos untuk mengganggu perusahaan.

No	Jenis	Target	Penjelasan
3.	<i>Vishing</i>	Panggilan suara (<i>Voice Call</i>).	Memanfaatkan rasa mendesak atau takut (misalnya, berita kecelakaan/masalah hukum) agar korban mentransfer uang. Menggunakan nomor tidak dikenal.
4.	<i>Smishing</i>	Pesan teks (SMS)	Menggunakan manipulasi, seringkali dengan modus memenangkan undian 5atau hadiah besar, menyamar sebagai pihak resmi.
5.	<i>Social Media-Based Phishing</i>	<i>Platform</i> media sosial (Facebook, Instagram, dll.).	Menyebarkan tautan <i>phishing</i> dari akun terpercaya atau membuat profil palsu untuk membangun hubungan dan kepercayaan.
6.	<i>Deepfake AI</i>	Teknologi AI untuk membuat video atau audio palsu yang sangat meyakinkan.	Membuat rekaman palsu mirip atasan/rekan kerja, meminta korban mentransfer uang atau memberikan informasi pribadi.
7.	<i>Scam Phishing</i>	Mencuri informasi pribadi sensitif (nomor rekening, kata sandi, dll.).	Mengirim tautan/berkas berisi malware untuk mencuri informasi, media umum: SMS, email, telepon, atau medsos.
8.	<i>Cloning and Blind Phishing</i>	<i>Cloning</i> meniru situs web asli. <i>Blind Phishing</i> adalah serangan yang tidak menargetkan individu spesifik.	<i>Cloning</i> : Membuat situs web palsu untuk mencuri data input korban. <i>Blind</i> : Menyebarkan pesan <i>phishing</i> secara massal tanpa target spesifik (misalnya via email/SMS).

Sumber: [4]

2.2 Deteksi Phishing pada Website

Deteksi *phishing* di *website* bertujuan untuk mengidentifikasi dan memblokir situs *website* yang berbahaya sebelum dapat merugikan penggunanya. Dalam melakukan deteksi *phishing*, ada beberapa pendekatan yang digunakan, antara lain *blacklist-*

based, heuristic-based, dan Machine learning-based. Blacklist-based sangat efisien dalam mendeteksi situs web *phishing* yang sudah dikenal, namun kurang efektif dalam menangani situs baru [24]. *Heuristic-based* dapat lebih fleksibel dan mendeteksi situs web dengan karakteristik mencurigakan, tetapi sering kali terhambat oleh keterbatasan dalam aturan yang ditetapkan [23][25].

Machine learning-based lebih kuat dalam mendeteksi situs *phishing* yang tidak dikenal, tetapi membutuhkan data pelatihan yang banyak dan sumber daya komputasi yang besar [26][23]. Dalam penelitian ini, penggabungan *Machine learning* dengan *rule-based filtering* dipilih karena kombinasi kedua pendekatan ini dinilai memberikan hasil yang optimal [25]. *Machine learning* memungkinkan deteksi situs *phishing* yang belum diketahui sebelumnya, sedangkan *rule-based filtering* dapat memberikan lapisan awal untuk menyaring situs web dengan ciri-ciri umum yang dapat dikenali lebih cepat [24][25]. Melalui integrasi kedua pendekatan tersebut, keunggulan masing-masing dapat dimanfaatkan untuk meningkatkan akurasi dan efisiensi sistem deteksi *website phishing* [26][25].

2.3 Feature Extraction

Feature extraction adalah proses penting dalam sistem deteksi *phishing*, di mana data mentah dari *website* diubah menjadi fitur numerik atau representasi lain yang dapat diproses oleh algoritma *Machine learning* [27]. Dalam konteks deteksi *phishing website* menggunakan *Random Forest, XGBoost, dan rule-based filtering, feature extraction* memegang peranan kunci karena memungkinkan sistem untuk mengubah data yang bersifat raw (seperti URL, konten HTML, informasi DNS, dan atribut domain) menjadi format yang lebih terstruktur dan dapat dianalisis oleh algoritma *Machine learning* [28][29]. Dengan mengaplikasikan teknik *feature extraction* yang efektif, model *Machine learning* dapat mempelajari pola-pola yang membedakan antara *website* yang sah dan yang berpotensi berbahaya, yang akan meningkatkan akurasi sistem dalam mendeteksi *website phishing* [27]. Proses ekstraksi fitur dari URL ini mengubah elemen-elemen URL yang berbeda menjadi fitur numerik yang dapat dianalisis lebih lanjut oleh algoritma deteksi, memungkinkan sistem untuk mendeteksi URL yang mencurigakan dengan lebih

efisien [30]. URL merupakan elemen utama yang sering dianalisis dalam deteksi *phishing* karena banyak serangan *phishing* memanfaatkan URL yang tampak mirip dengan situs web sah namun telah dimodifikasi untuk menipu pengguna [27]. Beberapa fitur yang dapat diekstraksi dari URL antara lain panjang URL, di mana URL *phishing* cenderung lebih panjang dan kompleks untuk menyamarkan tujuan aslinya jumlah subdomain yang berlebihan seperti pada *login.bank.fakewebsite.com* yang sering menjadi indikator aktivitas *phishing* [28], keberadaan simbol mencurigakan seperti tanda “@”, “/” ganda, atau banyak karakter pemisah “-” yang digunakan untuk menutupi tujuan sebenarnya [29].

Tabel 2. 2 menampilkan contoh URL sebelum dan sesudah melalui proses ekstraksi fitur, yang diadaptasi dari penelitian Aung & Yamana (2024) dalam jurnal berjudul “*PhiSN: Phishing URL Detection Using Segmentation and NLP Features*” Proses ekstraksi ini bertujuan untuk memisahkan komponen penting dalam URL yang dapat digunakan untuk mendeteksi indikasi *phishing*.

Tabel 2. 2 Contoh Proses Ekstraksi Fitur dari URL *Phishing*

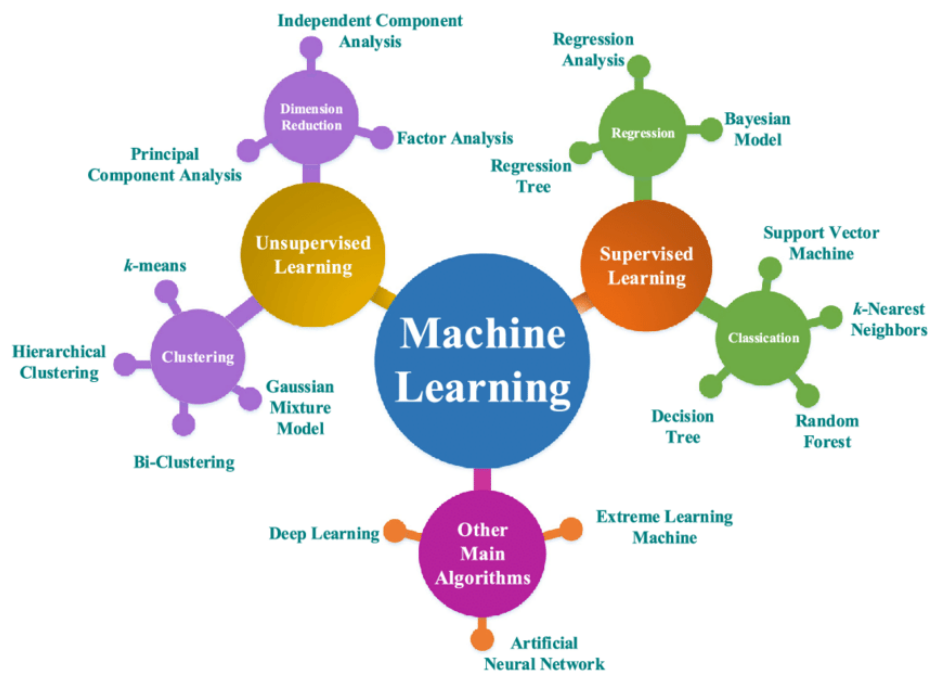
No	URL Asli	Ekstraksi Fitur	Keterangan
1.	http://www.amaz0necure.me/login	["amazon", "secure", "login"]	Terdapat penggantian karakter “0” → “o”; panjang URL > 75 karakter; mengandung kata “login”.
2.	https://paypal-update-info.com/verify	["paypal", "update", "info", "verify"]	Mengandung kata “update” dan “verify”; domain tidak resmi dari brand.
3.	http://31.192.106.250/bank/login	["31.192.106.250", "bank", "login"]	Domain menggunakan IP address, bukan <i>hostname</i> .
4.	https://secure-google-account.co/reset	["secure", "google", "account", "reset"]	Mengandung kata “secure” dan “reset”; domain menyerupai domain sah “google.com”.
5.	http://update-facebook-security.info/login	["update", "facebook", "security", "login"]	Panjang URL berlebih; mengandung kata “update” dan “login”.

Sumber: [11]

2.4 Machine learning

Machine learning merupakan cabang dari *artificial intelligence* yang memungkinkan komputer untuk belajar dari data guna melakukan prediksi atau klasifikasi [8]. Dalam konteks deteksi *phishing*, *Machine learning* digunakan untuk membedakan antara *website phishing* dan *benign* berdasarkan sejumlah fitur, seperti panjang URL, jumlah subdomain, penggunaan simbol mencurigakan, umur domain, dan validitas sertifikat SSL [31][32].

Penelitian [33] menunjukkan bahwa algoritma *Machine learning* seperti *Logistic Regression*, *K-Nearest Neighbor* (KNN), *Decision Tree*, *Random Forest*, dan *XGBoost* dapat digunakan untuk klasifikasi *phishing* dengan tingkat akurasi yang cukup tinggi [33]. Dalam hal ini, teknik *supervised learning* digunakan, yang di mana model dilatih menggunakan dataset berlabel, yang biasanya terdiri dari label *phishing* (*malicious*) dan *benign* (*non-phishing*) [32]. Dengan memanfaatkan data berlabel, model dapat mempelajari pola yang membedakan antara *website* berbahaya dan yang aman [33]. Pada Gambar 2. 1 adalah gambaran jenis *meachine learning*



Gambar 2. 1 Jenis *Machine Learning*

Sumber: [11]

2.5 Random Forest

2.5.1 Description

Random Forest adalah algoritma *ensemble learning* yang menggunakan pendekatan *bagging (Bootstrap Aggregating)* untuk membangun banyak *Decision Trees* secara acak dan menghasilkan prediksi berdasarkan *voting* mayoritas dari setiap *decision tree* [34]. Algoritma ini membangun banyak *decision tree*, di mana setiap *decision tree* berdasarkan bagian-bagian acak dari data pelatihan [35]. Pendekatan ini bertujuan untuk meningkatkan akurasi dan stabilitas model, dengan cara mengurangi kesalahan dan meningkatkan ketahanan terhadap *overfitting* (terlalu menyesuaikan model dengan data latihan), sehingga model dapat bekerja dengan lebih baik meskipun menggunakan data yang besar dan beragam fitur [36].

Keunggulan *Random Forest* adalah sebagai berikut:

- Mengurangi kesalahan dan meningkatkan stabilitas: Karena menghasilkan banyak *decision tree*, setiap *tree* membuat prediksi secara terpisah, dan hasil akhirnya ditentukan berdasarkan suara terbanyak dari setiap *tree*, yang mengurangi kesalahan model secara keseluruhan [37].
- Tahan terhadap *overfitting*: *Random Forest* dapat menangani *noise* (gangguan) dan kerumitan data tanpa kehilangan kemampuan untuk membuat prediksi yang akurat, menjadikannya efektif untuk data yang besar dan bervariasi [38].
- Performa baik meskipun dataset berukuran besar dengan banyak fitur: *Random Forest* tetap dapat mengelola berbagai fitur tanpa mengurangi performa, bahkan ketika dataset yang digunakan sangat besar dan memiliki banyak variabel [39].
- Relatif mudah dipahami: Algoritma ini menggunakan indeks *gini* atau *entropy* untuk memilih fitur terbaik dalam pemisahan data, sehingga lebih mudah dipahami dibandingkan dengan banyak algoritma lainnya [34].
- Dapat menangani data yang hilang tanpa perlu perbaikan tambahan: *Random Forest* dapat bekerja dengan data yang memiliki nilai kosong tanpa perlu melakukan langkah khusus untuk mengisi data yang hilang [37].

Penelitian dalam [37] dan [39] menunjukkan bahwa *Random Forest* sangat andal dalam mengklasifikasikan situs web *phishing* karena kemampuan algoritma ini untuk mengatasi gangguan dan ketahanan terhadap *noise* pada data [36][38].

2.5.2 Hyperparameters of Random Forests

Pada bagian ini merupakan beberapa *Hyperparameter* yang dalam algoritma *random forest* yaitu :

2.5.2.1 RF Hyperparameter num.trees

Number of trees adalah *hyperparameter* yang menentukan jumlah *tree* yang digabungkan dalam model *ansemble* secara keseluruhan. Pada umumnya semakin banyak *tree*, maka semakin baik kualitas performa pemodelan serta waktu yang dibutuhkan untuk melatih dan membuat prediksi dengan model semakin banyak.

Type:	Integer, Skalar
Default:	Log (500,2)
Sensitivity:	Menurut Breiman (2001), kesalahan generalisasi dari model akan konvergen dengan [24] meningkatnya jumlah <i>tree</i> ke suatu nilai batas yang lebih rendah. Hal ini berarti bahwa model akan menjadi kurang sensitif terhadap perubahan num.trees dengan nilai num.trees yang semakin tinggi. Hal ini juga terlihat dalam benchmark Louppe (2015). Hanya dengan nilai kecil (num.trees <50), model sangat sensitif terhadap perubahan parameter ini. Hasil empiris dari Probst et al. (2019) juga menunjukkan bahwa tingkat kemampuan menyesuaikan (<i>tunability</i>) num.trees diperkirakan rendah.
Heuristik:	Ada hasil teoritis tentang konvergensi model terkait num.trees (Breiman 2001; Scornet 2017). Namun, ini tidak menghasilkan pendekatan heuristik yang jelas untuk menentukan parameter ini. Rekomendasi umum adalah untuk memilih num.trees yang cukup tinggi (Probst et al. 2019) (karena lebih banyak <i>tree</i> biasanya lebih baik), sambil

memastikan bahwa waktu eksekusi model tidak menjadi terlalu lama.

Rentang:	$num. trees \in [1, \infty)$
Transformasi:	<i>trans_2pow_round</i>
Batas:	<i>Lower = 0; upper = 11.</i>
Keterbatasan:	Tidak ada.
Interaksi:	Tidak diketahui

2.5.2.2 RF Hyperparameter mtry

Hyperparameter mtry menentukan berapa banyak fitur yang dipilih secara acak untuk dipertimbangkan pada setiap pemisahan. Oleh karena itu, mtry mengontrol aspek penting, yaitu randomisasi *tree* individu. Nilai mtry $\ll n$ mengimplikasikan bahwa perbedaan antara *tree* akan lebih besar (lebih banyak randomisasi). Ini meningkatkan potensi kesalahan *tree* individu, tetapi keseluruhan *ensemble* menjadi lebih baik (Breiman 2001; Louppe 2015). Sementara itu, mtry $\ll n$ juga dapat mengurangi waktu *runtime* secara signifikan (Louppe 2015). Namun demikian, temuan tentang parameter ini sangat tergantung pada heuristik dan hasil empiris. Menurut Scornet (2017), tidak ada hasil teoritis tentang randomisasi fitur pemisahan yang tersedia.

Type:	Integer , Skalar
Default:	Floor (sqrt(nFeatures))
Sensivitas:	Menurut Breiman (2001), <i>Random Forest</i> relatif tidak sensitif terhadap perubahan mtry: "Namun prosedur ini tidak terlalu sensitif terhadap nilai F. Perbedaan absolut rata-rata antara tingkat kesalahan menggunakan F = 1 dan nilai yang lebih tinggi dari F kurang dari 1." (Breiman 2001) (di sini: F sesuai dengan mtry). Hal ini nampaknya bertentangan dengan hasil benchmark oleh Louppe (2015), yang menentukan bahwa mtry mungkin memang memiliki dampak yang cukup besar, terutama untuk nilai mtry yang rendah. Penelitian tentang "tunability" oleh Probst et al. (2019) juga mengidentifikasi

mtry sebagai parameter penting (yaitu, dapat diatur). Ini tidak selalu menjadi kontradiksi dengan pengamatan Breiman, karena Probst et al. (2019) menentukan *Random Forest* sebagai model yang paling tidak dapat diatur dalam investigasi eksperimental mereka. Jadi, meskipun mtry mungkin memiliki beberapa dampak (diban dingkan dengan parameter lain), mungkin kurang sensitif dibandingkan dengan *hyperparameter* dari model lain.

Heuristik: Breiman (2001) mengusulkan heuristik berikut: $mtry = \text{floor}(\log_2(n) + 1)$. Jika fitur kategorikal ada, Breiman menyarankan untuk menggandakan atau mengtripel nilai tersebut. Tidak ada motivasi teoretis yang diberikan. Suggestion yang sering digunakan adalah $mtry = \sqrt{n}$ (atau $mtry = \text{floor}(\sqrt{n})$). Meskipun ini digunakan dalam berbagai implementasi *Random Forest*, tidak ada motivasi teoretis yang jelas. Untuk $n < 20$ kedua heuristik memberikan nilai yang sangat mirip. Beberapa implementasi membedakan antara klasifikasi ($mtry = \sqrt{n}$) dan regresi ($mtry = n/3$). Hasil empiris dengan heuristik ini dijelaskan oleh Probst et al. (2019c).

Rentang: $mtry \in [1, n]$.
 Transformasi: `trans_id`
 Batas: `lower = 1; upper = nFeatures`
 Keterbatasan: Tidak ada
 Interaksi: Tidak diketahui

2.5.2.3 RF Hyperparameter `Sample.fraction`

Parameter `sample.fraction` menentukan jumlah pengamatan yang secara acak digunakan untuk melatih satu *tree* dalam model. Menurut Probst et al. (2019c), `sample.fraction` memiliki pengaruh yang sama seperti `mtry`, yaitu mempengaruhi sifat *tree* dalam model. Dengan menggunakan nilai `sample.fraction` yang kecil (sama dengan nilai `mtry` yang kecil), kualitas prediksi dari *tree* individu menjadi

lebih rendah, tetapi keberagaman *tree* meningkat. Hal ini berdampak positif pada kualitas model *ensemble*. Selain itu, nilai *sample.fraction* yang lebih kecil juga dapat mengurangi waktu *runtime* dari model.

Type:	Double, Skalar
Default:	1
Sensivitas:	<i>sample.fraction</i> dapat berdampak signifikan pada kualitas model. Scornet melaporkan: Namun, berdasarkan hasil empiris, tidak ada justifikasi untuk nilai default dalam <i>random forest</i> untuk sub sampling atau kedalaman <i>tree</i> , karena mengoptimalkan salah satunya menghasilkan kinerja yang lebih baik.
Heuristik:	Tidak diketahui
Rentang:	$sample.fraction \in (0, 1]$.
Transformasi:	<i>trans_id</i>
Batas:	$Lower = 0.1; upper = 1$
Keterbatasan:	Tidak ada.
Interaksi:	Potensialnya, <i>sample.fraction</i> berinteraksi dengan parameter yang mempengaruhi pelatihan <i>tree</i> individu DT (misalnya <i>maxdepth</i> , <i>minsplit</i> , <i>cp</i>). Scornet: "Menurut analisis teoritis dari median forests, bahwa tidak perlu mengoptimalkan ukuran sub-sampling dan kedalaman <i>tree</i> : mengoptimalkan hanya salah satu dari dua parameter ini menghasilkan kinerja yang sama dengan mengoptimalkan keduanya" (Scornet 2017). Namun, pengamatan teoritis ini hanya berlaku untuk median <i>trees</i> masing-masing dan tidak selalu untuk model <i>Random Forest</i> klasik yang menjadi pertimbangan.

2.5.2.4 RF Hyperparameter Replace (bootstrap)

Parameter *replace* menentukan apakah sampel yang diambil secara acak diganti, yaitu apakah sampel individu dapat diambil beberapa kali untuk pelatihan sebuah *tree* (*replace* = TRUE) atau tidak (*replace* = FALSE). Jika *replace* = TRUE,

kemungkinan dua *tree* menerima sampel data yang sama akan berkurang. Hal ini dapat lebih lanjut mengurangi korelasi antar *tree* dan meningkatkan kualitas.

Type:	Logikal
Default:	2 (TRUE)
Sensivitas:	Sensitivitas dari <i>replace</i> seringkali cukup kecil. Namun, survei Probst et al. (2019) mencatat adanya bias yang berpotensi merugikan untuk <i>replace</i> = TRUE, jika variabel kategorikal dengan jumlah level variabel hadir.
Heuristik:	Karena bias yang disebutkan di atas, pilihan bisa bergantung pada varian kardinalitas dalam fitur data. Namun, rekomendasi yang dapat diukur tidak tersedia.
Rentang:	$\text{replace} \in \{\text{TRUE}, \text{FALSE}\}$.
Transformasi:	Trans_id
Batas:	lower = 1 (FALSE); upper = 2 (TRUE).
Keterbatasan:	Tidak ada.
Interaksi:	Satu interaksi yang jelas terjadi dengan <i>sample.fraction</i> . Kedua parameter mengontrol pilihan acak data pelatihan untuk setiap <i>tree</i> . Pengaturan (<i>replace</i> = TRUE $\wedge$ <i>sample.fraction</i> = 1) serta pengaturan (<i>replace</i> = FALSE $\wedge$ <i>sample.fraction</i> < 1) menyiratkan bahwa <i>tree</i> individu tidak akan melihat seluruh set data.

2.5.2.5 RF Hyperparameter **Respect.unordered.factors**

Parameter ini menentukan bagaimana pembagian variabel kategori ditangani. Ada tiga opsi: *ignore*, *order*, atau *partition*, yang akan dijelaskan singkat di bawah ini. Diskusi terperinci dapat ditemukan pada Wright dan König (2019). Standar yang juga digunakan oleh Breiman (2001) adalah *respect.unordered.factors* = *partition*. Dalam kasus ini, semua pembagian potensial dari variabel nominal, kategorikal dipertimbangkan. Ini menghasilkan model yang baik, tetapi jumlah pembagian yang dipertimbangkan dapat menyebabkan waktu yang tidak efektif. Alternatif yang lebih sederhana adalah *respect.unordered.factors* = *ignore*. Di sini, sifat kategorikal

variabel akan diabaikan. Sebaliknya, diasumsikan bahwa variabel tersebut berurutan, dan pembagian dipilih seperti halnya dengan variabel numerik. Ini mengurangi waktu komputasi tetapi dapat menurunkan kualitas model. Pilihan yang lebih baik seharusnya adalah `respect.unordered.factors = order`. Di sini, setiap variabel kategori diurutkan terlebih dahulu, tergantung pada frekuensi setiap level dalam kelas pertama dari dua kelas (dalam kasus klasifikasi) atau tergantung pada rata-rata nilai variabel dependen (regresi). Setelah pengurutan ini, variabel dianggap sebagai numerik. Ini memungkinkan waktu komputasi yang sama dengan `respect.unordered.factors = ignore` tetapi dengan kualitas model yang lebih baik. Ini mungkin tidak memungkinkan untuk klasifikasi dengan lebih dari dua kelas, karena kurangnya kriteria pengurutan yang jelas (Wright dan König 2019; Wright 2020).

Dalam kasus tertentu, `respect.unordered.factors = ignore` mungkin berhasil dalam praktiknya. Ini dapat terjadi ketika variabel sebenarnya bersifat nominal (tidak diketahui oleh analis).

Type:	Karakter, Skalar
Default:	1 (<i>ignore</i>)
Sensivitas:	Tidak diketahui
Heuristik:	Tidak ada
Rentang:	$respect.unordered.factors \in \{ignore, order, partition\}$. Parameter <i>respect.unordered.factors</i> juga dapat dipahami sebagai nilai biner. Maka, <i>TRUE</i> sesuai dengan <i>order</i> dan <i>FALSE</i> dengan <i>ignore</i> (Wright 2020).
Transformasi:	<code>trans_id</code>
Batas:	<i>lower = 1 (ignore); upper = 2 (order)</i> .
Keterbatasan:	Tidak ada
Interaksi:	Tidak diketahui

2.6 XGBoost

2.6.1 Description

Extreme Gradient Boosting (XGBoost) adalah algoritma berbasis *boosting* yang dirancang untuk efisiensi dan performa tinggi, yang sangat cocok digunakan dalam tugas klasifikasi dan regresi [40]. *XGBoost* bekerja dengan membangun model secara bertahap, di mana setiap *decision tree* baru bertujuan untuk memperbaiki kesalahan yang dibuat oleh *decision tree* sebelumnya, yang memungkinkan model untuk meningkatkan kualitas prediksi secara bertahap [41].

Keunggulan *XGBoost*:

- Efisiensi Komputasi Tinggi dengan Dukungan Paralelisasi

XGBoost dirancang untuk paralelisasi, yang memungkinkan pemrosesan data dalam jumlah besar dengan cepat. Dukungan paralelisasi ini menjadikannya sangat efisien dalam menangani data dalam skala besar dan sangat sesuai untuk aplikasi *real-time* yang membutuhkan kecepatan pengolahan tinggi [41].

- Dapat Menangani Dataset Besar dengan Cepat

XGBoost sangat efisien dalam menangani dataset besar yang memiliki banyak fitur. Algoritma ini mengoptimalkan proses pembelajaran model dengan memanfaatkan teknik *gradient boosting* yang dapat memproses dataset dalam waktu singkat tanpa mengorbankan akurasi [22].

- Mekanisme Regularisasi untuk Mengurangi Risiko *Overfitting*

XGBoost dilengkapi dengan mekanisme regularisasi untuk mengurangi risiko *overfitting*, yang sering kali menjadi masalah pada model yang kompleks. Regularisasi ini membantu algoritma untuk menjaga keseimbangan antara kesalahan pelatihan dan kesalahan pengujian, sehingga menghasilkan model yang lebih stabil dan dapat dipercaya [40].

- Efektif untuk Data dengan Banyak Atribut, Termasuk Atribut yang Tidak Relevan atau Berisik

XGBoost juga efektif dalam menangani dataset dengan banyak atribut, bahkan jika atribut tersebut tidak relevan atau mengandung *noise*. Hal ini memungkinkan *XGBoost* untuk tetap memberikan performa yang tinggi

meskipun data yang digunakan memiliki kualitas yang bervariasi dan banyaknya atribut yang tidak penting [22].

- Peningkatan Performa dengan Teknik Seleksi Fitur

Penelitian dalam [22] menunjukkan bahwa *XGBoost* memberikan performa yang sangat tinggi dalam mendeteksi *phishing*, terutama ketika dipadukan dengan teknik seleksi fitur yang memungkinkan untuk memilih atribut yang paling relevan dan mengurangi dimensi data, sehingga meningkatkan efisiensi dan akurasi model [22].

Secara keseluruhan, *XGBoost* adalah algoritma yang sangat efisien dan kuat, ideal untuk aplikasi yang melibatkan dataset besar dan kompleks. Dengan kemampuan paralelisasi, regularisasi, dan kemampuannya dalam menangani atribut berisik, serta kinerjanya yang semakin baik dengan teknik seleksi fitur, *XGBoost* telah menjadi algoritma unggulan dalam deteksi *phishing* dan aplikasi pembelajaran mesin lainnya [40][41][22].

2.6.2 Hyperparameter of XGBoost

Pada bagian ini merupakan beberapa *Hyperparameter* yang dalam algoritma *XGBoost* yaitu

2.6.2.1 XGBoost Hyperparameter nrounds

XGBoost Hyperparameter nrounds Parameter *nrounds* menentukan jumlah langkah boosting. Karena sebuah *tree* dibuat di setiap langkah boosting individual, *nrounds* juga mengendalikan jumlah *tree* yang digabungkan ke dalam *ensemble* secara keseluruhan. Makna praktisnya dapat dijelaskan sebagai berikut: nilai *nrounds* yang lebih besar berarti model yang lebih kompleks dan mungkin lebih akurat, tetapi juga menyebabkan waktu eksekusi yang lebih lama. Makna praktisnya sangat mirip dengan *num.trees* dalam *random forests*. Berbeda dengan *num.trees*, *overfitting* menjadi risiko pada nilai yang sangat besar, tergantung pada parameter lain seperti *eta*, *lambda*, *alpha*. Sebagai contoh, hasil empiris Friedman (2001) menunjukkan bahwa dengan *eta* yang rendah, bahkan nilai *nrounds* yang tinggi tidak menyebabkan *overfitting*.

<i>Type:</i>	<i>Integer, scalar</i>
<i>Default:</i>	<i>0</i>
<i>Sensitivity/robustness:</i>	Serupa dengan parameter <code>num.trees</code> pada random forests, <code>nrounds</code> juga memiliki sensitivitas yang lebih tinggi, terutama pada nilai yang rendah.
<i>Heuristics:</i>	<i>Heuristics cannot be derived from the literature. Often values of several hundred to several thousand trees are set as the upper limit (Brownlee 2018).</i>
<i>Range:</i>	$\in [1, \infty[$. Hanya nilai bilangan bulat yang valid.
<i>Transformation:</i>	<i>trans_2pow_round.</i>
<i>Bounds:</i>	<i>lower = 0; upper = 11</i>
<i>Constraints:</i>	<i>none</i>
<i>Interactions:</i>	<i>There is a connection between the hyperparameters <code>beta</code>, <code>nrounds</code>, and <code>subsample</code>.</i>

2.6.2.2 XGBoost Hyperparameter `eta`

Parameter `eta` adalah laju pembelajaran dan juga disebut parameter “penyusutan”. Parameter ini mengontrol penurunan bobot pada setiap langkah boosting. Ini memiliki makna praktis menurunkan bobot membantu untuk mengurangi pengaruh *tree* tunggal pada ansambel. Ini juga dapat menghindari overfitting.

<i>Type:</i>	<i>double, Scalar</i>
<i>Default:</i>	<i>Log 2 {0.3}</i>
<i>Sensitivity:</i>	Hasil empiris menunjukkan bahwa XGBoost lebih sensitif terhadap <code>eta</code> ketika <code>eta</code> besar (Friedman 2001). Secara umum, nilai yang lebih kecil lebih baik. Dalam sebuah studi empiris, Probst et al. (2018) menggambarkan <code>eta</code> sebagai parameter dengan kemampuan penyesuaian yang relatif tinggi.
<i>Heuristics:</i>	Sebuah heuristik sulit untuk diformulasikan karena ketergantungannya pada parameter lain dan situasi data,

tetapi Hastie et al. (2017) merekomendasikan strategi terbaik tampaknya adalah menetapkan η sangat kecil ($\eta < 0,1$) dan kemudian memilih n rounds melalui penghentian dini. Hal ini dapat menyebabkan waktu berjalan yang lebih lama karena n rounds yang besar. Brownlee (2018) menyebutkan sebuah heuristik, yang menjelaskan rentang pencarian tergantung pada n rounds. $\eta \in [0,1]$. Menggunakan skala logaritmik tampaknya masuk akal, misalnya, $2^{-10}, \dots, 2^{-20}$), seperti yang digunakan dalam studi oleh Probst et al. (2018) atau Sigrist (2020), karena nilai yang mendekati nol sering menunjukkan hasil yang baik.

Range:

Transformation:

Bounds:

Constraints:

Interactions:

Trans_2pow

Lower = -10; upper = 0

none

Ada hubungan antara η dan n rounds: Jika salah satu dari kedua parameter meningkat, yang lain sebaiknya dikurangi jika kesalahan tetap sama (Friedman 2001; Probst et al. 2019a). Hal ini juga ditunjukkan oleh Hastie et al. (2017)

Nilai η yang lebih kecil mengarah ke nilai n rounds yang lebih besar untuk risiko pelatihan yang sama, sehingga terdapat trade-off antara keduanya.

Selain itu, Hastie et al. (2017) juga menunjukkan korelasi dengan parameter subsample: Dalam sebuah studi empiris, subsample=1 dan $\eta=1$ menunjukkan hasil yang secara signifikan lebih buruk dibandingkan subsample=0,5 dan $\eta=0,1$. Jika subsample=0,5 dan $\eta=1$, hasilnya bahkan lebih buruk daripada $\eta=1$ dan subsample=1. Dalam kasus terbaik (subsample=0,5 dan

eta=0,1), bagaimanapun, nilai nrounds yang lebih besar diperlukan untuk mencapai hasil optimal.

2.6.2.3 XGBoost Hyperparameter lambda

Parameter lambda digunakan untuk regularisasi model. Parameter ini memengaruhi kompleksitas model (mirip dengan parameter dengan nama yang sama dalam EN). Signifikansi praktisnya dapat dijelaskan sebagai berikut: sebagai parameter regularisasi, lambda membantu mencegah *overfitting* (Chen dan Guestrin 2016). Dengan nilai yang lebih besar, model yang lebih halus atau lebih sederhana diharapkan.

<i>Type:</i>	<i>Double, scalar</i>
<i>Default:</i>	0
<i>Sensitivity:</i>	<i>Not known</i>
<i>Heuristics:</i>	<i>None known</i>
<i>Range:</i>	lambda $\in [0, \infty[$. Skala logaritmik tampaknya berguna, misalnya, 2-10,...,210, seperti yang digunakan dalam studi oleh Probst et al. (2019a) untuk mencakup berbagai nilai yang sangat kecil dan sangat besar.
<i>Transformation:</i>	Trans_2pow
<i>Bounds:</i>	lower = -10; upper = 10.
<i>Constraints:</i>	<i>none</i>
<i>Interactions:</i>	Karena baik lambda maupun alpha mengontrol regularisasi model, interaksi kemungkinan besar terjadi.

2.6.2.4 XGBoost Hyperparameter alpha

Para penulis paket R xgboost, Chen dan Guestrin (2016), tidak menyebutkan parameter ini. Dokumentasi dari implementasi referensi juga tidak memberikan informasi rinci tentang alpha. Karena deskripsinya sebagai parameter untuk regularisasi dari bobot (Chen et al. 2020), penggunaan yang sangat mirip dengan parameter yang sama namanya pada elastic net dapat diasumsikan. Makna

praktisnya dapat dijelaskan sebagai berikut: mirip dengan λ , α juga berfungsi sebagai parameter regularisasi.

<i>Type:</i>	<i>Double, scalar</i>
<i>Default:</i>	-10
<i>Sensitivity:</i>	<i>unknown</i>
<i>Heuristics:</i>	<i>No heuristics are known.</i>
<i>Range:</i>	$\alpha \in [0, \infty[$. Skala logaritmik tampaknya berguna, misalnya, , seperti yang digunakan dalam studi oleh Probst et al. (2019) untuk mencakup rentang nilai yang sangat kecil hingga sangat besar.
<i>Transformation:</i>	Trans_2pow
<i>Bounds:</i>	<i>lower</i> = -10; <i>upper</i> = 10.
<i>Constraints:</i>	<i>none</i>
<i>Interactions:</i>	Karena baik λ maupun α mengontrol regularisasi model, interaksi kemungkinan besar terjadi.

2.6.2.5 XGBoost Hyperparameter subsample

Dalam setiap langkah *boosting*, *tree* baru yang akan dibuat biasanya hanya dilatih pada sebagian dari seluruh set data, mirip dengan *random forest*. Parameter subsample menentukan bagian dari data yang dipilih secara acak pada setiap iterasi. Signifikansi praktisnya dapat dijelaskan sebagai berikut: efek yang jelas dari nilai subsample yang kecil adalah waktu pelatihan yang lebih singkat untuk *tree* individu, yang sebanding dengan subsample.

<i>Type:</i>	<i>Double, scalar</i>
<i>Default:</i>	1
<i>Sensitivity:</i>	Studi oleh Friedman (2002) menunjukkan sensitivitas yang tinggi terhadap nilai yang sangat kecil atau besar dari subsample. Namun, dalam rentang nilai subsample yang relatif besar (sekitar 0,3 hingga 0,6), hampir tidak ada perbedaan yang diamati dalam kualitas model.

<i>Heuristics:</i>	Hastie et al. (2017) menyarankan subsample = 0,5 sebagai nilai awal yang baik, tetapi menunjukkan bahwa nilai ini dapat dikurangi jika nrounds meningkat. Dengan banyak <i>tree</i> (nround besar) sudah cukup jika setiap <i>tree</i> individu melihat bagian data yang lebih kecil, karena data yang tidak terlihat lebih mungkin diperhitungkan di <i>tree</i> lain.
<i>Range:</i>	subsample $\in]0,1]$. Berdasarkan hasil empiris Friedman (2002); Hastie et al. (2017), skala logaritmik tidak disarankan.
<i>Transformation:</i>	Trans_id
<i>Bounds:</i>	<i>lower</i> = 0.1; <i>upper</i> = 1.
<i>Constraints:</i>	<i>none</i>
<i>Interactions:</i>	Ada hubungan antara eta, nrounds, dan subsample.

2.6.2.4 XGBoost Hyperparameter colsample_bytree

Parameter colsample_bytree memiliki kemiripan dengan parameter mtry pada hutan acak. Di sini juga, sejumlah fitur acak dipilih untuk pemisahan sebuah *tree*. Namun, di *XGBoost*, pilihan ini dibuat hanya sekali untuk setiap *tree* yang dibuat, bukan untuk setiap pemisahan (pengembang xgboost 2020). Di sini colsample_bytree adalah faktor relatif. Jumlah fitur yang dipilih adalah colsample_bytree $\times$ n. Makna praktisnya mirip dengan mtry: colsample_bytree memungkinkan *tree* dalam ansambel memiliki keberagaman yang lebih besar. Waktu eksekusi juga berkurang, karena Jumlah pemisahan yang lebih kecil harus diperiksa setiap kali (jika colsample_bytree <1).

<i>Type:</i>	<i>Double, scalar</i>
<i>Default:</i>	1
<i>Sensitivity:</i>	Studi empiris oleh Probst et al. (2019a) menunjukkan bahwa model ini sangat sensitif terhadap perubahan nilai colsample_bytree yang mendekati 1. Namun, sensitivitas ini menurun di sekitar nilai yang lebih sesuai.

<i>Heuristics:</i>	<i>None known</i>
<i>Range:</i>	<i>colsample_bytree $\in]0,1]$. Brownlee (2018) mentions search ranges such as $\text{colsample_bytree} = 0.4, 0.6, 0.8, 1$, but mostly works with $\text{colsample_bytree} = 0.1, 0.2, \dots, 1$.</i>
<i>Transformation:</i>	<i>Trans_id</i>
<i>Bounds:</i>	<i>lower = $1/n\text{Features}$; upper = 1.</i>
<i>Constraints:</i>	<i>none</i>
<i>Interactions:</i>	<i>None known.</i>

2.6.2.7 XGBoost Hyperparameter gamma

Parameter dari satu *tree* keputusan ini sangat mirip dengan parameter *cp*: Seperti *cp*, *gamma* mengontrol jumlah pemisahan pada *tree* dengan mengasumsikan peningkatan minimal untuk setiap pemisahan. Menurut dokumentasinya. Pengurangan kerugian minimum yang diperlukan untuk membuat partisi lebih lanjut pada simpul daun dari *tree*. Semakin besar nilainya, semakin konservatif algoritma tersebut. Perbedaan utama antara *cp* dan *gamma* adalah definisi *cp* sebagai faktor relatif, sedangkan *gamma* didefinisikan sebagai nilai absolut. Ini juga berarti bahwa rentangnya berbeda.

<i>Default:</i>	<i>-10</i>
<i>Range</i>	<i>$\text{gamma} \in [0, \infty[$. Alogarithmic scale seems to make sense, e.g., $2^{-10}, \dots, 2^{10}$, as, e.g., in the study by Thomas et al. (2018) to cover a wide range of very small and very large values</i>
<i>Transformation:</i>	<i>Trans_2pow</i>
<i>Bounds:</i>	<i>lower = -10; upper = 10.</i>

2.6.2.8 XGBoost Hyperparameter maxdepth

Parameter dari satu *tree* keputusan ini sudah dikenal sebagai *maxdepth*.

<i>Default:</i>	6
<i>Heuristics/ Sensitivity:</i>	Meskipun dalam banyak aplikasi $J = 2$ akan kurang memadai, tidak mungkin $J > 10$ akan diperlukan. Pengalaman sejauh ini menunjukkan bahwa $4 \leq J \leq 8$ bekerja dengan baik dalam konteks boosting, dengan hasil yang cukup tidak sensitif terhadap pilihan tertentu di rentang ini.
<i>Transformation:</i>	Trans_id
<i>Bounds:</i>	$Lower = 1 ; upper = 15.$

2.6.2.9 XGBoost Hyperparameter min_child_weight

Seperti γ dan max_depth , min_child_weight membatasi jumlah pembelahan setiap *tree*. Dalam hal min_child_weight , batasan ini ditentukan menggunakan matriks Hessian dari fungsi kerugian (jumlahnya di semua pengamatan di setiap node terminal baru) (Chen et al., 2020; Sigrist, 2020). Dalam eksperimen oleh Sigrist (2020), parameter ini ternyata relatif sulit untuk disesuaikan: hasilnya menunjukkan bahwa penyetelan dengan min_child_weight memberikan hasil yang lebih buruk dibandingkan penyetelan dengan parameter serupa (batasan jumlah sampel per lempeng).

<i>Type:</i>	Double, scalar
<i>Default:</i>	0
<i>Sensitivity:</i>	unknown
<i>Heuristics:</i>	None known
<i>Range:</i>	$min_child_weight \in [0, \infty[$. Skala logaritmik tampaknya masuk akal, misalnya, 2–10,...,210, seperti yang digunakan dalam studi oleh Probst et al. (2019a) untuk mencakup rentang nilai yang sangat kecil hingga sangat besar.

Transformation: Trans_2pow

Bounds: lower = 0; upper = 7

Constraints: none

Interactions: Interaksi dengan parameter seperti gamma dan maxdepth kemungkinan terjadi, karena ketiga parameter tersebut memengaruhi kompleksitas *tree* individual dalam *ansamble*.

2.7 Rule-Based Filtering

Rule-based filtering adalah pendekatan berbasis aturan logis yang digunakan untuk menyaring data sebelum diproses oleh algoritma *Machine learning* [42]. Aturan ini biasanya mencakup panjang URL, keberadaan simbol mencurigakan (misalnya “@” atau “//”), jumlah subdomain, serta umur domain [42]. Karakteristik atau aturan yang sering digunakan dalam memfilter menggunakan *Rule Based Filtering* dalam penelitian terdahulu dapat dilihat dalam Tabel 2. 3 berikut:

Tabel 2. 3 Characteristics of Rule-based Filtering

No	Kategori	Rule	Deskripsi	Contoh
1.	<i>Search Engine-based</i>	URL Tidak Terindeks	URL tidak muncul di hasil pencarian Google, Bing, atau Yahoo, maka kemungkinan besar situs baru dibuat untuk <i>phishing</i> .	URL seperti http://secure-login-update.com
2.	<i>Search Engine-based</i>	Domain Tidak Terindeks	Domain utama tidak muncul di indeks mesin pencari, maka berpotensi <i>phishing</i> .	Domain login-verifikasi-bankid.net
3.	<i>Keyword - based</i>	<i>Keyword</i> Mencurigakan	URL mengandung kata seperti “ <i>verify</i> ”, “ <i>update</i> ”, “ <i>account</i> ”, “ <i>secure</i> ”, dll.	http://paypal-secure-update.com

No	Kategori	Rule	Deskripsi	Contoh
4.	<i>Obfuscation-based</i>	URL IP-based	Menggunakan IP address langsung sebagai URL.	http://192.168.0.10/login.php <i>phisher</i> sering memakai IP agar tidak mudah dilacak.
5.	<i>Obfuscation-based</i>	Karakter Mencurigakan	URL mengandung karakter seperti @, -, _ , atau <i>port</i> tidak standar.	http://login-paypal.com@fake.ru:8080
6.	<i>Obfuscation-based</i>	Panjang URL/Host	URL terlalu panjang atau banyak titik (dot).	http://secure-login.bank.update.account.verify.user.info.com mengandung banyak titik dan panjang.
7.	<i>Blacklist-based</i>	URL Masuk Daftar Hitam	URL tercantum dalam Google Safe Browsing atau PhishTank blacklist.	URL http://freegift-cardlogin.com ditemukan di PhishTank sebagai <i>phishing</i> .
8.	<i>Reputation-based</i>	Domain/IP Reputasi Buruk	Domain atau IP pernah dilaporkan <i>hosting phishing</i> .	Domain malwarehub.ru muncul dalam daftar reputasi buruk dari StopBadware.
9.	<i>Content-based</i>	<i>Form Password</i> Tidak Aman	Halaman memiliki <i>form password</i> yang tidak menggunakan HTTPS atau metode GET.	Situs http://bank-login.com/login memiliki <i>field password</i> namun tanpa SSL (HTTP).
10.	<i>Content-based</i>	<i>Form Password</i> ke Domain Eksternal	<i>Form login</i> mengirim data ke domain berbeda.	Situs http://bank-login.com mengirim <i>form</i> ke http://data-collector.net , indikasi <i>phishing</i> .
11.	<i>Content-based</i>	Meta Refresh ke	META tag melakukan <i>redirect</i> ke domain luar.	Tag <code><meta http-equiv="refresh"</code>

No	Kategori	Rule	Deskripsi	Contoh
		Domain Eksternal		content="2;url=http://phishingsite.com">.
12.	<i>Content-based</i>	<i>Redirect Server + Password Field</i>	Server melakukan <i>redirect</i> dan ada <i>password field</i> di halaman.	http://secure-update.com mengalihkan pengguna ke http://verify-login.net dengan form login.
13.	<i>Content-based</i>	IFrame ke Domain <i>Blacklist</i>	Halaman memiliki iframe yang memuat konten dari domain jahat/ <i>blacklist</i> .	<iframe src="http://phishform.com/login.html"> memuat form <i>phishing</i> dari domain luar.
14.	<i>Content-based</i>	Eksternal Link Lebih Banyak	Halaman memiliki lebih banyak link eksternal daripada internal, terutama dengan <i>password field</i> .	Halaman login palsu dengan banyak tautan eksternal menuju “ <i>bank verification</i> ”.
15.	<i>Content-based</i>	HTML Buruk + Password Field	Markup HTML rusak atau tidak sesuai standar dan memiliki <i>field password</i> .	Situs <i>phishing</i> hasil “ <i>copy-paste</i> ” dari halaman asli, tampilan rusak dengan form <i>login</i> palsu.

Sumber: [43]

Tabel 2. 4 menampilkan contoh penerapan metode rule-based dalam mendeteksi phishing. Setiap aturan (rule) dibuat berdasarkan fitur-fitur tertentu, seperti sertifikat SSL, otoritas penerbit sertifikat, serta keberadaan kata kunci mencurigakan, yang dapat menunjukkan indikasi phishing.

Tabel 2. 4 Contoh Rule-Based dalam Deteksi Phishing

No	Fitur	Deskripsi	Contoh
1.	SSL Certificate (HTTPS)	Memeriksa apakah link dalam email menggunakan HTTPS dengan SSL aktif. Email dengan link HTTP (tanpa SSL) dianggap mencurigakan.	Link: http://secure-login.com (tidak aman, tanpa SSL) terdeteksi <i>phishing</i> .
2.	Certificate Authority (CA)	Mengecek penerbit sertifikat SSL. Jika diterbitkan oleh otoritas tidak terpercaya (<i>self-signed</i> / palsu), maka email mencurigakan.	Sertifikat dari <i>UnknownCA</i> bukan dari <i>GoDaddy</i> atau <i>Comodo</i> .
3.	Blacklist Keywords	Mendeteksi kata kunci mencurigakan di isi email seperti “Click Now”, “Verify Now”, “Update Account”, dll.	Isi email: ‘Your account will expire in 24 hours! Verify now!’ terdeteksi <i>phishing</i> .
4.	Redirection URL	Mengecek apakah link melakukan redirect tersembunyi ke domain lain (<i>hidden proxy server</i>).	Klik link http://bank-login.com ternyata diarahkan ke http://phishsite.net .
5.	Hiding Links / Shortened URLs	Mendeteksi tautan yang disamarkan dengan layanan seperti goo.gl , bit.ly , atau <i>hyperlink</i> teks HTML palsu.	Link teks 'Klik di sini untuk masuk ke Gmail' tapi mengarah ke http://login-update.com .
6.	Clear IP Address in URL	Memeriksa apakah link berisi alamat IP langsung (bukan nama domain). Ini umum pada situs <i>phishing</i> sementara.	Link: https://50.10.125.26/login.php IP-based URL, mencurigakan.
7.	Website Traffic (Alexa Rank)	Mengevaluasi popularitas situs berdasarkan trafik (Alexa rank ≤ 150.000 biasanya aman).	Situs bankofamerica.com memiliki trafik tinggi, sementara bofa-login-secure.info tidak terdaftar.

No	Fitur	Deskripsi	Contoh
8.	<i>Webpage Age</i> (<i>Domain Age</i>)	Memeriksa umur domain. Domain baru (<1 tahun) biasanya digunakan untuk <i>phishing</i> sementara.	secure-account-verif.com baru dibuat 2 minggu terdeteksi mencurigakan.
9.	<i>Sender Email</i> <i>Address</i>	Mengecek konsistensi antara domain pengirim dan isi email.	Email dari support@dropbox.com.fake.co mengklaim dari Dropbox terdeteksi <i>phishing</i> .
10.	<i>Attached File</i> <i>Extension</i>	Mengecek ekstensi lampiran seperti .exe, .dll, atau .bat yang sering membawa malware.	Email dengan lampiran <i>invoice.exe</i> atau <i>update.dll</i> terdeteksi <i>phishing</i> .

Sumber: [44]

Fungsi utama *rule-based filtering* adalah:

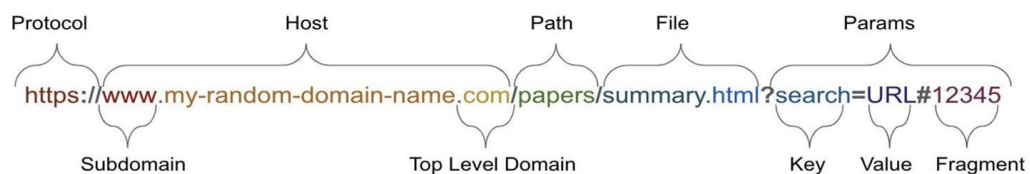
- Sebagai filter awal, untuk mendeteksi pola *phishing* klasik sehingga hanya kasus yang lebih kompleks diproses oleh algoritma *Machine learning*.
- Meningkatkan efisiensi komputasi, karena model *machine learning* tidak perlu memproses data yang sudah jelas *phishing*.
- Mengurangi kesalahan klasifikasi (*false positive* dan *false negative*), sehingga hasil prediksi lebih akurat.

Menurut [21], *rule-based filtering* terbukti mampu mengurangi *noise* data dan membantu meningkatkan performa sistem deteksi *phishing*, terutama ketika dikombinasikan dengan metode *Machine learning*.

2.7.1 Struktur URL

URL (*Uniform Resource Locator*) merupakan alamat unik yang digunakan untuk mengidentifikasi sumber daya di internet. Dalam konteks deteksi *phishing*, analisis struktur URL sangat penting karena sebagian besar serangan *phishing* dapat dikenali dari pola atau karakteristik tertentu pada URL [45].

Struktur URL terdiri dari beberapa bagian penting, yaitu *domain*, *path*, dan protokol. Domain ini seperti alamat yang memberi tahu tujuan penggunaan URL, apakah asli, dan dari negara mana asalnya. Selain itu, URL juga memiliki komponen lain seperti kata-kata, kategori, pengenalan, tanggal, atau singkatan khusus yang menunjukkan bagaimana informasi diatur atau bagaimana aplikasi menampilkan konten tersebut [45]. Karena fungsinya yang luas ini, URL sangat penting dalam berbagai aplikasi analisis data internet, terutama yang memerlukan pemeriksaan cepat seperti deteksi situs *phishing* atau malware.



Gambar 2. 2 Struktur URL

Sumber: [45]

Berikut ini adalah Tabel 2. 5 komponen dan data Fitur dari URL.

Tabel 2. 5 Komponen URL dan Data Fitur

No	Komponen	Subkomponen	Jenis Data Fitur
1.	<i>Protocol</i>		Menunjukkan jenis konten (seperti http atau https) dan tingkat keamanan.
2.	<i>Domain</i>	<i>Subdomain Top-Level Domain</i>	- Nama Utama situs yang menunjukkan Tujuan Keseluruhan - Menjelaskan Fungsi Khusus atau Pengkategorian tertentu dari konten di situs (misalnya, blog. atau support.). Menggambarkan Tujuan Bisnis/Layanan (.com, .org, .edu), Asal Geografis (.id, .uk), atau Kredibilitas situs.

No	Komponen	Subkomponen	Jenis Data Fitur
3.	<i>Path</i>		Menjelaskan Bagaimana Konten Disusun dan diorganisasikan di situs, mirip Struktur Folder di komputer.
4.	<i>File</i>		Menunjukkan Aturan Penamaan File, Maksud Pembuat File tersebut, dan Jenis File (misalnya: dokumen, gambar, atau program yang bisa dijalankan).
5.	<i>Parameter</i>	<i>Keys Values Fragment</i>	Berisi detail untuk Personalisasi pengguna (misalnya, bahasa), Elemen Keamanan, dan Informasi Pelacakan aktivitas pengguna.

Sumber: [45]

2.8 Evaluasi

Tahap evaluasi model bertujuan untuk menilai kinerja sistem deteksi *website phishing* yang telah dibangun berdasarkan hasil pelatihan dan pengujian. Evaluasi dilakukan guna mengukur sejauh mana model mampu membedakan antara *website phishing* dan *website benign* secara akurat [46]. Proses evaluasi ini dilakukan dengan menerapkan sejumlah metrik evaluasi yang relevan dalam konteks klasifikasi biner. Tahapan ini juga mencakup perbandingan antara performa model individual (*Random Forest* dan *XGBoost*) dengan model gabungan (*Rule-Based Filtering + Stacking Ensemble*). Dengan demikian, hasil evaluasi dapat menunjukkan efektivitas pendekatan kombinasi. Berikut merupakan Tabel 2. 6 *confusion matrix*.

Tabel 2. 6 *Confusion Matrix*

		Predicted		
		Yes	No	Total
Actual	Yes	TP	FN	P
	No	FP	TN	N
	Total	P	N	P+N

P (*positive*) adalah jumlah tupel positif, N (*negative*) adalah jumlah tupel negatif, TP (*True Positive*) adalah jumlah tupel positif yang diberi label dengan benar oleh pengklasifikasi, TN (*True Negative*) adalah jumlah tupel negatif yang diberi label dengan benar oleh pengklasifikasi, FP (*False Positive*) adalah jumlah tupel negatif yang dilabelkan sebagai positif, dan FN (*False Negative*) adalah jumlah tupel positif yang dilabelkan sebagai negatif. Berikut adalah rumus yang digunakan untuk menghitung nilai-nilainya [46].

1. Akurasi (*Accuracy*)

Akurasi mengukur seberapa banyak prediksi model yang benar dibandingkan dengan seluruh data uji. Metrik ini digunakan untuk mengetahui tingkat ketepatan model secara keseluruhan [47].

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

[46]

2. Presisi (*Precision*)

Presisi menunjukkan sejauh mana prediksi positif yang diberikan model benar-benar akurat [47]. Nilai presisi yang tinggi menunjukkan model jarang salah mendeteksi *website* aman sebagai *phishing*.

$$Precision = \frac{TP}{TP + FP}$$

[46]

3. Recall (*Sensitivity*)

Recall menunjukkan kemampuan model dalam menemukan seluruh data positif yang sebenarnya. Nilai recall tinggi berarti model mampu mendeteksi sebagian besar situs *phishing* yang ada.

$$Recall = \frac{TP}{TP + FN}$$

[46]

4. *F1-Score*

F1-score merupakan rata-rata harmonik antara presisi dan recall, digunakan untuk menilai keseimbangan antara keduanya, terutama ketika data tidak seimbang.

$$F1\ Score = \frac{2 \times Recall \times Precision}{Recall + Precision}$$

[46]

5. *Area Under Curve (AUC)*

AUC diukur berdasarkan *Receiver Operating Characteristic (ROC) curve*, yang menggambarkan hubungan antara *True Positive Rate* dan *False Positive Rate*. Nilai AUC yang mendekati 1 menunjukkan model memiliki performa yang baik dalam membedakan dua kelas.

2.9 Penelitian Terdahulu

Tabel 2. 7 Penelitian Terdahulu

No	Research Title		Author	Year	Method	Results	Disadvantages
1	PhishGuard: A Multi-Layered Ensemble Model for Optimal <i>Phishing Website</i> Detection		Mourtaji et al.	2021	Ensemble (<i>Random Forest</i> + <i>XGBoost</i>)	Kombinasi dua algoritma meningkatkan akurasi dibandingkan model tunggal	Belum mengintegrasikan rule-based filtering sebagai lapisan awal.
2	Hybrid Rule-Based Solution for <i>Phishing</i> URL Detection Using CNN		Patgiri et al.	2021	Rule-based + CNN	Rule-based filtering efektif menyaring URL <i>phishing</i> klasik sehingga efisiensi meningkat	Terbatas pada fitur URL sederhana, tidak menguji algoritma ensemble
3	Enhancing <i>Phishing</i> Detection: Integrating <i>XGBoost</i> with Feature Selection Technique		Zhang et al.	2022	<i>XGBoost</i> + Feature Selection	Seleksi fitur meningkatkan akurasi <i>XGBoost</i> dan mengurangi overfitting	Tidak menguji kombinasi dengan algoritma lain (misalnya <i>Random Forest</i>)
4	A High-Accuracy <i>Phishing Website</i> Detection Method Based on <i>Machine learning</i>		Jain & Gupta	2023	Perbandingan beberapa ML (RF, XGB, LR, KNN)	<i>XGBoost</i> dan <i>Random Forest</i> terbukti paling unggul dalam akurasi	Dataset hanya berasal dari satu sumber, keterbatasan generalisasi
5	<i>Phishing</i> Detection Using <i>Machine learning</i> Algorithm		Vishesh Bharuka, et al.	2024	Membandingkan algoritma ML (Decision Tree, RF, Multilayer Perceptron, <i>XGBoost</i> , ANN, SVM)	<i>Random Forest</i> dan <i>XGBoost</i> mengungguli algoritma lain dalam metrik kinerja.	Efektivitas bergantung pada pilihan fitur, dataset, dan algoritma.

No	Research Title		Author	Year	Method	Results	Disadvantages
6	URL Feature Analysis for Effective <i>Phishing</i> Detection using <i>Machine learning</i>		Subashini, Dr. S. et al.	2025	Membandingkan algoritma ML (Decision Tree, RF, SVM, <i>XGBoost</i>) menggunakan 17 fitur URL (domain, HTML/JS).	Klasifikasi menggunakan <i>XGBoost</i> mencapai akurasi tertinggi.	Akurasi bergantung pada dataset hanya menganalisis fitur URL (bukan konten halaman).
7	Deteksi <i>Website Phishing</i> Menggunakan Teknik <i>Machine learning</i>		Lukito, W. et al.	2025	Membandingkan algoritma ML (<i>XGBoost</i> , <i>Decision Tree</i> , dan RF) dengan <i>hyperparameter tuning</i>	<i>XGBoost</i> dengan Grid Search tuning memberikan akurasi terbaik 96.14%.	Masih ada ruang untuk peningkatan akurasi. Perlu meningkatkan jumlah dan keragaman dataset.

BAB III

ANALISIS

Bab ini membahas analisis pengumpulan data, pembersihan data, eksplorasi data, pra-pemrosesan data, analisis model klasifikasi, dan analisis evaluasi penelitian. Analisis ini dilakukan agar mengetahui struktur data serta metode yang menjadi acuan dalam tahap perancangan.

3.1 Analisis Dataset

Subbab ini membahas dataset yang digunakan dalam penelitian. Dataset berbentuk tabular ini berasal dari beberapa sumber pada platform Kaggle, dan telah dilengkapi dengan berbagai atribut serta label yang membedakan antara URL *phishing* dan *non-phishing*.

3.1.1 Data Preparation

Subbab ini membahas tahapan persiapan data sebelum digunakan dalam proses analisis dan pemodelan. Kegiatan utama pada tahap ini meliputi pengumpulan data URL *phishing* dan *non-phishing*, identifikasi serta pemilihan atribut yang relevan, proses ekstraksi fitur dari URL, serta analisis awal terhadap hubungan antar atribut. Seluruh tahap ini dilakukan untuk memastikan bahwa data berada dalam kondisi yang siap digunakan pada tahap pemodelan selanjutnya.

3.1.1.1 Pengumpulan Data

Pada tahap ini dijelaskan dataset yang digunakan dalam penelitian memiliki karakteristik pada Tabel 3. 1 sebagai berikut:

Tabel 3. 1 Karakteristik Dataset Phishing

No	Karakteristik	Deskripsi
1.	Nama Dataset	dataset_phishing
2.	Sumber	https://www.kaggle.com/datasets/hafisafrizal/phising-dataset-url?select=dataset_phishing.csv
3.	Jumlah Sampel	11.430 baris data
4.	Jumlah Fitur	89 fitur (88 fitur independen + 1 label target)

No	Karakteristik	Deskripsi
5.	Label Target	<i>status</i> (0 = Non-phishing, 1 = Phishing)

3.1.1.2 Atribut Dataset

Pada tahap ini dilakukan analisis mendalam terhadap atribut–atribut yang terdapat pada dataset yang digunakan dalam penelitian. Analisis ini bertujuan untuk memahami karakteristik setiap fitur secara menyeluruh. Pemahaman yang tepat mengenai deskripsi dan makna dari setiap atribut menjadi landasan penting sebelum memasuki tahapan *preprocessing data*, pemilihan fitur, maupun pembangunan model *machine learning*. Tabel 3. 2 ini adalah penjelasan dari atribut dataset pertama “dataset_phishing” yang berisi 11.431 URL, 89 *features*, serta non-phishing 5715 dan phishing 5715.

Tabel 3. 2 Atribut pada Dataset dataset_phishing

No	Feature	Deskripsi
1.	url	<i>String</i> teks mentah yang berisi alamat lengkap suatu situs web (URL) yang dianalisis. URL ini menjadi data dasar dari mana seluruh fitur lain diturunkan, karena setiap karakter, struktur, dan elemennya diproses untuk menghasilkan nilai pada fitur-fitur berikutnya.
2.	length_url	Menghitung total jumlah karakter (huruf, angka, simbol) dalam <i>string</i> URL.
3.	length_hostname	Menghitung panjang karakter hanya pada bagian nama host (domain utama dan <i>subdomain</i>), tidak termasuk protokol (<i>http://</i>) atau <i>path</i> (<i>/folder/file</i>).
4.	ip	Mendeteksi apakah URL menggunakan alamat IP (<i>numerik</i>) sebagai pengganti nama domain teks.
5.	nb_dots	Jumlah karakter '.'
6.	nb_hyphens	Jumlah karakter '-'
7.	nb_at	Jumlah karakter '@'
8.	nb_qm	Jumlah karakter '?'
9.	nb_and	Jumlah karakter '&'
10.	nb_or	Jumlah karakter ' '

No	Feature	Deskripsi
11.	nb_eq	Jumlah karakter '='
12.	nb_underscore	Jumlah karakter '_'
13.	nb_tilde	Jumlah karakter '~'
14.	nb_percent	Jumlah karakter '%'
15.	nb_slash	Jumlah karakter '/'
16.	nb_star	Jumlah karakter '*'
17.	nb_colon	Jumlah karakter ':'
18.	nb_comma	Jumlah karakter ','
19.	nb_semicolumn	Jumlah karakter ';'.
20.	nb_dollar	Jumlah karakter '\$'
21.	nb_space	Jumlah spasi atau %20 dalam URL
22.	nb_www	Menghitung kemunculan kata "www"
23.	nb_com	Menghitung kemunculan kata "com".
24.	nb_dslash	Menghitung garis miring ganda (//) pada URL, selain pada protokol http://
25.	http_in_path	Mencari kata "http" yang muncul di bagian belakang URL (<i>path</i>), bukan di depan sebagai protokol.
26.	https_token	Memeriksa apakah ada token "https" di dalam domain atau <i>path</i> .
27.	ratio_digits_url	Persentase karakter angka dalam URL. $ratio_digits_url = \frac{\text{Jumlah digit di seluruh URL}}{\text{Total panjang URL}}$
28.	ratio_digits_host	Persentase angka dalam hostname saja. $ratio_digits_host = \frac{\text{Jumlah digit pada hostname}}{\text{Total panjang hostname}}$
29.	punycode	Deteksi format Punycode (biasanya diawali xn--). Ini cara menulis domain berkarakter asing menggunakan huruf latin.
30.	port	Mendeteksi keberadaan nomor <i>port</i> eksplisit (angka setelah titik dua di <i>host</i>). Angka ini menunjukkan <i>port</i> eksplisit yang ditulis langsung di URL: 80, 443, 21, 8080, 3000, 5000.
31.	tld_in_path	Mendeteksi apakah ekstensi domain (seperti .com, .net, .org) muncul di bagian <i>path</i> (file/folder).

No	Feature	Deskripsi
32.	tld_in_subdomain	Mendeteksi apakah TLD muncul di bagian <i>subdomain</i> (di depan domain utama).
33.	abnormal_subdomain	Fitur biner yang menandai jika pola subdomain tidak sesuai standar umum (misal www bukan di depan).
34.	nb_subdomains	Menghitung jumlah <i>level subdomain</i> yang dipisahkan titik.
35.	prefix_suffix	Menandai keberadaan tanda strip (-) di nama domain utama (bukan <i>path</i>).
36.	random_domain	Apakah domain terlihat acak berdasarkan perhitungan karakter.
37.	shortening_service	Mencocokkan domain dengan daftar layanan penyingkat URL (seperti bit.ly, goo.gl).
38.	path_extension	Mendeteksi apakah URL diakhiri dengan ekstensi file umum (.html, .php, .txt).
39.	nb_redirection	Jumlah pengalihan halaman yang terjadi saat mengakses URL tersebut.
40.	nb_external_redirection	Jumlah <i>redirect</i> menuju domain lain.
41.	length_words_raw	Jumlah total kata yang ditemukan dalam URL mentah (biasanya dipisahkan oleh tanda baca).
42.	char_repeat	Menghitung jumlah karakter yang diulang secara berurutan (misal aaa).
43.	shortest_words_raw	Panjang (jumlah huruf) dari kata terpendek yang ditemukan dalam <i>string</i> URL.
44.	shortest_word_host	Panjang dari kata terpendek yang hanya berada di bagian <i>hostname</i> (domain) saja.
45.	shortest_word_path	Kata terpendek setelah domain
46.	longest_words_raw	Panjang (jumlah karakter) dari kata terpanjang yang ditemukan pada seluruh URL (<i>host + path + query + fragment</i>).
47.	longest_word_host	Jumlah karakter dari kata terpanjang di dalam domain
48.	longest_word_path	Jumlah karakter dari kata terpanjang di dalam path

No	Feature	Deskripsi
49.	avg_words_raw	Rata-rata dari jumlah karakter yang diekstrak dari seluruh URL (<i>subdomain</i> , <i>domain</i> , dan <i>path</i>) dibagi dengan jumlah katanya. Kata dipisahkan berdasarkan delimiter: /, ., ?, &, =, _, - dan TLD tidak ikut dihitung.
50.	avg_word_host	Rata-rata dari panjang karakter dibagi dengan kata yang terdapat di dalam <i>host</i> /domain saja (misalnya: <i>www</i> , <i>subdomain</i> , nama domain, TLD).
51.	avg_word_path	Rata-rata dari panjang karakter dibagi dengan kata dalam bagian <i>path</i> URL (setelah domain).
52.	phish_hints	Fitur yang menghitung jumlah kemunculan kata kunci curiga (seperti <i>login</i> , <i>secure</i> , <i>verify</i> , <i>account</i> , <i>bank</i> , <i>update</i> , <i>password</i> , dll.) di seluruh bagian URL (<i>host</i> , <i>subdomain</i> , <i>path</i> , dan <i>query</i>).
53.	domain_in_brand	Di dalam domain mengandung nama <i>brand</i> populer seperti <i>google</i> , <i>paypal</i> , <i>apple</i> , <i>amazon</i> .
54.	brand_in_subdomain	<i>Brand</i> muncul pada <i>subdomain</i> .
55.	brand_in_path	<i>Brand</i> muncul pada path URL.
56.	suspicious_tld	fitur URL menggunakan Top-Level Domain (TLD) yang murah, mudah didaftarkan, atau kurang memiliki pengawasan TLD seperti <i>.tk</i> , <i>.ml</i> , <i>.ga</i> , <i>.cf</i> , <i>.gq</i> , <i>.xyz</i> , <i>.top</i> , dan <i>.pw</i> .
57.	statistical_report	Domain atau URL telah dilaporkan sebagai berbahaya pada database reputasi keamanan, seperti <i>PhishTank</i> , <i>Spamhaus</i> , <i>Google SafeBrowsing</i> , atau layanan <i>blacklist</i> lainnya.
58.	nb_hyperlinks	Jumlah elemen <i>hyperlink</i> <code></code> dalam halaman web.
59.	ratio_intHyperlinks	Jumlah <i>hyperlink</i> ke domain yang sama didalam web seperti <i>login</i> ke web tersebut <i>href="/login"</i> .
60.	ratio_extHyperlinks	Jumlah <i>hyperlink</i> ke luar domain dari web seperti <i>https://google.com</i> .

No	Feature	Deskripsi
61.	ratio_nullHyperlinks	Jumlah <i>hyperlink</i> yang tidak valid (<i>#</i> , <i>javascript</i> , <i>void(0)</i>).
62.	nb_extCSS	Jumlah <i>file</i> CSS dari domain luar (<i><link href="external.css"></i>).
63.	ratio_intRedirection	Jumlah perpindahan otomatis ke halaman dalam domain yang sama
64.	ratio_extRedirection	Jumlah perpindahan otomatis ke domain berbeda
65.	ratio_intErrors	Jumlah <i>error</i> HTTP dari domain internal (404, 403).
66.	ratio_extErrors	Jumlah permintaan <i>error</i> yang tidak berhasil dimuat dari domain lain dibagikan dengan jumlah semua permintaan
67.	login_form	Halaman yang bukan harusnya <i>login</i> tapi memiliki form dengan <i>password input</i>
68.	external_favicon	<i>Favicon</i> diambil dari domain lain.
69.	links_in_tags	Jumlah URL yang muncul di dalam <i>tag</i> HTML seperti <i><meta></i> , <i><script></i> , dan <i><link></i> , yaitu link yang dipakai oleh <i>website</i> di belakang layar untuk memuat <i>file</i> tambahan, bukan <i>link</i> yang bisa diklik oleh pengguna.
70.	submit_email	<i>Form</i> meminta mengirimkan data lewat email (<i>action=mailto:</i>).
71.	ratio_intMedia	Jumlah perbandingan gambar/video/audio dari domain itu sendiri dengan total semua gambar/video/audio.
72.	ratio_extMedia	Jumlah perbandingan gambar/video/audio dari luar domain dengan total semua gambar/video/audio.
73.	Sfh(Server Form Handler)	<i>Form</i> yang mengirim data ke domain luar atau kosong
74.	iframe	Halaman yang menampilkan tampilan web lain yang seharusnya bukan web itu.
75.	popup_window	Halaman yang menampilkan atau meminta data dari pengguna memakai <i>popup</i> JS.

No	Feature	Deskripsi
76.	safe_anchor	Total <i>link</i> aman yang mengarah ke domain itu sendiri dibagi dengan total semua <i>link</i> yang menuju domain itu sendiri maupun di luar domain
77.	onmouseover	Teks yang dibuat terlihat aman tapi sebenarnya itu adalah link <i>phishing</i> apabila di klik.
78.	right_click	Halaman menonaktifkan klik kanan. Digunakan untuk mencegah inspeksi elemen.
79.	empty_title	Tag <title> kosong atau tidak ada.
80.	domain_in_title	Domain muncul di title halaman.
81.	domain_with_copyright	Halaman menampilkan <i>copyright</i> (©) + domain yang biasanya aman.
82.	whois_registered_domain	Domain valid & teregister di WHOIS (<i>Who is the owner of this domain?</i>)
83.	domain_registration_length	Lama pendaftaran domain (tahun). Domain <i>phishing</i> sering umur pendek (<1 tahun).
84.	domain_age	Usia domain sejak dibuat. Domain baru (<6 bulan) cenderung <i>phishing</i> .
85.	web_traffic	<i>Ranking trafik</i> domain dari Alexa/SimilarWeb. Peringkat kunjungan <i>website</i> , situs dengan peringkat tinggi (angka besar) cenderung <i>phishing</i> .
86.	dns_record	Domain memiliki DNS <i>records</i> yang valid (A, MX, NS).
87.	google_index	Google mengindeks URL tersebut.
88.	page_rank	Seberapa banyak atau sering web itu dikunjungi maka akan diberi rank atau nilai yang tinggi(terpercaya)
89.	status	Kolom label atau target variabel dalam dataset yang menunjukkan klasifikasi keamanan suatu URL. Kolom ini memiliki dua kategori <i>output</i> : • <i>Phishing</i> = Tidak aman • <i>Legitimate</i> = Sah atau Aman

3.1.2 Data Preprocessing

Tahapan data *preprocessing* yang dilakukan pada penelitian ini, terdiri dari beberapa tahapan yang bertujuan untuk memperoleh data yang relevan dalam mendeteksi *phishing* dan non-*phishing*. Tahapan data *preprocessing* yang dilakukan yaitu *data selection*, *data transformation*, *data cleaning*.

3.1.2.1 Data Selection

Data selection dilakukan untuk mengurangi atribut yang tidak terlalu berkontribusi dalam model, sehingga akan menyederhanakan *tree* pada model yang mempengaruhi kecepatan waktu pengaplikasian model terhadap tupel baru serta meningkatkan akurasi prediksi. Dilakukan langkah *drop column* untuk memilih beberapa kolom yang relevan dengan struktur URL dan label yang akan digunakan. Hasil *feature selection* ditunjukkan pada Tabel 3. 3 berikut ini.

Tabel 3. 3 *Feature Selection*

No	Feature	Digunakan		Alasan
		Ya	Tidak	
1.	url	✓		Dipilih karena menjadi sumber utama seluruh informasi, tempat semua pola <i>phishing</i> (struktur, simbol, kata kunci) berasal.
2.	length_url	✓		Dipilih karena URL <i>phishing</i> umumnya dibuat sangat panjang untuk menyembunyikan domain asli dan menipu pengguna agar tidak mudah mengenali tujuan sebenarnya.
3.	length_hostname	✓		Digunakan karena <i>hostname phishing</i> cenderung panjang dan kompleks akibat penggunaan <i>subdomain</i> berlapis untuk menyamarkan domain utama.
4.	ip	✓		Penggunaan alamat IP sebagai pengganti nama domain jarang ditemukan pada website sah dan sering digunakan oleh <i>phishing</i> .

No	Feature	Digunakan		Alasan
		Ya	Tidak	
5.	nb_dots	✓		Digunakan karena banyaknya titik mencerminkan jumlah <i>subdomain</i> yang sering dimanfaatkan <i>phishing</i> untuk menyamarkan domain utama.
6.	nb_hyphens	✓		Digunakan karena tanda strip sering dipakai untuk meniru nama <i>brand</i> resmi.
7.	nb_at	✓		Digunakan sebagai indikator karena karakter '@' dapat menyebabkan browser mengabaikan bagian awal URL.
8.	nb_qm	✓		Dipilih karena penggunaan tanda tanya berlebih mengindikasikan manipulasi parameter URL.
9.	nb_and	✓		Dipertimbangkan karena karakter '&' yang berlebihan jarang ditemukan pada URL normal.
10.	nb_or		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
11.	nb_eq	✓		dipertimbangkan karena tanda '=' berlebih mengindikasikan manipulasi parameter URL
12.	nb_underscore	✓		dipilih karena garis bawah jarang digunakan pada domain resmi
13.	nb_tilde	✓		dipertimbangkan karena karakter '~' sering muncul pada struktur URL tidak standar.
14.	nb_percent	✓		dipilih karena simbol '%' sering digunakan untuk menyembunyikan karakter berbahaya melalui <i>encoding</i> .
15.	nb_slash	✓		digunakan untuk mengukur kompleksitas <i>path</i> yang sering diperpanjang pada URL <i>phishing</i> .

No	Feature	Digunakan		Alasan
		Ya	Tidak	
16.	nb_star	✓		dipilih karena karakter '*' tidak umum digunakan pada URL <i>website</i> resmi.
17.	nb_colon	✓		digunakan karena tanda ':' di luar protokol standar menunjukkan struktur URL tidak umum.
18.	nb_comma	✓		dipilih karena koma jarang digunakan dalam URL resmi dan dapat menandakan URL buatan.
19.	nb_semicolumn	✓		digunakan karena titik koma sering dimanfaatkan untuk menyisipkan parameter tersembunyi.
20.	nb_dollar	✓		dipertimbangkan karena simbol '\$' sering dikaitkan dengan manipulasi atau injeksi data.
21.	nb_space	✓		digunakan sebagai indikator karena spasi atau '%20' menunjukkan URL tidak rapi dan mencurigakan.
22.	nb_www	✓		digunakan karena pengulangan kata 'www' sering menjadi upaya peniruan domain resmi
23.	nb_com	✓		Digunakan karena pengulangan kata "com" sering digunakan untuk meniru domain resmi .
24.	nb_dslash	✓		Digunakan karena penggunaan // berlebih di luar protokol dapat mengindikasikan manipulasi <i>path</i> URL.
25.	http_in_path	✓		Digunakan karena keberadaan kata "http" di dalam <i>path</i> sering digunakan untuk mengecoh pengguna agar terlihat sah.
26.	https_token	✓		dipertimbangkan karena kemunculan kata 'https' pada domain atau <i>path</i> sering

No	Feature	Digunakan		Alasan
		Ya	Tidak	
				digunakan untuk memberikan kesan aman palsu
27.	ratio_digits_url	✓		dipilih untuk mengukur proporsi angka yang tinggi pada URL <i>phishing</i>
28.	ratio_digits_host	✓		Digunakan karena <i>hostname phishing</i> sering mengandung banyak angka yang tidak umum pada domain resmi.
29.	punycode	✓		Digunakan karena <i>Punycode</i> sering dimanfaatkan untuk serangan <i>homograph attack</i> dengan karakter mirip huruf.
30.	port	✓		Digunakan karena penggunaan port tidak standar (selain 80/443) jarang ditemukan pada <i>website benign</i> .
31.	tld_in_path	✓		Digunakan karena TLD yang muncul di <i>path</i> merupakan teknik peniruan domain resmi.
32.	tld_in_subdomain	✓		Digunakan karena <i>phishing</i> sering meletakkan TLD palsu pada <i>subdomain</i> untuk mengelabui pengguna.
33.	abnormal_subdomain	✓		Digunakan karena struktur <i>subdomain</i> tidak wajar menunjukkan domain buatan atau otomatis.
34.	nb_subdomains	✓		dipilih karena semakin banyak <i>subdomain</i> , semakin sulit pengguna mengenali domain utama.
35.	prefix_suffix	✓		Digunakan karena tanda '-' pada domain sering digunakan untuk meniru nama <i>brand</i> resmi.
36.	random_domain	✓		Digunakan karena domain acak umumnya dibuat secara otomatis dan jarang digunakan <i>website</i> sah

No	Feature	Digunakan		Alasan
		Ya	Tidak	
37.	shortening_service	✓		Digunakan karena URL <i>shortener</i> sering dimanfaatkan untuk menyembunyikan domain berbahaya.
38.	path_extension	✓		Digunakan karena ekstensi file mencurigakan pada <i>path</i> (mis. <i>.exe</i> , <i>.php</i>) sering ditemukan pada <i>phishing</i> .
39.	nb_redirection	✓		Digunakan karena <i>phishing</i> sering menggunakan banyak <i>redirection</i> untuk menyamarkan tujuan akhir
40.	nb_external_redirection	✓		Digunakan karena pengalihan ke domain eksternal sering ditemukan pada <i>website phishing</i> .
41.	length_words_raw	✓		Digunakan untuk mengukur kompleksitas teks URL secara keseluruhan.
42.	char_repeat	✓		digunakan karena pengulangan karakter berlebih menunjukkan URL tidak natural
43.	shortest_words_raw	✓		Digunakan karena panjang kata paling pendek pada keseluruhan URL dapat menunjukkan penggunaan token tidak bermakna atau hasil pembangkitan otomatis yang umum pada URL <i>phishing</i> .
44.	shortest_word_host	✓		Digunakan karena <i>hostname phishing</i> sering mengandung potongan kata sangat pendek atau tidak natural yang jarang ditemukan pada domain resmi.
45.	shortest_word_path	✓		Digunakan karena <i>path</i> pada URL <i>phishing</i> sering mengandung kata sangat pendek sebagai bagian dari struktur buatan atau parameter tersembunyi.

No	Feature	Digunakan		Alasan
		Ya	Tidak	
46.	longest_words_raw	✓		Digunakan karena kata terpanjang yang ekstrem pada URL mengindikasikan upaya penyamaran melalui <i>string</i> panjang yang tidak mudah dibaca pengguna.
47.	longest_word_host	✓		Digunakan karena <i>hostname phishing</i> sering memuat kata sangat panjang akibat penggabungan nama <i>brand</i> , kata kunci, dan karakter tambahan.
48.	longest_word_path	✓		Digunakan karena <i>path</i> URL <i>phishing</i> sering berisi kata sangat panjang yang digunakan untuk mengelabui atau menyembunyikan tujuan sebenarnya.
49.	avg_words_raw	✓		Digunakan karena rata-rata panjang kata pada URL mencerminkan tingkat kealamian struktur URL, di mana URL <i>phishing</i> cenderung memiliki pola kata yang tidak seimbang.
50.	avg_word_host	✓		Digunakan karena rata-rata panjang kata pada <i>hostname phishing</i> umumnya lebih tidak stabil dibandingkan <i>website</i> resmi yang terstruktur dengan baik.
51.	avg_word_path	✓		Digunakan karena rata-rata panjang kata pada <i>path</i> membantu mengidentifikasi URL <i>phishing</i> yang memiliki struktur path kompleks dan tidak natural.
52.	phish_hints	✓		Digunakan karena keberadaan kata kunci seperti <i>login</i> , <i>verify</i> , <i>secure</i> merupakan indikator <i>phishing</i> kuat.
53.	domain_in_brand	✓		Digunakan karena pencantuman domain pada <i>brand</i> sering digunakan untuk peniruan

No	Feature	Digunakan		Alasan
		Ya	Tidak	
54.	brand_in_subdomain	✓		Digunakan karena <i>phishing</i> sering meletakkan nama <i>brand</i> pada <i>subdomain</i> .
55.	brand_in_path	✓		Digunakan karena <i>path</i> sering dimanfaatkan untuk menyisipkan nama <i>brand</i> palsu.
56.	suspicious_tld	✓		dipilih karena TLD murah atau minim pengawasan sering digunakan oleh <i>website phishing</i> .
57.	statistical_report	✓		Digunakan karena mencerminkan reputasi domain berdasarkan laporan keamanan.
58.	nb_hyperlinks	✓		Digunakan karena <i>phishing</i> sering memiliki jumlah <i>link</i> tidak wajar
59.	ratio_intHyperlinks	✓		digunakan untuk mengukur proporsi link internal yang umumnya tinggi pada <i>website</i> sah
60.	ratio_extHyperlinks	✓		dipertimbangkan karena <i>phishing</i> sering memiliki rasio <i>link</i> eksternal yang tinggi
61.	ratio_nullHyperlinks		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
62.	nb_extCSS	✓		dipilih karena ketergantungan pada <i>file</i> CSS eksternal dapat mengindikasikan <i>website</i> tidak terpercaya
63.	ratio_intRedirection		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
64.	ratio_extRedirection	✓		dipilih karena <i>redirect</i> ke domain luar sering digunakan untuk menyembunyikan tujuan sebenarnya
65.	ratio_intErrors		✓	dimana yang seluruh baris datanya hanya berisi nilai 0

No	Feature	Digunakan		Alasan
		Ya	Tidak	
66.	ratio_extErrors	✓		Digunakan karena banyak <i>error</i> eksternal mengindikasikan <i>website</i> tidak stabil atau palsu.
67.	login_form	✓		Digunakan karena <i>form login</i> pada halaman mencurigakan merupakan ciri utama <i>phishing</i> .
68.	external_favicon	✓		digunakan karena <i>favicon</i> yang diambil dari domain lain menunjukkan <i>website</i> tidak mandiri
69.	links_in_tags	✓		Digunakan karena manipulasi <i>link</i> pada <i>tag</i> HTML sering ditemukan pada <i>phishing</i> .
70.	submit_email		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
71.	ratio_intMedia	✓		Digunakan karena <i>website</i> sah umumnya menggunakan media internal secara konsisten.
72.	ratio_extMedia	✓		Digunakan karena <i>phishing</i> sering memuat media dari domain eksternal
73.	Sfh(Server Form Handler)		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
74.	iframe	✓		Digunakan karena <i>iframe</i> sering dipakai untuk menampilkan konten palsu dari domain lain.
75.	popup_window	✓		Digunakan karena <i>popup</i> sering digunakan untuk meminta data sensitif.
76.	safe_anchor	✓		Digunakan karena <i>anchor</i> tidak aman sering mengarah ke URL berbahaya.
77.	onmouseover	✓		Digunakan karena manipulasi <i>JavaScript</i> pada <i>hover</i> sering digunakan untuk menyembunyikan <i>link</i> asli.

No	Feature	Digunakan		Alasan
		Ya	Tidak	
78.	right_click	✓		Digunakan karena pembatasan klik kanan sering ditemukan pada <i>website phishing</i>
79.	empty_title	✓		Digunakan karena halaman <i>phishing</i> sering memiliki judul kosong atau tidak relevan.
80.	domain_in_title	✓		Digunakan karena <i>website</i> resmi umumnya mencantumkan domain pada <i>title</i> .
81.	domain_with_copyright	✓		Digunakan karena <i>website</i> sah biasanya mencantumkan domain pada <i>copyright</i> .
82.	whois_registered_domain	✓		Digunakan karena domain <i>phishing</i> sering tidak terdaftar secara resmi.
83.	domain_registration_length	✓		Digunakan karena domain <i>phishing</i> biasanya berumur pendek.
84.	domain_age	✓		Digunakan karena usia domain merupakan indikator kuat legitimasi <i>website</i> .
85.	web_traffic	✓		Digunakan karena <i>website phishing</i> umumnya memiliki <i>traffic</i> rendah.
86.	dns_record	✓		Digunakan karena domain <i>phishing</i> sering tidak memiliki <i>DNS record valid</i> .
87.	google_index	✓		Digunakan karena banyak <i>website phishing</i> belum terindeks Google.
88.	page_rank	✓		Digunakan karena <i>website phishing</i> umumnya memiliki PageRank rendah.
89.	status	✓		digunakan sebagai variabel target untuk menentukan kelas URL, yaitu <i>phishing</i> atau <i>benign</i>

Berdasarkan hasil analisis dataset pada Tabel 3. 3, terdapat 6 fitur yang tidak digunakan dalam proses pemodelan, yaitu *nb_or*, *ratio_nullHyperlinks*, *ratio_intRedirection*, *ratio_extErrors*, *submit_email*, dan *sfh*, karena seluruh baris datanya hanya berisi nilai 0. Kolom-kolom dengan varians nol (*zero variance*) ini tidak digunakan karena tidak memiliki nilai pembeda yang berguna bagi model dalam mengenali pola antara kelas *phishing* dan *non-phishing*.

3.1.2.2 Data Transformation

Data transformation merupakan tahap penting dalam *preprocessing* data yang bertujuan untuk mengubah format dan struktur data mentah menjadi bentuk yang sesuai dengan kebutuhan algoritma *machine learning*. Pada penelitian ini, proses *data transformation* difokuskan pada transformasi label kelas dari bentuk kategorikal menjadi numerik.

Dataset yang diperoleh dari tahap *data selection* memiliki kolom status sebagai label target yang merepresentasikan klasifikasi URL. Nilai pada kolom tersebut masih berupa data kategorikal dalam bentuk teks, yaitu *phishing* untuk URL berbahaya dan *legitimate* untuk URL yang aman. Oleh karena itu, dilakukan transformasi label kelas dengan mengonversi nilai *phishing* menjadi nilai numerik 1 dan *legitimate* menjadi nilai numerik 0. Setelah proses konversi dilakukan, nama kolom status kemudian diubah menjadi label agar lebih sesuai dengan kebutuhan proses pemodelan *machine learning*.

3.1.2.3 Data Cleaning

Setelah tahap *data transformation*, dilakukan proses *data cleaning* untuk menjamin kualitas dan integritas dataset sebelum masuk ke tahap pemodelan. Proses pembersihan data dalam penelitian ini difokuskan pada aspek utama, yaitu penanganan data duplikat.

Pada *data cleaning* yang dilakukan adalah penanganan data duplikat. Proses ini dilakukan dengan mengidentifikasi dan menghapus baris data yang memiliki entri identik, khususnya pada fitur URL. Penghapusan baris dengan *link* yang sama

bertujuan untuk menghindari redundansi informasi dan mencegah terjadinya bias pada saat model melakukan proses *training*, sehingga setiap sampel data yang masuk ke tahap pemodelan bersifat unik.

3.2 Analisis Machine Learning

Subbab ini membahas analisis penerapan metode *machine learning* yang akan digunakan, yaitu *Random Forest* dan *XGBoost*. Analisis ini mencakup masing-masing model dalam mengolah data URL hasil *preprocessing*, serta perannya dalam mengklasifikasikan URL sebagai *phishing* atau *non-phishing*.

3.2.1 Analisis Random Forest

Pada penelitian ini, algoritma *Random Forest* dioptimalkan menggunakan beberapa *hyperparameter* utama. Penjelasan mengenai konsep dan karakteristik *hyperparameter Random Forest* telah diuraikan pada sub bab 2.5 Random Forest. Oleh karena itu, pada bagian ini pembahasan difokuskan pada pemilihan dan penggunaan *hyperparameter* dalam proses optimasi model. *Hyperparameter* yang dianalisis meliputi *num.trees*, *mtry*, *sample.fraction*, *replace (bootstrap)*, dan *respect.unordered.factors*, karena parameter-parameter tersebut mengontrol mekanisme utama *Random Forest* dalam membentuk *ensemble decision tree*.

Pemilihan *hyperparameter* ini dinilai telah mencukupi untuk mengendalikan kompleksitas model, meningkatkan diversitas antar *tree*, serta menjaga kemampuan generalisasi model dalam mendeteksi *website phishing*. *Hyperparameter* yang dipilih dalam penelitian ini merupakan *hyperparameter* inti *Random Forest* yang secara langsung memengaruhi mekanisme pembentukan *ensemble* dan paling relevan dengan karakteristik data *phishing* berbasis fitur hasil *rule-based filtering*. Ringkasan analisis pemilihan *hyperparameter* tersebut disajikan pada Tabel 3. 4 berikut.

Tabel 3. 4 Analisis Pemilihan *Hyperparameter Random Forest*

No	Hyperparameter	Alasan Pemilihan	Nilai
1.	<i>num.trees</i>	<i>Hyperparameter</i> ini mengatur jumlah <i>Decison Tree</i> yang dibangun dalam model <i>Random Forest</i> . <i>Hyperparameter</i> ini digunakan karena semakin banyak jumlah <i>tree</i> , semakin stabil hasil prediksi karena varians model berkurang. Namun, jumlah <i>tree</i> yang terlalu besar akan meningkatkan waktu komputasi. Oleh karena itu, <i>num.trees</i> dianalisis untuk mendapatkan keseimbangan antara performa model dan efisiensi komputasi.	Nilai diuji pada beberapa tingkat dengan transformasi <i>trans_2pow_round</i>. Jumlah <i>tree</i> dipilih cukup besar untuk memastikan stabilitas model. Batas : <i>Lower</i> = 0; <i>upper</i> = 11. Rentang: <i>num. trees</i> $\in [1, \infty)$
2.	<i>mtry</i>	<i>Hyperparameter</i> <i>mtry</i> menentukan jumlah fitur yang dipilih secara acak pada setiap pemisahan node. Parameter ini berpengaruh terhadap tingkat keberagaman antar <i>tree</i> . Nilai <i>mtry</i> yang lebih kecil menghasilkan <i>tree</i> yang lebih beragam dan mengurangi korelasi antar <i>tree</i> , sehingga meningkatkan kinerja model <i>ensemble</i> , khususnya pada data <i>phishing</i> dengan banyak fitur berbasis aturan.	Nilai <i>mtry</i> ditentukan dengan nilai awal mengikuti <i>heuristik</i> umum $\sqrt{n}$. Nilai terbaik diperoleh melalui evaluasi performa model. Batas: <i>Lower</i> = 1 <i>upper</i> = <i>nFeatures</i> Rentang : <i>mtry</i> $\in [1, n]$.

No	Hyperparameter	Alasan Pemilihan	Nilai
3.	<i>sample.fraction</i>	<i>Hyperparameter</i> ini dipilih karena mampu mengatur proporsi data yang digunakan untuk melatih setiap <i>tree</i> . Penggunaan <i>sample.fraction</i> yang lebih kecil dari satu meningkatkan variasi data antar <i>tree</i> dan membantu mengurangi <i>overfitting</i> . Hal ini penting untuk meningkatkan kemampuan model dalam mengenali pola <i>phishing</i> yang beragam.	Nilai optimal dipilih berdasarkan hasil evaluasi performa dan waktu komputasi. Batas: <i>Lower</i> = 0.1; <i>upper</i> = 1 Rentang: <i>sample.fraction</i> $\in (0, 1]$.
4.	<i>replace (bootstrap)</i>	Parameter <i>replace</i> menentukan apakah pengambilan sampel data dilakukan dengan pengembalian (<i>bootstrap</i>). <i>Bootstrap</i> sampling meningkatkan variasi antar <i>tree</i> dan mengurangi korelasi antar <i>tree</i> , sehingga membantu menurunkan varians model. Parameter ini dianalisis untuk memastikan mekanisme bagging bekerja secara optimal.	Nilai yang digunakan adalah TRUE atau FALSE dan dievaluasi bersamaan dengan <i>sample.fraction</i> untuk memperoleh kombinasi sampling yang paling stabil. Rentang: <i>replace</i> $\in \{\text{TRUE}, \text{FALSE}\}$. Batas: <i>lower</i> = 1 (FALSE); <i>upper</i> = 2 (TRUE).
5.	<i>respect.unordered.factors</i>	<i>Hyperparameter</i> ini mengatur cara penanganan fitur kategorikal tanpa urutan alami. Parameter ini sangat relevan karena dataset	Parameter <i>respect.unordered.factors</i> dapat dianggap sebagai nilai biner, di mana TRUE menunjukkan

No	Hyperparameter	Alasan Pemilihan	Nilai
		<i>phishing</i> mengandung banyak fitur kategorikal berbasis aturan. Opsi order dipertimbangkan untuk menyeimbangkan kualitas model dan efisiensi komputasi dibandingkan partition yang lebih mahal secara waktu.	bahwa urutan dipertimbangkan, dan FALSE mengabaikan urutan. Pemilihan nilai ini bergantung pada jumlah kelas dan efisiensi komputasi. Batas: $lower = 1$ (<i>ignore</i>); $upper = 2$ (<i>order</i>). Rentang : $respect.unordered.factors \in \{ignore, order, partition\}$.

3.2.2 Analisis XGBoost

Pada penelitian ini, algoritma *XGBoost* digunakan sebagai salah satu metode utama dalam mendeteksi *website phishing* karena kemampuannya dalam memodelkan hubungan nonlinier dan menangani data dengan kompleksitas tinggi. Penjelasan konseptual mengenai mekanisme kerja dan karakteristik *hyperparameter XGBoost* telah diuraikan pada Subbab 2.6 XGBoost. Oleh karena itu, pada bagian ini pembahasan difokuskan pada pemilihan *hyperparameter* yang digunakan dalam proses pelatihan dan optimasi model, serta keterkaitannya dengan karakteristik data URL *phishing* yang dianalisis. *Hyperparameter* yang dipilih merupakan *hyperparameter* inti *XGBoost* yang secara langsung memengaruhi proses *boosting*, kompleksitas model, serta kemampuan generalisasi, sehingga relevan untuk digunakan dalam konteks penelitian ini. Ringkasan *hyperparameter XGBoost* yang digunakan dalam penelitian ini beserta alasan pemilihannya disajikan Tabel 3. 5 berikut.

Tabel 3. 5 Analisis Pemilihan Hyperparameter XGBoost

No	Hyperparameter	Alasan Pemilihan	Nilai
1.	<i>nrounds</i>	Dipilih karena <i>nrounds</i> mengontrol jumlah tree dalam proses <i>boosting</i> . Parameter ini berpengaruh langsung terhadap kompleksitas dan performa model. Pengaturannya memungkinkan model belajar secara bertahap tanpa meningkatkan risiko <i>overfitting</i> secara berlebihan, terutama ketika dikombinasikan dengan nilai <i>eta</i> yang kecil.	Rentang nilai dengan transformasi <i>trans_2pow_round</i> Bounds: <i>lower</i> = 0; <i>upper</i> = 11 Range : $\in [1, \infty[$. <i>Only integer values are valid.</i>
2.	<i>eta (learning rate)</i>	Dipilih untuk mengontrol kecepatan pembelajaran model. Nilai <i>eta</i> yang kecil membantu mengurangi pengaruh setiap <i>tree</i> tunggal, sehingga model menjadi lebih stabil dan memiliki kemampuan generalisasi yang lebih baik pada data <i>phishing</i> yang kompleks dan berisik.	Nilai kecil pada skala logaritmik (<i>trans_2pow</i>) Range <i>eta</i> $\in [0, 1]$. Bounds: <i>Lower</i> = -10; <i>upper</i> = 0
3.	<i>lambda (L2 regularization)</i>	Dipilih untuk mengendalikan kompleksitas model dan mencegah <i>overfitting</i> . Parameter ini berfungsi memperhalus bobot model sehingga hasil prediksi lebih stabil, terutama pada dataset dengan jumlah fitur yang cukup banyak.	Skala logaritmik (<i>trans_2pow</i>) Range <i>lambda</i> $\in [0, \infty[$. Bounds: <i>lower</i> = -10; <i>upper</i> = 10.

No	Hyperparameter	Alasan Pemilihan	Nilai
4.	<i>alpha (L1 regularization)</i>	Dipilih sebagai pelengkap lambda dalam mekanisme regularisasi. <i>Alpha</i> membantu mengurangi pengaruh fitur yang kurang relevan, sehingga model lebih fokus pada fitur penting dalam mendeteksi pola <i>phishing</i> .	Skala logaritmik (<i>trans_2pow</i>) Range : <i>alpha</i> ∈ [0,∞[. Bounds: <i>lower</i> = -10; <i>upper</i> = 10.
5.	<i>subsample</i>	Dipilih untuk meningkatkan diversitas antar <i>tree</i> dengan melatih setiap <i>tree</i> hanya pada sebagian data. Pendekatan ini membantu mengurangi <i>overfitting</i> dan meningkatkan kemampuan generalisasi model tanpa menurunkan performa secara signifikan.	Range: <i>subsample</i> ∈]0,1]. Bounds: <i>lower</i> = 0.1; <i>upper</i> = 1.
6.	<i>colsample_bytree</i>	Dipilih karena mengontrol jumlah fitur yang digunakan pada setiap <i>tree</i> . Parameter ini meningkatkan keberagaman antar <i>tree</i> dan mengurangi ketergantungan model pada fitur tertentu, mirip dengan peran <i>mtry</i> pada <i>Random Forest</i> .	Range: <i>colsample_bytree</i> ∈]0,1]. Bounds: (1/ <i>nFeatures</i> – 1.0)
7.	<i>gamma</i>	Dipilih untuk membatasi pemisahan node yang tidak memberikan peningkatan <i>loss</i> yang signifikan. Parameter ini membuat model lebih konservatif dan membantu	Range: <i>gamma</i> ∈ [0,∞[. Skala logaritmik (<i>trans_2pow</i>) Bounds: <i>lower</i> = -10;

No	Hyperparameter	Alasan Pemilihan	Nilai
		mengurangi pembentukan <i>tree</i> yang terlalu kompleks.	<i>upper</i> = 10.
8.	<i>max_depth</i>	Dipilih untuk mengontrol kedalaman <i>tree</i> dan mencegah model menjadi terlalu kompleks. Pembatasan kedalaman membantu menjaga keseimbangan antara kemampuan belajar pola dan risiko <i>overfitting</i> .	<i>trans_id</i> Rentang nilai terbatas (misalnya 1–15)
9.	<i>min_child_weight</i>	Dipilih untuk membatasi pembentukan <i>node</i> dengan jumlah informasi yang terlalu kecil. Parameter ini membantu mengurangi <i>split</i> yang tidak stabil dan meningkatkan ketahanan model terhadap <i>noise</i> pada data <i>phishing</i> .	Skala logaritmik (trans_2pow) Range : min_child_weight $\in [0, \infty[$. Bounds: <i>lower</i> = 0; <i>upper</i> = 7.

3.2.3 Analisis Stacking Ensemble menggunakan Logistic Regression

Setelah menganalisis kinerja model dasar *Random Forest* dan *XGBoost*, langkah selanjutnya adalah menggabungkan prediksi dari kedua algoritma tersebut menggunakan pendekatan *ensemble learning* yaitu *Stacking* (*Stacked Generalization*). Penggabungan ini dilakukan untuk mengatasi keterbatasan yang dimiliki oleh masing-masing model dasar dan menghasilkan keputusan yang lebih seimbang. Berikut Tabel 3. 6 terdapat beberapa metode penggabungan model yang umum digunakan dalam *ensemble learning*.

Tabel 3. 6 Metode Umum *Ensamble Learning*

No	Metode	Cara Kerja	Kekurangan
1.	<i>Voting (Hard & Soft Voting)</i>	Memilih hasil prediksi terbanyak (<i>Hard</i>) atau rata-rata probabilitas (<i>soft</i>).	Menganggap setiap model punya bobot yang sama atau ditentukan manual. Tidak bisa mendeteksi jika mayoritas model salah arah.
2.	<i>Averaging</i>	Menghitung rata-rata angka dari semua prediksi model.	Tidak fleksibel terhadap pola non-linear. Jika ada satu model yang hasilnya sangat buruk (<i>outlier</i>), rata-ratanya akan ikut rusak.
3.	<i>Bagging (Bootstrap Aggregating)</i>	Melatih banyak model sejenis secara paralel.	Biasanya hanya menggunakan satu jenis algoritma. Jika algoritma tersebut punya kelemahan bawaan, semua model akan punya kelemahan yang sama.
4.	<i>Boosting</i>	Model dilatih berurutan, lalu model baru memperbaiki kesalahan model sebelumnya.	Sangat agresif mengejar kesalahan, sehingga jika data kotor, model akan "menghafal" sampah data tersebut (<i>overfitting</i>).
5.	<i>Stacking (Stacked Generalization)</i>	Menggunakan model tambahan (<i>Meta-Model</i>) untuk belajar menggabungkan prediksi.	Butuh waktu training lebih lama dan memori lebih besar karena melatih banyak lapisan model.

Stacking (Stacked Generalization) dipilih dalam penelitian ini karena metode ini menawarkan apa yang tidak dimiliki oleh teknik *ensemble* lain seperti *Voting* atau *Averaging*. Jika *Voting* hanya mengandalkan suara terbanyak, *Stacking* mampu mempelajari hubungan kompleks antar model. Alasan terkuatnya adalah kemampuan untuk menggabungkan algoritma yang berbeda yaitu *Random Forest* dan *XGBoost*. *Stacking* bertindak sebagai penengah yang secara otomatis

memberikan bobot lebih tinggi kepada model yang paling akurat dan mengurangi pengaruh model yang sering salah, sehingga hasil akhirnya jauh lebih stabil dan akurat dalam menghadapi data baru.

Proses *Stacking* berjalan dalam dua tahap utama. Tahap pertama disebut *Level 0*, di mana beberapa model dasar (*base-learners*) dilatih menggunakan dataset asli. Setelah terlatih, model-model ini akan memprediksi data. Hasil prediksi dari masing-masing model tersebut tidak langsung dianggap sebagai jawaban akhir, melainkan dikumpulkan dan disusun menjadi sebuah dataset baru yang disebut *meta features*. Dataset baru inilah yang kemudian diteruskan ke tahap kedua, yaitu *Level 1*, di mana sebuah model pemutus yang disebut *meta learner* akan dilatih menggunakan input dari prediksi-prediksi *Level 0* tadi untuk menghasilkan keputusan akhir yang paling optimal.

Meta learner adalah "otak" di balik arsitektur *Stacking* yang bertugas sebagai pengambil keputusan final. *Meta learner* tidak lagi melihat data mentah (seperti fitur asli), melainkan belajar dari pola prediksi yang dihasilkan oleh *base-learners*. Tugas utamanya adalah mengenali kapan suatu model dasar bisa dipercaya dan kapan tidak. Misalnya, jika Model A selalu salah saat memprediksi data kategori tertentu, *meta learner* akan belajar untuk lebih mempercayai Model B pada kondisi tersebut. Secara spesifik dalam penelitian ini, *Logistic Regression* digunakan sebagai *meta learner* karena sederhana namun efektif dalam menggabungkan prediksi dari beberapa *base learner* dan merupakan *meta learner* yang umum digunakan karena performanya yang konsisten dan stabil dalam metode *stacking*.

3.3 Analisis Rule based Filtering

Pada sub-bab ini dilakukan analisis penerapan *rule-based filtering* sebagai tahap awal dalam mendeteksi indikasi *phishing* pada sebuah URL. Pendekatan ini digunakan untuk mengevaluasi karakteristik struktural URL yang umum ditemukan pada situs *phishing*, sehingga sistem dapat memberikan penilaian awal secara cepat tanpa memerlukan proses komputasi yang kompleks. *Rule-based filtering*

memanfaatkan seperangkat aturan berbasis fitur URL untuk mengidentifikasi pola mencurigakan sebelum dilakukan proses analisis lanjutan.

Penelitian ini menggunakan total 89 fitur dalam proses deteksi *phishing*. Namun, tidak seluruh fitur tersebut digunakan sebagai dasar pembentukan aturan (*rule-based filtering*). Hal ini disebabkan oleh perbedaan karakteristik antar fitur, di mana tidak semua fitur memiliki pola kemunculan yang jelas dan konsisten untuk diformulasikan dalam bentuk aturan statis. Beberapa fitur menunjukkan indikasi *phishing* yang dapat dikenali secara langsung dari keberadaan karakter, struktur URL, atau atribut domain tertentu. Sebaliknya, terdapat fitur-fitur lain yang bersifat statistik, berbasis rasio, atau mencerminkan kompleksitas struktur URL, sehingga memerlukan proses pembelajaran pola dari data untuk dapat dimanfaatkan secara optimal.

Oleh karena itu, pemilihan fitur untuk *rule-based filtering* dilakukan melalui analisis karakteristik data, tinjauan terhadap penelitian terdahulu, serta evaluasi terhadap kejelasan pola dan kekuatan indikasi masing-masing fitur terhadap perilaku *phishing*. Fitur-fitur yang dijadikan dasar pembentukan aturan merupakan fitur yang dapat diidentifikasi secara langsung dari struktur URL, penggunaan *token* mencurigakan, serta informasi domain yang bersifat eksplisit. Sementara itu, fitur-fitur lainnya tetap digunakan pada tahap pembelajaran *machine learning* karena kontribusinya lebih efektif ketika dipelajari melalui hubungan non-linear antar fitur.

Setiap aturan merepresentasikan satu kondisi tertentu yang sering muncul pada URL *phishing*, seperti penggunaan karakter khusus, struktur domain yang kompleks, serta pola penyamaran URL. Tabel 3. 7 menampilkan daftar fitur yang digunakan, kategori tingkat kepentingannya, serta aturan deteksi yang diterapkan pada masing-masing fitur.

Tabel 3. 7 Rule base Filtering

No	Rule	Kategori	Deskripsi Rule
1.	suspicious_tld	Sangat Penting	TLD yang ditandai sebagai mencurigakan seperti:

No	Rule	Kategori	Deskripsi Rule
			TLD Negara: • .pw (Palau) - sering digunakan untuk <i>phishing</i> • .top - populer untuk spam dan malware • .xyz - banyak digunakan untuk situs mencurigakan • .cc (Cocos Islands) - sering untuk konten ilegal • .ws (Samoa) - digunakan untuk skema penipuan • .biz - reputasi buruk karena spam TLD Generik Baru (New gTLD): • .club - sering untuk situs perjudian ilegal • .online - tinggi penyalahgunaan phishing • .site - banyak malware distribution • .live - digunakan untuk streaming ilegal • .work - sering untuk scam pekerjaan • .icu - tinggi aktivitas phishing • .info - spam dan konten <i>low-quality</i> TLD dengan Reputasi Buruk: • .cn (China) - banyak situs spam (meski benign juga banyak)

No	Rule	Kategori	Deskripsi Rule
			• .ru (Russia) - sering dikaitkan dengan cybercrime • .zip - baru tapi sangat berisiko karena mirip ekstensi file • .loan - jelas untuk pinjaman mencurigakan • .download - distribusi malware • .click - clickbait dan malware
2.	nb_at	Sangat Penting	≥ 1 karakter "@" ditandai sebagai mencurigakan, jika "@" sebelum <i>hostname</i> ditandai sebagai <i>phishing</i> .
3.	ip	Sangat Penting	• IP = 1, jika <i>hostname</i> URL mengandung IP Address. • IP = 0, jika <i>hostname</i> URL menggunakan nama domain dan tidak mengandung IP Address.
4.	nb_underscore	Sangat Penting	> 3 karakter garis bawah (_) ditandai sebagai mencurigakan, jika pada domain ditandai sebagai sangat mencurigakan.
5.	ratio_digits_url	Penting	Persentase angka dalam <i>hostname</i> . Jika > 0.3 (lebih dari 30% URL adalah angka).
6.	nb_subdomains	Penting	> 3 <i>subdomain</i> ditandai sebagai mencurigakan.
7.	nb_percent	Penting	> 5 karakter persen (%) ditandai sebagai mencurigakan.
8.	nb_tilde	Penting	≥ 1 karakter tilde "~" ditandai sebagai mencurigakan.
9.	nb_semicolumn	Penting	≥ 1 karakter titik koma (;) ditandai sebagai mencurigakan.
10.	nb_star	Penting	≥ 1 karakter <i>star</i> (*) ditandai sebagai mencurigakan.
11.	nb_comma	Penting	≥ 1 karakter koma (,) ditandai sebagai mencurigakan.

No	Rule	Kategori	Deskripsi Rule
12.	random_domain	Penting	Domain acak/tidak bermakna menunjukkan pembuatan otomatis (DGA-like).
13.	length_hostname	Cukup Penting	> 30 panjang karakter <i>hostname</i> ditandai sebagai mencurigakan.
14.	nb_dollar	Cukup Penting	≥ 1 karakter dolar "\$" ditandai sebagai mencurigakan.
15.	nb_qm	Cukup Penting	> 2 karakter tanda tanya "?" ditandai sebagai mencurigakan.
16.	nb_colon	Cukup Penting	> 1 karakter titik dua (:) (lebih dari protokol)
17.	nb_eq	Cukup Penting	> 8 karakter sama dengan (=) ditandai sebagai mencurigakan.
18.	nb_dots	Cukup Penting	> 4 karakter titik (.) ditandai sebagai mencurigakan.
19.	nb_slash	Cukup Penting	> 7 karakter garis miring (/) ditandai sebagai mencurigakan.
20.	nb_and	Cukup Penting	> 3 karakter and "&" ditandai sebagai mencurigakan.
21.	nb_hyphens	Cukup Penting	> 3 karakter minus (-) ditandai sebagai mencurigakan.
22.	http_in_path	Cukup Penting	Jika kata http muncul pada path, menandakan upaya penyamaran URL
23.	https_token	Cukup Penting	Token https di domain/path digunakan untuk memberi kesan aman palsu.
24.	port	Cukup Penting	Penggunaan port selain <i>default</i> (Standard Ports: 21, 22, 23, 80, 443, 445, 1433, 1521, 3306, and 3389). ditandai mencurigakan.
25.	shortening_service	Cukup Penting	Jika URL menggunakan layanan pemendek (bit.ly, tinyurl), ditandai mencurigakan

Aturan-aturan pada Tabel 3. 7 berfungsi sebagai aturan dasar yang mengevaluasi keberadaan karakteristik mencurigakan pada sebuah URL secara individual. Setiap aturan menghasilkan keluaran berupa kondisi terpenuhi atau tidak terpenuhi, tanpa langsung menentukan keputusan akhir.

Hasil evaluasi dari seluruh aturan tersebut kemudian dikombinasikan melalui mekanisme *rule-based filtering* untuk menentukan tingkat risiko *phishing* secara keseluruhan. Dalam mekanisme ini, kategori kepentingan fitur (sangat penting, penting, dan cukup penting) digunakan sebagai dasar untuk menilai seberapa besar kontribusi masing-masing aturan terhadap risiko *phishing*.

Proses pengambilan keputusan dilakukan melalui dua tahapan. Tahap pertama menggunakan aturan keras (*hard rule*), yaitu keputusan langsung yang diambil apabila ditemukan kombinasi fitur dengan tingkat kepentingan sangat penting yang menunjukkan pola *phishing* yang sangat kuat. Apabila kondisi ini tidak terpenuhi, sistem melanjutkan ke tahap kedua menggunakan aturan berbasis ambang batas (*threshold-based rule*), di mana skor risiko dihitung dari akumulasi bobot fitur yang terdeteksi dan dibandingkan dengan nilai ambang batas (*threshold*) untuk menentukan kategori URL.

3.3.1 Rule 1: High-Confidence Rule

Aturan keras digunakan untuk mendeteksi pola *phishing* dengan tingkat keyakinan sangat tinggi.

Ketentuan *Rule 1*:

Apabila ditemukan satu atau lebih (≥ 1) fitur dengan tingkat kepentingan sangat penting, maka URL langsung dikategorikan sebagai *phishing*, tanpa melalui proses perhitungan *threshold* lebih lanjut.

Rule 1 menetapkan bahwa URL langsung diklasifikasikan sebagai *phishing* apabila terdeteksi satu atau lebih fitur dengan tingkat kepentingan sangat tinggi. Pendekatan ini didasarkan pada asumsi bahwa fitur-fitur yang tergolong sangat penting

memiliki kemampuan yang kuat dalam membedakan antara URL phishing dan URL yang sah.

Keberadaan minimal satu fitur sangat penting sudah cukup untuk menunjukkan indikasi pola yang khas pada URL phishing, yang umumnya jarang ditemukan pada website benign. Oleh karena itu, jika satu atau lebih fitur sangat penting terdeteksi, URL dianggap memiliki tingkat risiko phishing yang sangat tinggi dan dapat langsung diklasifikasikan sebagai phishing tanpa perlu melalui proses perhitungan skor risiko berbasis threshold.

3.3.2 Rule 2 : Threshold-Based Rule

Apabila *Rule 1* tidak terpenuhi, maka sistem akan menerapkan perhitungan skor risiko berdasarkan akumulasi poin dari fitur-fitur yang terdeteksi. Pada Tabel 3. 8 diberikan bobot sebagai berikut:

Tabel 3. 8 Kategori Fitur

Kategori Fitur	Bobot
Sangat penting	3
Penting	2
Cukup penting	1

Skor risiko dihitung dengan menerapkan aturan umum *threshold-based risk scoring*, di mana setiap fitur memberikan kontribusi bobot tertentu terhadap nilai risiko. Rumus perhitungan skor risiko dinyatakan sebagai berikut:

$$Risk\ Score = (3 \times sangat\ penting) + (2 \times penting) + (1 \times cukup\ penting)$$

Berdasarkan skor risiko yang diperoleh, URL diklasifikasikan ke dalam beberapa kategori tingkat risiko menggunakan pendekatan *threshold-based classification*. Penentuan kategori dilakukan dengan membandingkan nilai skor risiko terhadap ambang batas (*threshold*) yang telah ditetapkan. Berikut adalah Tabel 3. 9.

Tabel 3. 9 Nilai Skor Risiko pada Rule-based

Risk Score	Kategori
≥ 7	Phishing

<i>Risk Score</i>	Kategori
1-6	<i>Suspicious</i>
0	<i>Benign</i>

Berdasarkan Tabel 3.9, penentuan kategori pada *rule-based filtering* didasarkan pada nilai *risk score* yang dihitung dari sejumlah aturan yang telah ditentukan. URL dengan skor risiko ≥ 7 dikategorikan sebagai *phishing*, sedangkan URL dengan skor 0–6 dikategorikan sebagai *Suspicious*.

URL yang termasuk dalam kategori *phishing* merupakan URL yang memenuhi sejumlah aturan kritis atau memiliki kombinasi indikator yang sangat kuat mengarah pada aktivitas phishing. Pada kondisi ini, sistem tidak melanjutkan URL ke tahap *machine learning*, karena tingkat keyakinan terhadap karakteristik phishing sudah tinggi. Dengan demikian, keputusan dapat diambil secara langsung untuk meminimalkan potensi *false negative* serta meningkatkan efisiensi proses deteksi dengan mengurangi beban komputasi.

Sementara itu, URL dengan skor risiko pada rentang 0-6 dikategorikan sebagai *Suspicious*. Kategori ini mencakup berbagai kondisi, mulai dari URL dengan indikasi lemah hingga URL yang menunjukkan beberapa karakteristik mencurigakan, namun belum cukup kuat untuk diklasifikasikan secara langsung sebagai *phishing*. Oleh karena itu, seluruh URL dalam kategori *Benign* diteruskan ke tahap *machine learning* untuk dianalisis lebih lanjut menggunakan model yang telah dibangun, seperti *Random Forest* dan *XGBoost*.

Pendekatan ini menunjukkan bahwa *rule-based filtering* berfungsi sebagai mekanisme penyaringan awal (*first-pass filtering*), yang hanya mengambil keputusan langsung pada kasus dengan tingkat risiko tinggi, sementara kasus dengan tingkat ketidakpastian tetap diproses oleh model *machine learning*. Dengan strategi ini, sistem mampu menjaga keseimbangan antara efisiensi dan akurasi, di mana keputusan cepat diambil untuk kasus yang jelas, dan analisis lebih mendalam dilakukan untuk kasus yang ambigu.

Meskipun demikian, seluruh URL dengan skor rendah tetap memiliki potensi mengandung pola *phishing* yang tidak terdeteksi oleh aturan sederhana. Oleh karena itu, integrasi dengan model *machine learning* menjadi penting untuk meningkatkan kemampuan deteksi, khususnya dalam mengenali pola phishing yang lebih kompleks dan adaptif. Dengan kombinasi ini, sistem diharapkan mampu meminimalkan *false negative* sekaligus mempertahankan efisiensi pemrosesan secara keseluruhan.

3.4 Analisis Dataset Eksperimen

Pada sub-bab ini, dilakukan analisis terhadap dataset eksperimen yang terdiri dari 40 sampel URL independen yaitu 20 URL *phishing* dan 20 URL *benign*. Dataset ini bersifat terpisah dari data yang digunakan pada proses *training* maupun *testing*, dengan tujuan untuk mengamati bagaimana model yang telah dilatih berperilaku terhadap data baru yang belum pernah dikenali sebelumnya.

3.4.1 Analisis *Parsing*

Analisis *parsing* merupakan tahap penting yang dilakukan untuk mengurai struktur lengkap dari setiap URL. Pada tahap ini dilakukan proses pemisahan (*parsing*) terhadap 40 sampel yang diambil di luar dataset yang digunakan pada proses *training* dan *testing*. Proses ini penting karena sebuah URL bukan sekadar deretan teks, melainkan tersusun atas beberapa elemen yang memiliki fungsi dan aturan tertentu. Melalui proses *parsing*, setiap URL diuraikan menjadi beberapa bagian utama seperti *scheme* (protokol), *hostname* (nama domain), *path* (jalur direktori), serta *query parameter*.

Hasil dari proses *parsing* ini kemudian dipetakan ke dalam 81 fitur yang merepresentasikan berbagai karakteristik struktur URL secara menyeluruh. Proses ini dilakukan untuk mentransformasikan data mentah URL ke dalam format vektor numerik, sehingga setiap URL dapat direpresentasikan dalam bentuk data terstruktur yang siap digunakan pada tahap ekstraksi.

3.4.2 Analisis Feature Extraction

Setelah URL diproses melalui tahap *parsing*, langkah selanjutnya adalah *feature extraction* untuk mengubah karakteristik URL menjadi data numerik yang dapat diproses oleh algoritma *machine learning*.

Penelitian ini menggunakan pendekatan *URL-based feature extraction*, yaitu teknik ekstraksi fitur yang memanfaatkan karakteristik pada struktur URL tanpa perlu mengakses atau memuat isi halaman web. Pendekatan ini dipilih karena mampu menangkap berbagai pola yang sering muncul pada URL berbahaya, seperti penggunaan karakter khusus yang berlebihan, domain yang mencurigakan, serta indikasi manipulasi struktur tautan.

Secara keseluruhan, penelitian ini menggunakan 81 fitur, yang terdiri dari 37 fitur *URL-based*, serta 44 fitur tambahan non-URL. Fitur non-URL diperoleh dari analisis konten halaman web, informasi domain, serta karakteristik tambahan dari URL.

3.4.2.1 URL-Based (37 Fitur)

Fitur ini diperoleh langsung dari struktur URL tanpa perlu mengakses isi halaman web. Karakteristik yang dianalisis meliputi panjang URL, jumlah karakter khusus, jumlah subdomain, serta indikasi manipulasi URL. Berikut pada Tabel 3. 10

Tabel 3. 10 URL Based

"length_url", "length_hostname", "ip", "nb_dots", "nb_hyphens", "nb_at", "nb_qm", "nb_and", "nb_eq", "nb_underscore", "nb_tilde", "nb_percent", "nb_slash", "nb_star", "nb_colon", "nb_comma", "nb_semicolumn", "nb_dollar", "nb_space", "nb_www", "nb_com", "nb_dslash", "http_in_path", "https_token", "ratio_digits_url", "ratio_digits_host", "punycode", "port", "tld_in_path", "tld_in_subdomain", "abnormal_subdomain", "nb_subdomains", "prefix_suffix", "random_domain",

"shortening_service", "path_extension", "nb_redirection"
--

3.4.2.2 Non-URL (44 Fitur)

Selain fitur berbasis URL-based, penelitian ini juga menggunakan 44 fitur tambahan yang memberikan informasi lebih mendalam mengenai struktur halaman web, domain, serta karakteristik tambahan dari URL. Fitur-fitur ini dibagi menjadi tiga kelompok utama.

a. Web Content / HTML-DOM (19 Fitur)

Fitur ini diperoleh dari analisis struktur HTML dan *Document Object Model* (DOM) halaman web untuk mendeteksi pola yang sering muncul pada halaman *phishing*, seperti penggunaan *iframe* tersembunyi, formulir *login* mencurigakan, atau banyaknya tautan eksternal, sehingga tidak dapat diekstraksi hanya dari *string* URL dan memerlukan akses langsung ke isi halaman web. Daftar fitur ini disajikan pada Tabel 3. 11.

Tabel 3. 11 Fitur Berbasis Web Content / HTML-DOM

"nb_hyperlinks", "ratio_intHyperlinks", "ratio_extHyperlinks", "nb_extCSS", "ratio_extRedirection", "ratio_extErrors", "login_form", "external_favicon", "links_in_tags", "ratio_intMedia", "ratio_extMedia", "iframe", "popup_window", "safe_anchor", "onmouseover", "right_click", "empty_title", "domain_in_title", "domain_with_copyright"
--

b. Domain/WHOIS/Reputation (8 Fitur)

Fitur ini bergantung pada sumber data eksternal seperti layanan WHOIS, *database* reputasi domain, indeks mesin pencari, serta metrik *traffic web*. Ketergantungan pada layanan pihak ketiga ini menyebabkan fitur-fitur tersebut tidak selalu tersedia secara *real-time* dan dapat menghambat sistem deteksi. Daftar fitur ini disajikan pada Tabel 3. 12.

Tabel 3. 12 Fitur Berbasis Domain, WHOIS, dan Reputasi

"statistical_report", "whois_registered_domain", "domain_registration_length",
"domain_age", "web_traffic", "dns_record", "google_index", "page_rank"

c. URL/Host-derived tambahan (17 Fitur)

Fitur-fitur ini mencakup statistik berbasis kata (*word-based statistics*) seperti panjang kata terpendek dan terpanjang pada *host* maupun *path*, serta fitur-fitur yang berkaitan dengan indikasi merek (*brand-related features*) dan kemiripan dengan kata kunci *phishing*. Daftar fitur ini disajikan pada Tabel 3. 13.

Tabel 3. 13 Fitur URL/Host Turunan Lanjutan

"nb_external_redirection", "length_words_raw", "char_repeat",
"shortest_words_raw", "shortest_word_host", "shortest_word_path",
"longest_words_raw", "longest_word_host", "longest_word_path",
"avg_words_raw", "avg_word_host", "avg_word_path",
"phish_hints", "domain_in_brand", "brand_in_subdomain",
"brand_in_path", "suspicious_tld"

3.5 Analisis Eksperimen

Subbab ini membahas analisis eksperimen dilakukan untuk mengevaluasi kinerja berbagai pendekatan yang digunakan dalam sistem deteksi URL *phishing*, yaitu model *Random Forest*, *XGBoost*, penggabungan keduanya menggunakan metode *stacking*, serta integrasi *rule-based filtering* dengan model *machine learning*.

3.5.1 Eksperimen Random Forest

Pada eksperimen pertama ini, menggunakan algoritma *Random Forest* sebagai salah satu model klasifikasi tunggal yang akan diuji dalam mendeteksi URL *phishing*. *Random Forest* memanfaatkan kumpulan *decision tree* yang dibangun dari subset data dan fitur yang berbeda, sehingga mampu mengurangi *overfitting* dan mampu bekerja dengan baik pada data baru yang belum pernah dilihat

sebelumnya. Karakteristik ini menjadikan *Random Forest* relevan untuk menangani pola *non-linear* pada fitur URL, seperti panjang URL, struktur hostname, dan keberadaan karakter khusus.

Analisis eksperimen terhadap *Random Forest* direncanakan untuk mengamati sejauh mana model ini mampu mengklasifikasikan URL *phishing* dan *non-phishing* secara tepat. Selain itu, evaluasi juga akan difokuskan pada kemampuan model dalam mendeteksi URL *phishing* secara menyeluruh, yang tercermin dari metrik evaluasi. Hasil pengujian *Random Forest* nantinya akan digunakan sebagai acuan *baseline* dalam membandingkan performa dengan pendekatan lainnya.

3.5.2 Eksperimen XGBoost

Pada eksperimen kedua, menggunakan algoritma *XGBoost* sebagai salah satu model klasifikasi tunggal yang dirancang sebagai model pembanding yang akan diuji untuk meningkatkan performa deteksi URL *phishing*. Algoritma ini menerapkan metode *boosting*, di mana model dibangun secara bertahap dengan berfokus pada perbaikan kesalahan dari model sebelumnya. Pendekatan tersebut memungkinkan *XGBoost* mempelajari pola-pola kompleks yang sulit ditangkap oleh satu model klasifikasi tunggal.

Analisis eksperimen *XGBoost* direncanakan untuk mengevaluasi pengaruh mekanisme *boosting* terhadap peningkatan performa klasifikasi pada metrik evaluasi. Selain itu, aspek kompleksitas komputasi dan kebutuhan pengaturan parameter juga akan diperhatikan dalam analisis, guna menilai keseimbangan antara peningkatan performa dan efisiensi sistem.

3.5.3 Eksperimen Stacking

Dalam eksperimen ketiga ini, menggunakan metode *Stacking* (*Stacked Generalization*) digunakan sebagai pendekatan *ensemble* untuk meningkatkan performa model dalam melakukan prediksi. Metode *stacking* dipilih karena mampu menggabungkan keunggulan beberapa model *machine learning* dengan cara yang lebih adaptif dibandingkan metode *ensemble* sederhana seperti *voting* atau

averaging. Pada metode *voting*, setiap model memiliki bobot yang sama, sehingga perbedaan tingkat kepercayaan dan pola kesalahan antar model tidak diperhitungkan secara eksplisit.

Algoritma *Random Forest* dan *XGBoost* dipilih sebagai *base learners* karena keduanya merupakan algoritma berbasis *tree*, tetapi memiliki mekanisme pembelajaran yang berbeda. *Random Forest* menggunakan metode *bagging* dengan membangun banyak *tree* yang belajar dari sampel data secara acak, sedangkan *XGBoost* menggunakan *boosting* yang membangun *tree* secara bertahap dengan memfokuskan diri pada kesalahan model sebelumnya. Perbedaan prinsip pembelajaran ini menghasilkan dua model dengan pola prediksi yang tidak identik. Sebaliknya, *XGBoost* merupakan algoritma *boosting* yang membangun model secara bertahap dengan fokus memperbaiki kesalahan model sebelumnya. Pendekatan ini memungkinkan *XGBoost* untuk menangkap pola-pola kompleks dan hubungan *non-linear* antar fitur, meskipun memiliki sensitivitas yang lebih tinggi terhadap *overfitting*. Dengan menggabungkan kedua model sebagai *base learners*, sistem mampu memanfaatkan keunggulan *Random Forest* dalam stabilitas dan keunggulan *XGBoost* dalam ketelitian pembelajaran.

Stacking mengambil sejumlah algoritma *learning* $\{A_1, \dots, A_N\}$ dan menjalankannya pada dataset T yang sedang dipertimbangkan (yaitu, data tingkat dasar) untuk menghasilkan serangkaian model $\{h_1, \dots, h_N\}$. Kemudian, dataset baru T dibuat dengan mengganti deskripsi setiap *instance* dalam dataset tingkat dasar dengan prediksi dari setiap model tingkat dasar untuk *instance* tersebut.

Meta-learner berperan sebagai pengambil keputusan akhir dalam arsitektur *stacking*. Dalam penelitian ini, *meta-learner* dilatih menggunakan *output* probabilistik dari *Random Forest* dan *XGBoost* sebagai fitur masukan. Dengan pendekatan ini, *meta-learner* tidak hanya menggabungkan prediksi, tetapi juga mempelajari kondisi di mana salah satu model lebih dapat dipercaya dibandingkan model lainnya.

Peran *meta-learner* sangat penting dalam mengatasi konflik prediksi antara *Random Forest* dan *XGBoost*, terutama pada kasus URL *phishing* yang bersifat ambigu. *Meta-learner* memungkinkan sistem untuk menyeimbangkan bias dan varians dari kedua *base learners*, sehingga menghasilkan keputusan yang lebih stabil dan akurat. Dengan demikian, penggunaan *stacking ensemble* tidak hanya meningkatkan performa prediksi, tetapi juga memperkuat kemampuan sistem dalam beradaptasi terhadap variasi pola serangan *phishing* yang terus berkembang.

3.5.4 Eksperimen Rule based Filtering dengan Stacking

Pada eksperimen keempat mengintegrasikan *rule-based filtering* sebagai lapisan penyaring awal dengan model *stacking ensemble* yang menggabungkan *Random Forest* dan *XGBoost*. Pendekatan *hybrid* ini dirancang untuk memanfaatkan keunggulan dari metode berbasis aturan yang relevan, serta kekuatan *machine learning* yang mampu menangkap pola kompleks. Integrasi ini bertujuan untuk meningkatkan akurasi deteksi, mengurangi kesalahan klasifikasi, dan meningkatkan efisiensi komputasi sistem secara keseluruhan.

Rule-based filtering berperan sebagai mekanisme deteksi cepat yang mengevaluasi URL berdasarkan karakteristik struktural yang telah diidentifikasi sebagai indikator kuat *phishing*. Aturan-aturan ini didasarkan pada analisis fitur yang telah dilakukan pada Subbab sebelumnya, mencakup karakteristik seperti penggunaan TLD mencurigakan, keberadaan karakter khusus tertentu, penggunaan alamat IP sebagai domain, dan pola-pola struktural URL yang tidak wajar.

Proses *filtering* dimulai dengan ekstraksi fitur dari URL yang akan dievaluasi. Fitur-fitur ini kemudian diperiksa terhadap serangkaian aturan yang telah didefinisikan, dengan setiap aturan memiliki tingkat kepentingan yang berbeda. Aturan dikategorikan menjadi tiga tingkat kepentingan yaitu sangat penting, penting, dan cukup penting, yang mencerminkan seberapa kuat indikasi aturan tersebut terhadap klasifikasi *phishing*.

Aturan dengan kategori sangat penting mencakup fitur-fitur yang memiliki korelasi tinggi dengan URL *phishing* dan jarang ditemukan pada URL *benign*. Contohnya adalah penggunaan TLD mencurigakan seperti *.xyz*, *.top*, *.pw*, atau *.icu* yang sering digunakan untuk aktivitas *phishing* karena mudah didaftarkan dan memiliki pengawasan minimal. Keberadaan karakter '@' dalam URL, terutama jika muncul sebelum *hostname*, juga merupakan indikator kuat karena dapat menyebabkan browser mengabaikan bagian awal URL. Penggunaan alamat IP sebagai domain menggantikan nama domain teks juga termasuk kategori sangat penting karena jarang digunakan oleh *website* sah.

Aturan dengan kategori "penting" mencakup fitur-fitur yang berkontribusi signifikan dalam membedakan *phishing* dan situs *benign*, namun tidak cukup untuk langsung menetapkan klasifikasi secara definitif. Contohnya adalah jumlah *subdomain* yang berlebihan (>3), penggunaan karakter persen (%) yang tinggi, atau keberadaan karakter-karakter khusus seperti tilde (~) dan *semicolon* (;) yang tidak umum dalam URL normal. Fitur-fitur ini perlu dikombinasikan dengan indikator lain untuk menghasilkan keputusan yang lebih akurat.

Aturan dengan kategori "cukup penting" merupakan fitur pendukung yang memiliki pengaruh relatif kecil namun dapat meningkatkan akurasi ketika digabungkan dengan fitur lain. Contohnya adalah panjang *hostname* yang berlebihan, jumlah karakter tertentu seperti tanda tanya (?), sama dengan (=), atau garis miring (/) yang melebihi batas normal. Meskipun fitur-fitur ini sendiri tidak cukup kuat sebagai indikator *phishing*, kombinasinya dapat memberikan informasi tambahan yang berharga.

Evaluasi *rule-based filtering* dilakukan menggunakan sistem skoring kumulatif berdasarkan tingkat kepentingan aturan yang terpenuhi. Berdasarkan skor risiko yang diperoleh, URL diklasifikasikan ke dalam tiga kategori, yaitu *Phishing*, *Suspicious*, dan *Benign*, sesuai dengan ketentuan klasifikasi skor risiko. URL dengan skor risiko tinggi atau yang memenuhi satu atau lebih aturan sangat penting dapat langsung diklasifikasikan sebagai *Phishing* tanpa melibatkan model *machine*

learning. URL dengan skor risiko menengah dikategorikan sebagai *Suspicious* dan diteruskan ke tahap analisis menggunakan model *machine learning*. Sementara itu, URL dengan skor risiko rendah diklasifikasikan sebagai *Benign* dan tetap diverifikasi oleh model *machine learning* untuk memastikan keakuratan klasifikasi.

Integrasi *rule-based filtering* dengan *stacking ensemble* bertujuan untuk memanfaatkan keunggulan kedua pendekatan secara optimal. *Rule-based filtering* memberikan keputusan cepat untuk kasus-kasus *phishing* yang jelas, sehingga dapat mengurangi beban komputasi sistem. Di sisi lain, *stacking ensemble* digunakan untuk menganalisis URL dengan tingkat ambiguitas yang lebih tinggi, khususnya pada kategori *Suspicious* dan *Benign*.

Pada tahap *stacking ensemble*, *Random Forest* dan *XGBoost* berperan sebagai *base learners* yang menganalisis fitur-fitur URL secara komprehensif. Kedua model menghasilkan prediksi probabilitas yang kemudian dikombinasikan oleh *meta-learner* untuk menghasilkan keputusan klasifikasi akhir. Pendekatan ini memungkinkan sistem untuk menangkap pola-pola kompleks yang tidak dapat diidentifikasi oleh aturan sederhana.

Keunggulan pendekatan hibrid ini terletak pada kemampuannya menggabungkan kecepatan dan kejelasan alasan *rule-based filtering* dengan adaptabilitas serta kemampuan pembelajaran dari *machine learning*. Selain itu, pendekatan ini juga berkontribusi dalam menekan *false positive* dan *false negative*. *Rule-based filtering* dapat dengan cepat mengidentifikasi *phishing* klasik, sementara *stacking ensemble* mampu mendeteksi teknik *phishing* yang lebih canggih dan terus berkembang.

Analisis kinerja sistem hibrid dilakukan dengan membandingkan hasil evaluasi terhadap tiga pendekatan sebelumnya, yaitu model *Random Forest* tunggal, *XGBoost* tunggal, dan *stacking ensemble* dengan *rule-based filtering*. Evaluasi mencakup metrik *accuracy*, *precision*, *recall*, *F1-score*, dan *AUC*, serta efisiensi komputasi yang diukur melalui waktu pemrosesan rata-rata per URL.

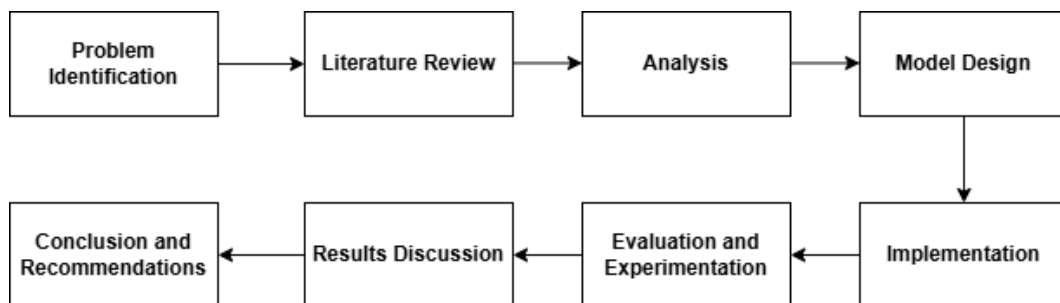
BAB IV

DESAIN

Pada bab ini menjelaskan desain sistem yang akan diimplementasikan dan metode penelitian yang akan diterapkan dalam penelitian ini. Desain sistem yang dimaksud meliputi beberapa aspek utama, meliputi penerapan metode *rule-based*, algoritma *Random Forest*, *XGBoost*, serta pendekatan *Stacking Ensemble* sebagai metode penggabungan.

4.1 Metode Penelitian

Metode Penelitian ini menggunakan serangkaian langkah - langkah, pengumpulan data serta instrumen penelitian yang akan digunakan untuk mencapai hasil. Langkah - langkah penelitian yang dilakukan dapat dilihat pada Gambar 4. 1 Tahapan Penelitian berikut.



Gambar 4. 1 Tahapan Penelitian

Berikut adalah penjelasan tahapan penelitian Tugas Akhir yang terdapat pada gambar diatas.

1. *Problem Identification*

Tahap pembentukan masalah merupakan hal utama yang ditentukan selama penelitian awal. Rumusan masalah adalah pernyataan khusus mengenai ruang lingkup masalah yang sedang dipelajari serta upaya untuk menyatakan pertanyaan mana yang akan dibahas.

2. *Literature Review*

Tahap studi literatur dilakukan untuk memperoleh pemahaman teoritis dan empiris terkait topik penelitian. Tahapan ini bertujuan untuk memperkuat

dasar teori, meninjau penelitian sebelumnya, dan menentukan pendekatan metodologis yang paling sesuai dengan tujuan penelitian.

3. *Analysis*

Pada tahap ini dilakukan analisis data dan metode yang digunakan. Metode pengumpulan informasi adalah dengan mencari dataset terkait topik penelitian ini di kaggle.com yang menyediakan data URL *phishing* dan *benign*. Data yang dikumpulkan meliputi berbagai atribut yang relevan seperti struktur URL, panjang domain, jumlah subdomain, dan umur domain.

4. *Model Design*

Pada tahap perancangan akan dilakukan proses perancangan arsitektur dan sistem yang akan dibangun berdasarkan hasil analisis data dan algoritma yang digunakan. Perancangan dirancang untuk dijadikan sebagai acuan dalam implementasi dan pengembangan sistem.

5. *Implementation*

Pada tahap ini, penulis akan menerapkan teknik- teknik yang telah dianalisis dan dirancang sebelumnya. Penulis akan melakukan implementasi yang melibatkan pendekatan *machine learning* terhadap desain yang di analisis untuk mendapatkan desain yang diharapkan. Eksperimen dilakukan untuk membandingkan kinerja model ensemble yang menggabungkan *Rule-Based Filtering*, *Random Forest*, dan *XGBoost* dengan model-model individual yang dijalankan secara terpisah.

6. *Evaluation and Experimentation*

Pada tahap ini, dilakukan evaluasi kinerja keberhasilan proyek dan hasil eksperimen untuk menilai performa model deteksi *phishing* yang dikembangkan. Pengukuran dilakukan menggunakan metrik akurasi, *precision*, *recall*, *F1-score*, dan *AUC* untuk melihat kemampuan model dalam membedakan URL *phishing* dan *benign*. Evaluasi juga membandingkan model tunggal dan model hybrid dengan *rule-based filtering* guna mengetahui efektivitas kombinasi metode dalam menurunkan kesalahan klasifikasi.

7. *Results Discussion*

Pada tahap ini berisi pembahasan dari hasil yang didapatkan dari evaluasi dan eksperimen terhadap model. Hasil dari eksperimen digunakan untuk menentukan konfigurasi model terbaik berdasarkan keseimbangan akurasi dan efisiensi deteksi.

8. *Conclusion and Recommendations*

Tahap ini dimuat untuk mengambil keputusan dari hasil evaluasi dan memberikan saran untuk pengembangan selanjutnya. Tahapan akhir penelitian ini adalah menyusun kesimpulan untuk menjawab efektivitas kombinasi *rule-based filtering* dengan algoritma *Random Forest* dan *XGBoost* dalam mendeteksi *phishing*.

4.2 Rancangan Sistem

Pada subbab ini dijelaskan rancangan sistem yang digunakan dalam penelitian untuk mendeteksi URL *phishing* dengan pendekatan *hybrid*. Sistem ini dirancang untuk mengintegrasikan komponen *rule-based Filtering* dengan model *machine learning*.

4.2.1 Desain Umum Sistem

Arsitektur sistem dirancang secara bertahap dan terstruktur untuk memastikan bahwa data yang digunakan memiliki kualitas yang baik serta proses deteksi dapat dilakukan secara akurat dan efisien. Secara umum, arsitektur ini mengombinasikan dua pendekatan utama, yaitu pendekatan *rule-based filtering* dan *machine learning* berbasis *ensemble*, yang saling melengkapi dalam mendeteksi URL *phishing*. Pendekatan *rule-based* berfungsi sebagai *filtering* awal yang cepat, sedangkan pendekatan *machine learning* digunakan untuk melakukan analisis yang lebih mendalam terhadap URL yang tidak dapat diputuskan secara tegas oleh aturan.

Tahap awal dalam arsitektur sistem adalah *Data Preparation*. Pada tahap ini, dataset URL dikumpulkan dari sumber yang relevan dan terpercaya sebagai bahan utama dalam proses pembelajaran model. Selanjutnya dilakukan analisis atribut dataset untuk memahami karakteristik setiap fitur, baik fitur berbasis struktur URL,

domain, maupun konten. Analisis ini bertujuan untuk memastikan bahwa atribut yang digunakan memiliki relevansi terhadap pola phishing serta mendukung proses ekstraksi fitur pada tahap berikutnya. Dengan adanya tahap *data preparation*, sistem memiliki dasar data yang terstruktur dan siap diproses lebih lanjut.

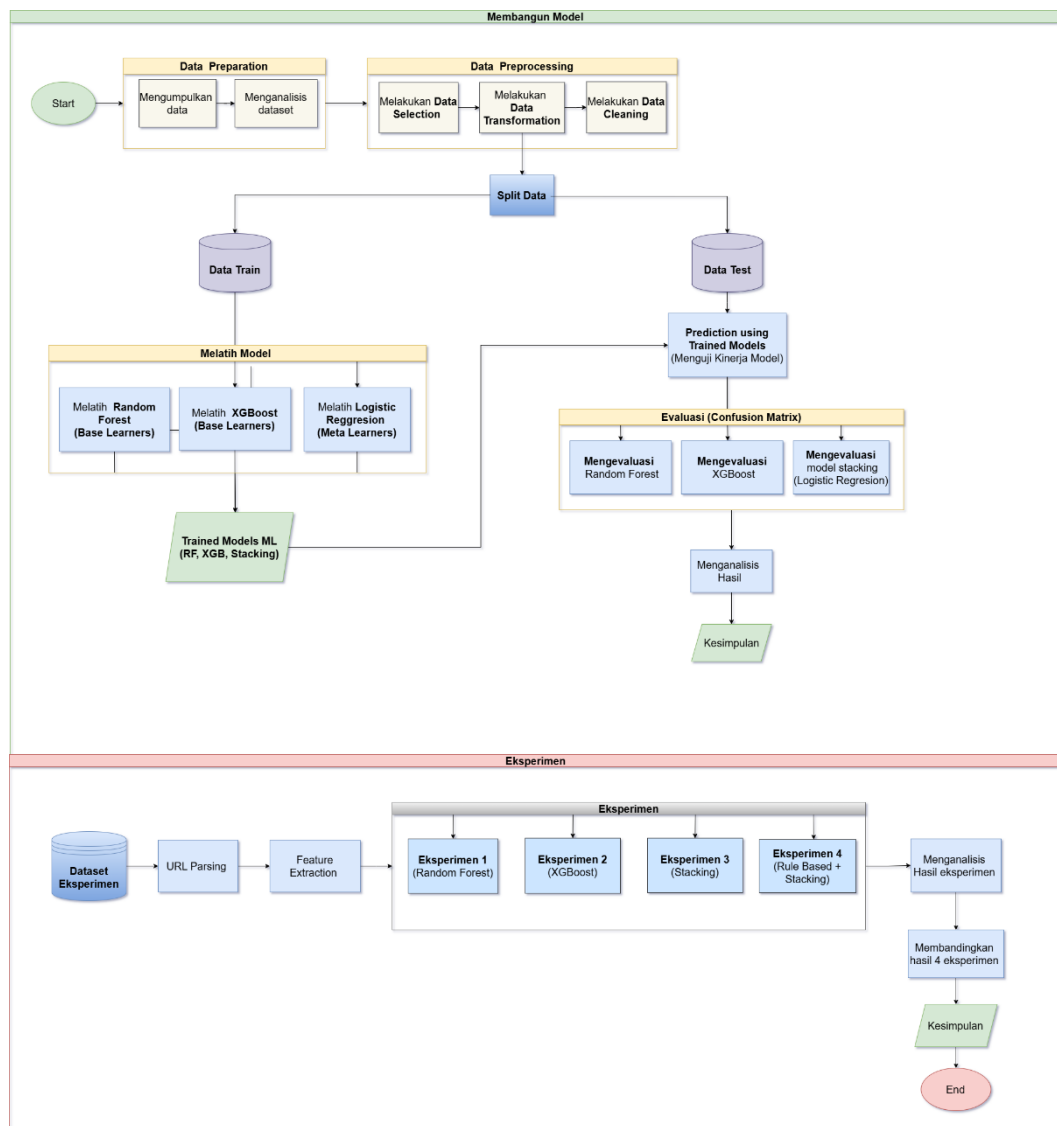
Setelah data dipersiapkan, sistem memasuki tahap *Data Preprocessing*. Tahap ini mencakup beberapa proses penting, yaitu *data selection*, dan *data cleaning*. *Data selection* dilakukan untuk memilih fitur-fitur yang paling relevan dalam mendeteksi phishing, sehingga dapat mengurangi kompleksitas model dan meningkatkan performa. *Data cleaning* bertujuan untuk menghapus data duplikat, data tidak lengkap, atau nilai yang tidak valid agar tidak mengganggu proses pembelajaran. Sehingga seluruh data dapat digunakan dalam satu format yang konsisten.

Dataset yang telah melalui tahap *preprocessing* kemudian dibagi pada tahap *Data Split* menjadi *training data* dan *testing data*. Data latih digunakan untuk membangun dan melatih model *machine learning*, sedangkan data uji digunakan untuk mengevaluasi performa model. Pada proses pelatihan, data latih dimasukkan ke dalam base learner yang terdiri dari algoritma *Random Forest* dan *XGBoost*. *Random Forest* memanfaatkan teknik *bagging* untuk meningkatkan stabilitas dan mengurangi *overfitting*, sedangkan *XGBoost* menggunakan teknik *boosting* yang berfokus pada perbaikan kesalahan prediksi secara bertahap. Hasil prediksi dari kedua *base learner* ini kemudian digabungkan menggunakan *meta learner* dengan pendekatan *stacking*, di mana algoritma *Logistic Regression* digunakan sebagai *meta-model* untuk menghasilkan keputusan akhir yang lebih optimal.

Pada tahap eksperimen atau pengujian, URL baru yang akan diuji terlebih dahulu diproses melalui *rule-based filtering*. Aturan-aturan ini dirancang berdasarkan karakteristik umum *phishing*, seperti panjang URL yang tidak wajar, penggunaan simbol mencurigakan, atau pola domain tertentu. Jika URL memenuhi aturan *phishing* secara jelas, maka sistem dapat langsung mengklasifikasikannya tanpa harus melalui model *machine learning*. Sebaliknya, jika URL tidak memenuhi aturan secara tegas, URL tersebut akan diteruskan ke model *machine learning*

untuk dianalisis lebih lanjut menggunakan *Random Forest*, *XGBoost*, dan *meta learner*.

Tahap akhir dari arsitektur ini adalah *hybridisasi*, yaitu penggabungan hasil dari *rule-based filtering* dan model *machine learning*. Pendekatan *hybrid* ini bertujuan untuk meningkatkan ketepatan deteksi, mengurangi kesalahan klasifikasi, serta menekan kemungkinan *false positive* dan *false negative*. *Output* akhir dari sistem berupa label biner, yaitu *phishing* atau *benign* (0-1), yang menjadi keputusan akhir dari proses deteksi. Berikut adalah Gambar 4. 2 Desain Umum Sistem.



Gambar 4. 2 Desain Umum Sistem

Diagram alur sistem menggunakan notasi *flowchart* standar untuk merepresentasikan proses deteksi *phishing* URL secara sistematis. Proses dimulai dari simbol terminator (*Start*) yang menandai awal sistem. Tahap pertama adalah *data preparation*, yang mencakup kegiatan mengumpulkan data dan menganalisis dataset untuk memahami karakteristik serta kualitas data yang akan digunakan. Setelah itu, data memasuki tahap *data preprocessing* yang terdiri dari beberapa proses utama, yaitu *data selection*, *data transformation*, dan *data cleaning*. Tahap ini bertujuan menyeleksi fitur yang relevan, membangun fitur baru yang lebih informatif, mentransformasikan data ke bentuk yang sesuai, serta membersihkan data dari nilai yang tidak valid atau tidak konsisten sehingga dataset siap digunakan dalam proses pemodelan.

Dataset yang telah diproses kemudian disimpan dan selanjutnya dilakukan pembagian data (*split data*) menjadi dua bagian utama, yaitu *data train* dan *data test*. *Data train* digunakan pada tahap pelatihan model, di mana sistem melatih beberapa algoritma *machine learning* yang terdiri dari *Random Forest* dan *XGBoost* sebagai *base learners*, serta *Logistic Regression* sebagai *meta learner* dalam pendekatan model *stacking*. Hasil dari proses ini adalah kumpulan trained models yang mencakup model *Random Forest*, *XGBoost*, *stacking*, serta kombinasi model *rule-based* dengan *machine learning*.

Selanjutnya, *data test* digunakan pada tahap *prediction using trained models* untuk menguji kinerja model yang telah dilatih. Hasil prediksi kemudian dievaluasi menggunakan *confusion matrix* melalui beberapa eksperimen, yaitu evaluasi *Random Forest*, *XGBoost*, model *stacking*, serta model *rule-based* yang dikombinasikan dengan *stacking*. Setiap hasil eksperimen kemudian dianalisis pada tahap analisis hasil dan *benchmarking* untuk membandingkan performa masing-masing model. Berdasarkan hasil analisis tersebut, sistem menghasilkan kesimpulan mengenai model yang memiliki performa terbaik dalam mendeteksi *phishing* URL, yang kemudian menandai akhir dari proses sistem pada simbol terminator (*End*).

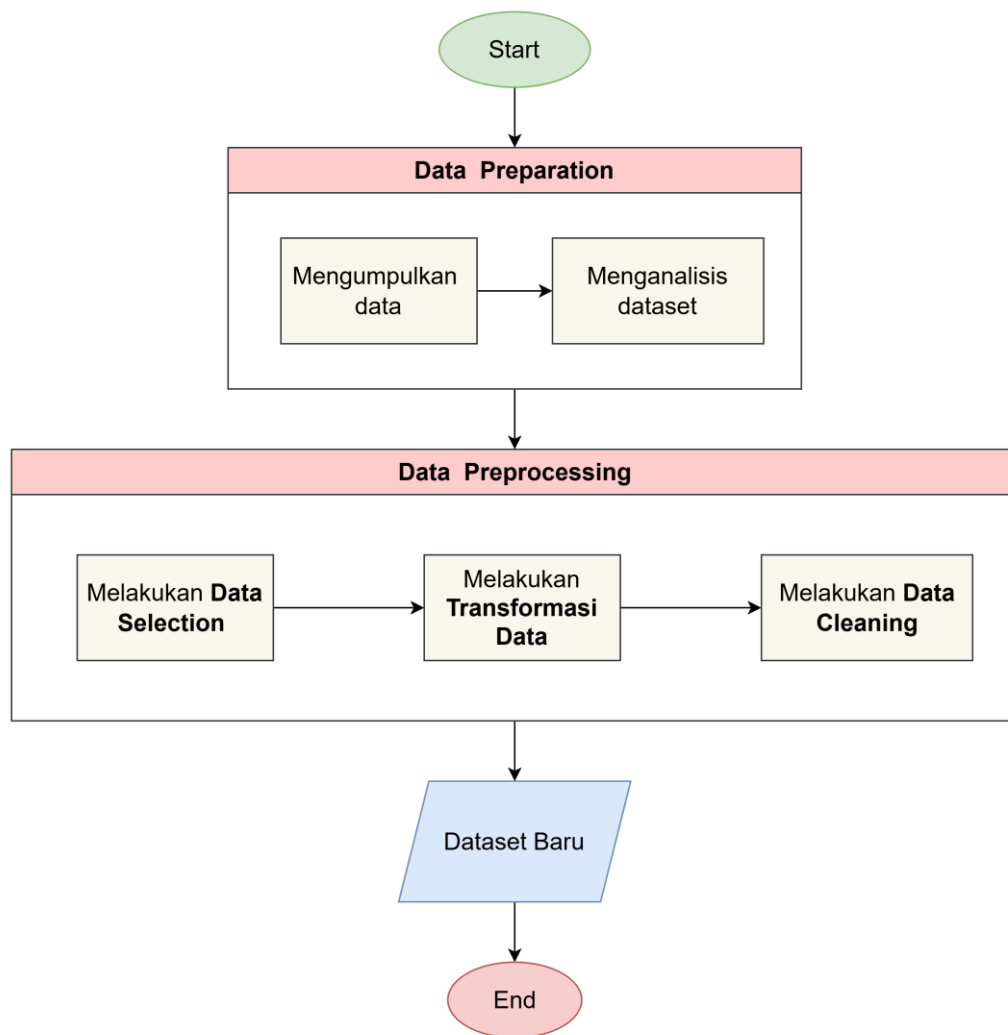
4.3 Desain Analisis Dataset

Desain Analisis Dataset merupakan kerangka kerja sistematis yang digunakan untuk mengelola dan menyiapkan data URL agar dapat digunakan secara optimal pada tahap pemodelan sistem deteksi *phishing*. Perancangan analisis dataset bertujuan untuk memastikan bahwa data yang digunakan telah melalui tahapan pengolahan yang sesuai, memiliki kualitas yang baik, serta relevan dengan tujuan penelitian.

Proses analisis dataset diawali dengan tahap *Data Preparation*, yang mencakup kegiatan pengumpulan dataset URL *phishing* dan *legitimate* dari platform Kaggle, serta analisis awal terhadap struktur dataset. Pada tahap ini dilakukan identifikasi jumlah data, jumlah atribut, serta karakteristik fitur yang tersedia guna memperoleh pemahaman menyeluruh terhadap dataset sebelum dilakukan pengolahan lebih lanjut.

Selanjutnya, proses dilanjutkan dengan tahap *Data Preprocessing* sebagai upaya penyiapan data agar berada dalam format yang seragam dan siap digunakan dalam proses pemodelan. Tahap *Data Preprocessing* terdiri dari beberapa sub-tahapan, yaitu *Data Selection*, *Data Transformation*, dan *Data Cleaning*. *Data Selection* dilakukan untuk memilih fitur-fitur yang relevan dengan kebutuhan deteksi *phishing*. *Data Transformation* digunakan untuk menyesuaikan format data, termasuk konversi label kelas ke dalam bentuk numerik. Sementara itu, *Data Cleaning* dilakukan untuk menghapus data duplikat serta fitur atau nilai yang tidak informatif sehingga dapat meningkatkan kualitas dataset.

Keseluruhan rangkaian tahapan analisis dataset menghasilkan dataset akhir yang bersih, terstruktur, dan konsisten, sehingga layak dan valid untuk digunakan pada tahap pemodelan *machine learning*. Desain alur Analisis Dataset pada penelitian ini ditampilkan pada Gambar 4. 3, yang menggambarkan hubungan antar tahapan pengolahan data secara menyeluruh.



Gambar 4. 3 Desain Analisis Dataset

4.3.1 Desain Data Preparation

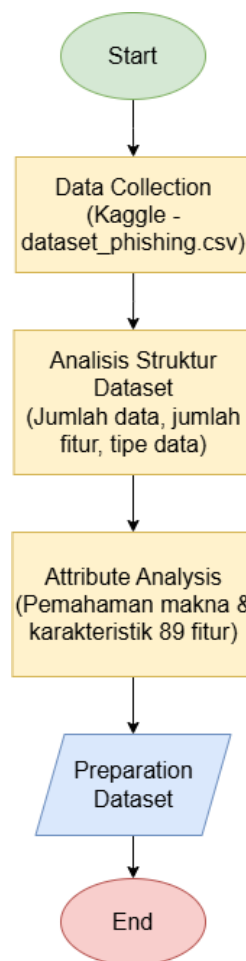
Tahap *Data Preparation* pada penelitian ini merupakan proses persiapan data sebelum digunakan pada tahap pemodelan dan evaluasi sistem deteksi *phishing*. Pada tahap ini, fokus utama kegiatan meliputi pengumpulan dataset URL, analisis atribut yang terdapat dalam dataset, serta penyesuaian struktur data agar siap digunakan dalam proses pengolahan dan pemodelan selanjutnya.

Proses *Data Preparation* dimulai dengan pengambilan dataset mentah URL *phishing* dan non-*phishing* dari *platform* Kaggle. Dataset yang digunakan memiliki total 11.431 data URL dengan 89 atribut fitur, yang mencerminkan berbagai karakteristik URL seperti struktur domain, penggunaan simbol khusus, serta elemen

pendukung lainnya. Dataset tersebut telah dilengkapi dengan label klasifikasi yang membedakan antara URL *phishing* dan URL *legitimate*.

Setelah proses pengumpulan data, dilakukan analisis terhadap atribut–atribut yang terdapat pada dataset untuk memahami karakteristik masing-masing fitur, termasuk struktur kolom, tipe data, dan relevansinya terhadap tujuan penelitian. Analisis ini bertujuan untuk memastikan bahwa setiap atribut dipahami dengan baik sebelum dilakukan proses pengolahan data lebih lanjut, seperti *preprocessing*, seleksi fitur, dan penerapan metode *rule-based filtering* serta *machine learning*.

Desain alur *Data Preparation* pada penelitian ini ditampilkan pada Gambar 4. 4, yang menggambarkan tahapan pengambilan data, analisis atribut, dan kesiapan dataset sebelum memasuki tahap pemodelan sistem deteksi *phishing*.



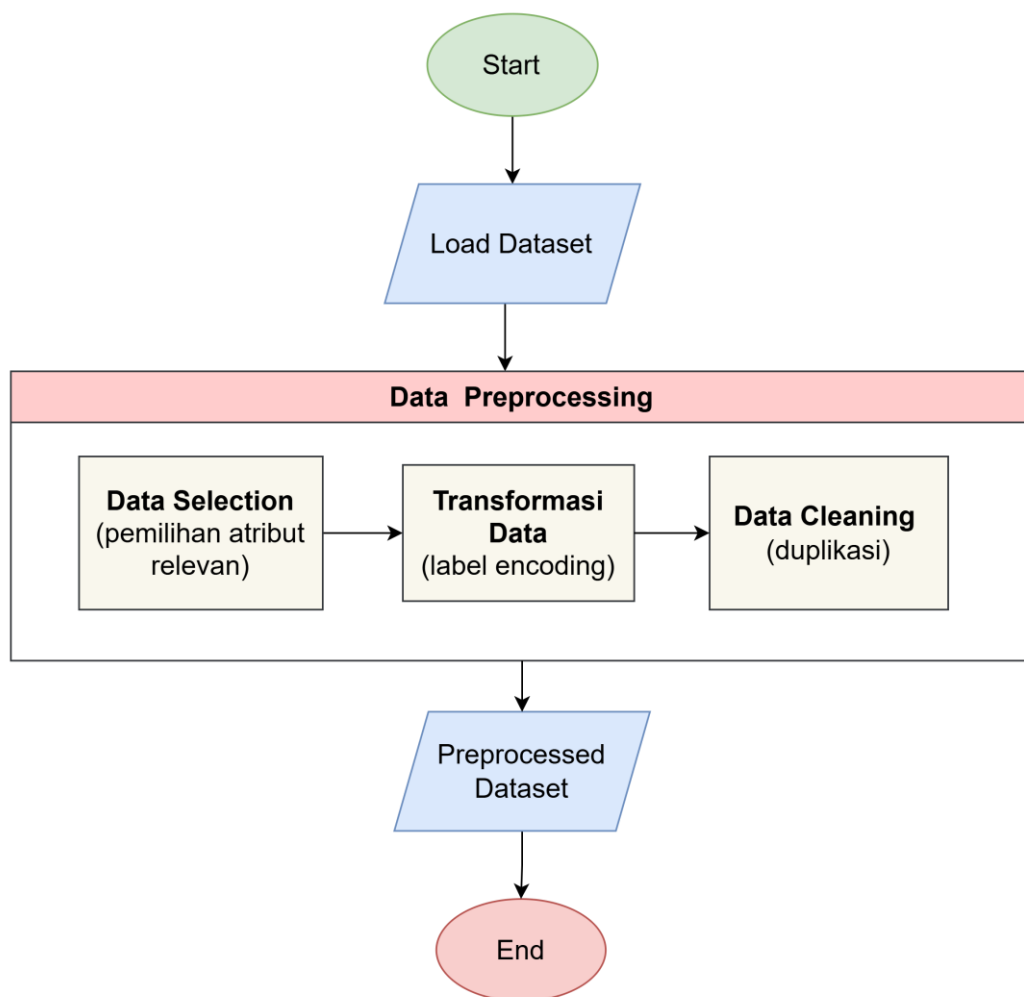
Gambar 4. 4 Desain *Data Preparation*

4.3.2 Desain Data Preprocessing

Desain *Data Preprocessing* pada penelitian ini digunakan untuk menyiapkan data URL agar berada dalam kondisi optimal sebelum diproses oleh sistem deteksi phishing. Proses diawali dengan tahap *Load Dataset*, di mana dataset yang berisi URL *phishing* dan *non-phishing* dimuat dari sumbernya untuk kemudian diproses lebih lanjut.

Tahap selanjutnya adalah *Data Selection* yaitu pemilihan atribut relevan, dilakukan proses seleksi terhadap atribut-atribut yang relevan dan memiliki kontribusi signifikan terhadap proses deteksi *phishing*. Pada tahap ini, hanya fitur-fitur yang berkaitan dengan karakteristik URL *phishing* yang dipilih, seperti panjang URL, struktur domain, keberadaan karakter khusus, penggunaan protokol HTTPS, dan pola URL yang mencurigakan. Tahap berikutnya adalah *Transformasi Data (label encoding)*, yaitu proses konversi data kategorikal menjadi format numerik yang dapat diterima oleh algoritma *machine learning*. Pada tahap ini, label kelas yaitu *phishing* atau *legitimate* diencode menjadi nilai numerik, misalnya 0 untuk *benign* dan 1 untuk *phishing*. Setelah itu dilakukan *Data Cleaning* (duplikasi), yaitu proses penghapusan data duplikat untuk menghindari bias dalam pembelajaran model atau penghapusan baris.

Setelah seluruh tahapan *preprocessing* selesai, dihasilkan *Preprocessed Dataset* yang telah bersih, terstruktur, dan siap untuk digunakan dalam proses *training* dan *testing* model deteksi *phishing*. Berikut ditampilkan Gambar 4. 5 yang menggambarkan desain proses *data preprocessing*.



Gambar 4. 5 Desain *Data Preprocessing*

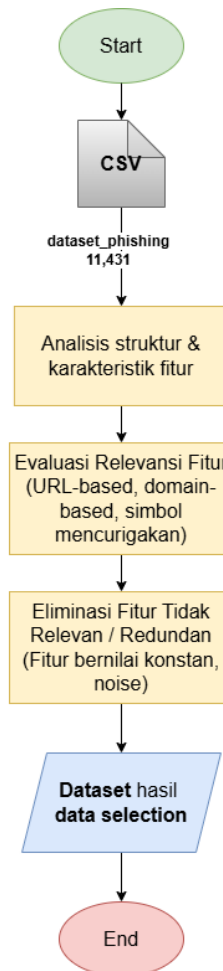
4.3.2.1 Desain Data Selection

Desain *Data Selection* pada penelitian ini bertujuan untuk menentukan atribut-atribut yang relevan dalam mendukung proses deteksi website phishing. Tahapan ini dirancang untuk memastikan bahwa hanya fitur-fitur yang memiliki keterkaitan langsung dengan karakteristik phishing yang digunakan pada tahap pengolahan data selanjutnya, sehingga proses analisis menjadi lebih efisien dan terfokus.

Data Selection dilakukan setelah dataset berhasil dimuat dan sebelum tahap pembersihan data, dengan mempertimbangkan struktur serta karakteristik awal dataset. Pada tahap ini, dilakukan identifikasi terhadap seluruh atribut yang tersedia, kemudian dipilih fitur-fitur yang mampu merepresentasikan pola *phishing*, seperti karakteristik URL, struktur domain, serta penggunaan simbol-simbol yang bersifat

mencurigakan. Atribut yang tidak memberikan kontribusi signifikan terhadap proses klasifikasi atau bersifat redundan dieliminasi dari dataset.

Hasil dari tahapan *Data Selection* berupa dataset dengan jumlah fitur yang lebih ringkas dan informatif, tanpa mengurangi kemampuan representasi karakteristik phishing. Dataset hasil seleksi fitur ini selanjutnya digunakan sebagai masukan pada proses *rule-based filtering* serta pada tahap pelatihan model *machine learning*, yaitu *Random Forest* dan *XGBoost*. Desain alur *Data Selection* pada penelitian ini ditampilkan pada Gambar 4. 6.



Gambar 4. 6 Desain *Data Selection*

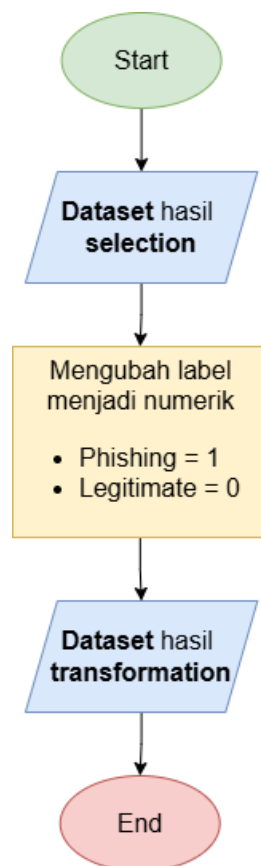
4.3.2.2 Desain Transformation Data

Desain *Data Transformation* pada penelitian ini bertujuan untuk mengubah format data agar sesuai dengan kebutuhan algoritma *machine learning* yang digunakan.

Tahapan ini difokuskan pada penyesuaian representasi data tanpa mengubah informasi dasar yang terkandung di dalam dataset.

Proses *Data Transformation* diawali dengan penggunaan dataset hasil *Data Selection* sebagai masukan. Pada tahap ini dilakukan konversi label kelas dari bentuk kategorikal menjadi bentuk numerik, sehingga dapat diproses oleh algoritma klasifikasi. Label *phishing* dikonversi menjadi nilai 1, sedangkan label *legitimate* dikonversi menjadi nilai 0.

Hasil dari tahapan *Data Transformation* adalah dataset hasil *transformation* yang telah memiliki label numerik dan format data yang sesuai untuk digunakan pada tahap *Data Cleaning* dan proses pemodelan selanjutnya. Desain alur *Data Transformation* pada penelitian ini ditampilkan pada Gambar 4. 7.

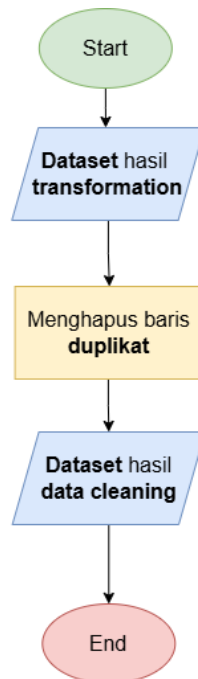


Gambar 4. 7 Desain *Transformation Data*

4.3.2.3 Desain Data Cleaning

Pada penelitian ini, proses *data cleaning* dilakukan setelah tahap *transformation data*. Proses *data cleaning* pada penelitian ini hanya difokuskan pada penanganan data duplikat yang terdapat pada dataset.

Desain *data cleaning* ini dilakukan untuk mengidentifikasi dan mengatasi data duplikat yang dapat menyebabkan pengulangan informasi dalam dataset. Dengan menangani data duplikat, dataset menjadi lebih rapi dalam kondisi data yang sebenarnya. Proses ini memastikan bahwa setiap data yang digunakan bersifat unik, sehingga dapat membantu meningkatkan keakuratan proses pelatihan dan evaluasi model klasifikasi. Berikut adalah Gambar 4. 8 dari desain *data cleaning*.



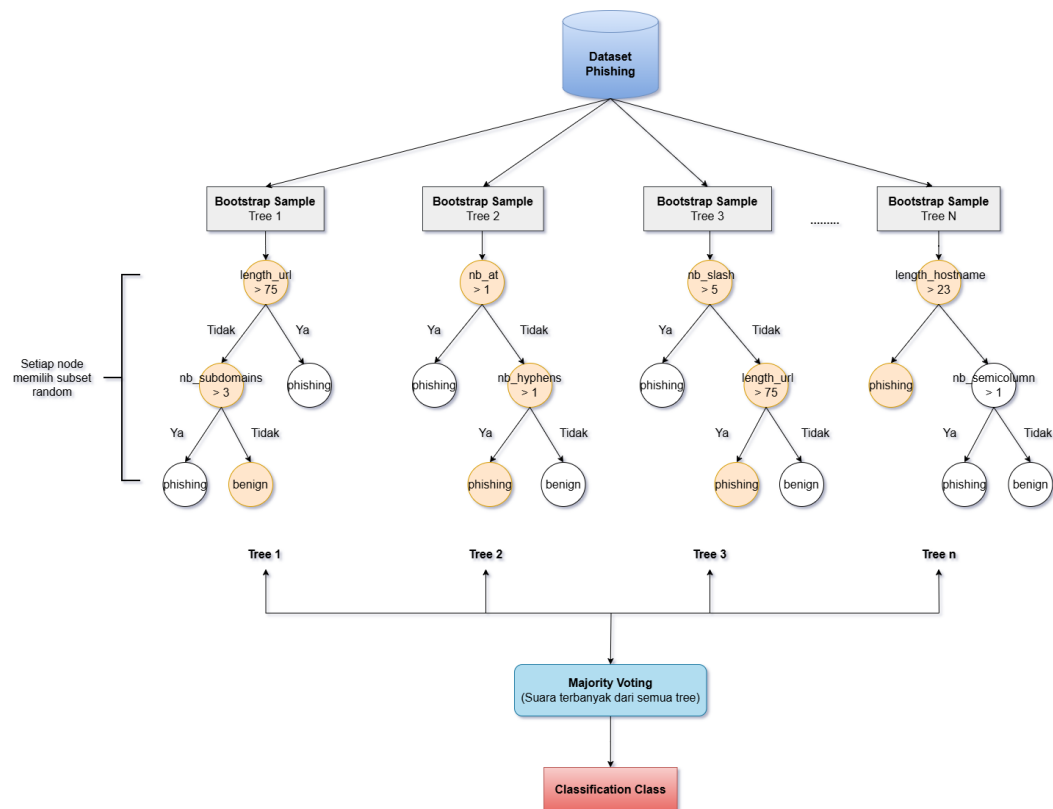
Gambar 4. 8 Desain *Data Cleaning*

4.4 Rancangan Arsitektur Machine Learning

Subbab ini merupakan desain dan struktur dari dua model *machine learning* yang akan digunakan, yaitu *Random Forest* dan *XGBoost*. Rancangan arsitektur menggambarkan bagaimana model bekerja, mulai dari menerima data *input*, memproses data, hingga menghasilkan prediksi.

4.4.1 Rancangan Arsitektur Random Forest

Desain arsitektur *Random Forest* untuk klasifikasi phishing URL ditunjukkan pada Gambar 4. 9. Arsitektur ini menggambarkan bagaimana model *Random Forest* memanfaatkan teknik *ensemble learning* dengan membangun banyak *decision tree*. Proses klasifikasi diawali dengan penggunaan dataset *phishing* yang telah melalui tahap *preparation* dan *preprocessing*, sehingga setiap URL direpresentasikan dalam bentuk fitur-fitur *numerik* yang relevan untuk proses pembelajaran model.



Gambar 4. 9 Rancangan Arsitektur *Random Forest*

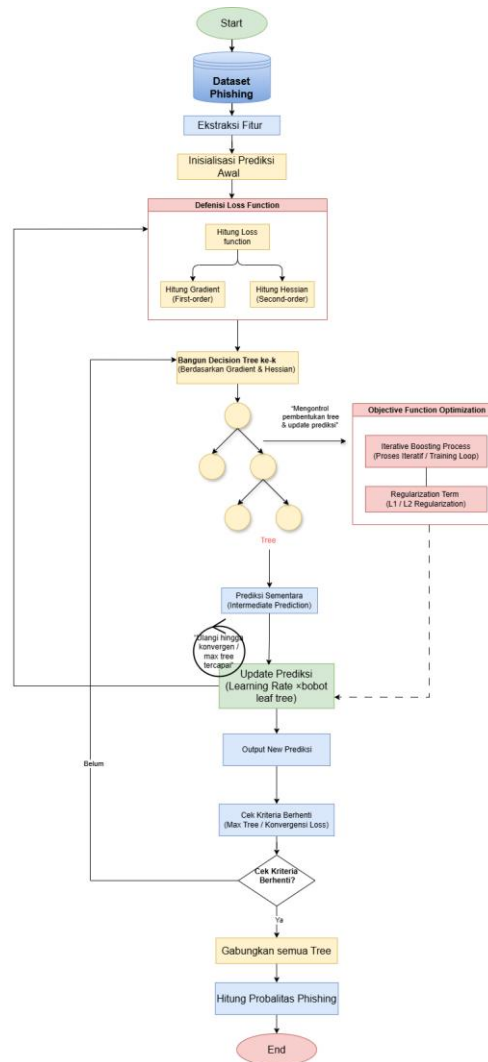
1. Dataset yang telah diproses kemudian dibagi menjadi beberapa subset data menggunakan teknik *bootstrap sampling*, yaitu pengambilan sampel secara acak dengan pengembalian dari dataset asli. Setiap subset *bootstrap* digunakan untuk membangun satu decision tree yang bersifat *independen*. Dengan mekanisme ini, terbentuk sejumlah *tree* seperti *Tree 1*, *Tree 2*, *Tree 3*, hingga *Tree N*, sesuai dengan jumlah estimator yang ditentukan pada model *Random Forest*.

Setelah tahap *bootstrap sampling*, setiap *decision tree* dibangun secara paralel dengan menerapkan pemilihan subset fitur secara acak pada setiap node. Fitur-fitur yang digunakan dalam proses pemisahan meliputi *length_url*, *nb_at*, *nb_slash*, *length_hostname*, *nb_subdomains*, *nb_hyphens*, *nb_semicolumn*, serta fitur-fitur lain yang telah melalui proses *feature selection*. Pada setiap node internal, algoritma *Random Forest* akan memilih fitur terbaik dari subset fitur acak tersebut berdasarkan kriteria *Gini* atau *entropy* untuk memisahkan data. Proses pemisahan ini berlangsung sehingga mencapai kondisi berhenti, seperti batas kedalaman maksimum *tree* atau jumlah minimum sampel pada *leaf node*, yang kemudian menghasilkan keputusan klasifikasi berupa *phishing* atau *benign*.

Setelah seluruh *decision tree* menghasilkan prediksi masing-masing, hasil tersebut digabungkan menggunakan mekanisme *majority voting*. Pada tahap ini, setiap *tree* memberikan satu suara terhadap kelas tertentu, dan kelas dengan jumlah suara terbanyak dipilih sebagai hasil klasifikasi akhir. Pendekatan *ensemble* ini membuat *Random Forest* memiliki tingkat akurasi dan kestabilan yang lebih baik, karena mampu mengurangi *overfitting* serta meningkatkan kemampuan generalisasi model terhadap data baru yang belum pernah dilihat sebelumnya.

4.4.2 Rancangan Arsitektur XGBoost

Desain arsitektur *XGBoost* untuk klasifikasi phishing URL ditunjukkan pada Gambar 4. 10. Arsitektur ini menggambarkan bagaimana model *XGBoost* memanfaatkan teknik *gradient boosting* dengan membangun *decision tree* secara sekuensial untuk meningkatkan akurasi prediksi. Proses klasifikasi diawali dengan penggunaan dataset *phishing* yang telah melalui tahap *preparation* dan *preprocessing*, sehingga setiap URL direpresentasikan dalam bentuk fitur-fitur *numerik* yang relevan untuk proses pembelajaran model.



Gambar 4. 10 Rancangan Arsitektur *XGBoost*

Proses deteksi phishing diawali dengan tahap *Load Dataset*, di mana kumpulan data yang berisi sampel URL atau aktivitas jaringan yang telah memiliki label (*phishing* atau *benign*) dimuat ke dalam sistem. Setelah data tersedia, dilakukan tahap ekstraksi fitur untuk mengubah informasi mentah menjadi variabel numerik yang dapat diproses oleh mesin. Langkah selanjutnya adalah melakukan inisialisasi prediksi awal, di mana model menetapkan nilai dasar atau nilai prediksi yang sama mewakili label yang paling dominan untuk seluruh data. Tahapan ini menjadi titik awal yang digunakan dalam perhitungan selanjutnya.

Setelah inisialisasi, sistem masuk ke tahap inti algoritma *XGBoost*, yaitu penghitungan *Gradient* dan Residual. Pada tahap ini, *Gradient* menunjukkan arah

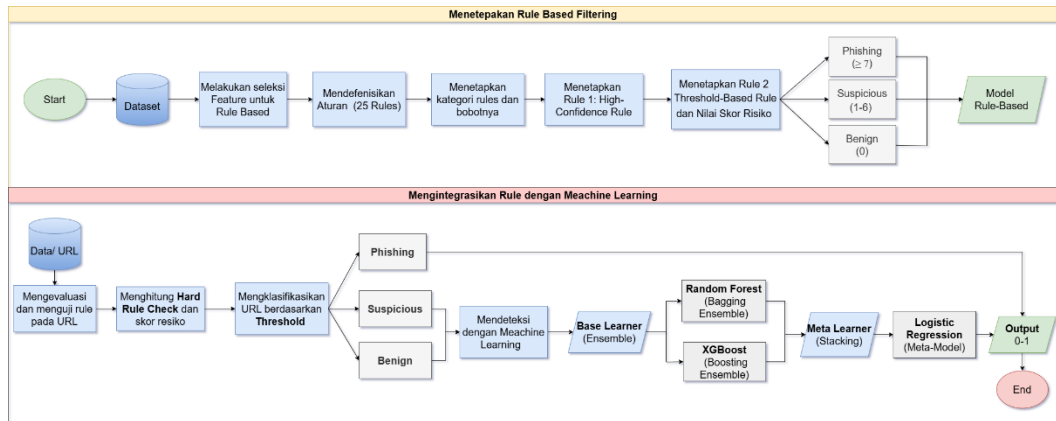
dan ukuran perubahan yang diperlukan untuk mengurangi kesalahan tersebut. Residual mengukur selisih antara nilai aktual dengan prediksi saat ini untuk menemukan tingkat kesalahan yang perlu diperbaiki. Kesalahan tersebut kemudian digunakan sebagai landasan untuk tahap *Build Tree* atau pembangunan pohon keputusan. Pohon ini dibangun secara sekuensial atau berurutan untuk mengoreksi kesalahan (residual). Pada setiap langkah, pohon baru ditambahkan untuk meminimalkan kesalahan residual. Selama proses ini, diterapkan pula *L1/L2 Regularization* untuk menjaga kompleksitas pohon agar model tidak mengalami *overfitting* dan tetap mampu menggeneralisasi data baru dengan baik.

Setiap hasil dari pembangunan pohon baru tidak langsung ditambahkan secara utuh, melainkan dikalikan terlebih dahulu dengan *Learning Rate* pada tahap *update* prediksi. Hal ini dilakukan agar proses belajar model berjalan secara bertahap dan lebih stabil. Melalui proses *iteratif*, langkah-langkah penghitungan residual dan pembangunan pohon diulang hingga mencapai jumlah pohon yang ditentukan atau tingkat kesalahan yang minimum. Pada tahap akhir, sistem akan menggabungkan semua *tree* yang telah terbentuk menjadi satu model prediksi yang komprehensif. Hasil penggabungan ini kemudian dikonversi melalui tahap hitung probabilitas *phishing* untuk menghasilkan nilai akhir yang menentukan apakah suatu data termasuk kategori *phishing* atau *benign*.

4.5 Rancangan Arsitektur Rule based Filtering

Sistem deteksi URL *phishing* pada penelitian ini menggabungkan pendekatan *hybrid* antara rule-based filtering dan machine learning. Proses dimulai dengan ekstraksi fitur dari URL untuk merepresentasikan karakteristik struktural dan pola dalam bentuk *numerik*. Fitur tersebut kemudian dievaluasi melalui *rule-based filtering* dengan aturan logis untuk mengidentifikasi *phishing*. URL yang memenuhi kriteria yang jelas langsung diklasifikasikan sebagai *phishing* atau *benign*. URL yang tidak dapat dipastikan dikategorikan sebagai *suspicious*. Selanjutnya, URL yang termasuk kategori *suspicious* dan *benign* diproses menggunakan machine learning berbasis ensemble, dengan menggunakan Random Forest dan XGBoost. Prediksi dari kedua model ini digabungkan melalui stacking

dengan Logistic Regression sebagai meta-learner untuk memberikan klasifikasi akhir. Dengan adanya kategori tambahan **benign**, sistem ini meningkatkan akurasi dan efisiensi dalam mendeteksi *phishing*. Berikut adalah Gambar 4. 11 dari Rancangan Arsitektur *Rule based Filtering*.



Gambar 4. 11 Rancangan Arsitektur *Rule based Filtering*

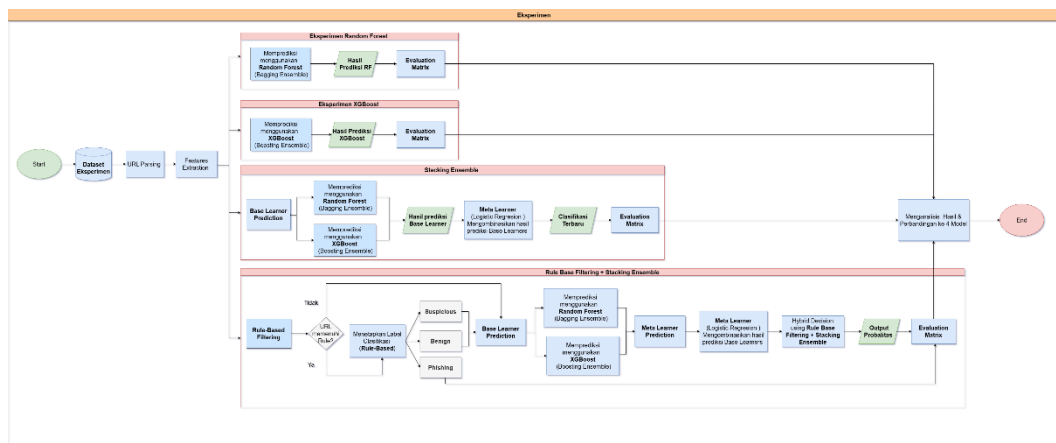
4.6 Rancangan Arsitektur Eksperimen

Rancangan arsitektur eksperimen pada Gambar 4. 12 disusun untuk mengevaluasi kinerja berbagai pendekatan dalam mendeteksi URL phishing. Proses eksperimen diawali dengan penggunaan dataset yang telah melalui tahapan data preparation, data preprocessing, dan *feature extraction*, sehingga data berada dalam kondisi siap digunakan pada seluruh skenario pengujian. Tahapan awal ini bertujuan untuk memastikan bahwa setiap skenario eksperimen menggunakan data dengan struktur dan karakteristik yang sama, sehingga hasil evaluasi yang diperoleh dapat dibandingkan secara adil dan objektif.

Eksperimen dilakukan melalui empat skenario pengujian. Skenario pertama menggunakan model Random Forest sebagai base learner dengan pendekatan bagging ensemble. Skenario kedua menggunakan model XGBoost sebagai *base learner* dengan pendekatan *boosting ensemble*. Skenario ketiga menerapkan metode *stacking ensemble*, di mana Random Forest dan XGBoost berperan sebagai *base learner*, sedangkan Logistic Regression digunakan sebagai *meta-learner* untuk mengombinasikan hasil prediksi dari kedua model tersebut. Skenario

keempat mengombinasikan rule-based filtering dengan metode stacking ensemble, di mana URL yang memenuhi aturan tertentu akan diklasifikasikan secara langsung oleh sistem berbasis aturan, sedangkan URL yang tidak memenuhi aturan tersebut diproses lebih lanjut menggunakan metode stacking ensemble.

Pada seluruh skenario eksperimen, setiap URL akan menghasilkan satu label prediksi akhir sesuai dengan pendekatan yang digunakan pada masing-masing skenario. Khusus pada skenario keempat, label prediksi akhir diperoleh dari dua sumber, yaitu hasil klasifikasi *rule-based filtering* untuk URL yang memenuhi aturan dan hasil prediksi *stacking ensemble* untuk URL yang tidak memenuhi aturan. Seluruh label prediksi akhir tersebut kemudian dibandingkan dengan label asli pada dataset untuk membentuk *confusion matrix*. Berdasarkan confusion matrix tersebut, dilakukan perhitungan metrik evaluasi berupa *accuracy*, *precision*, *recall*, dan *F1-score* menggunakan rumus yang telah dijelaskan pada Subbab 2.8. Hasil perhitungan metrik evaluasi ini selanjutnya digunakan untuk menganalisis serta membandingkan efektivitas masing-masing pendekatan dalam mendeteksi URL *phishing*.



Gambar 4. 12 Desain Eksperimen

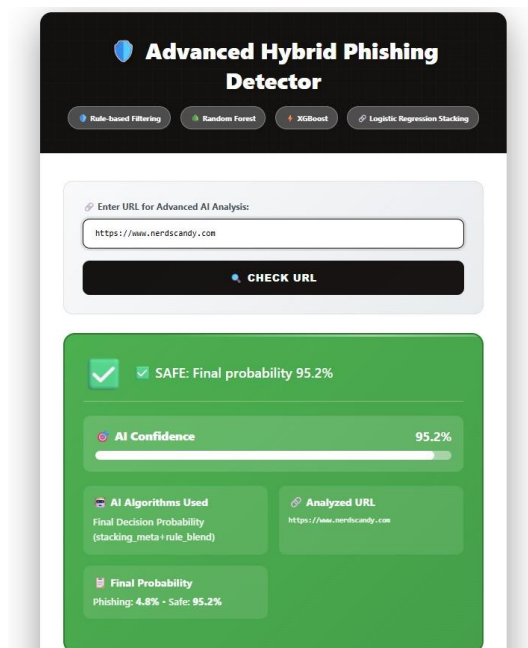
4.7 Mockup Desain Sistem

Mockup desain sistem merupakan gambaran awal tampilan antarmuka dari sistem deteksi *phishing* yang dikembangkan. *Mockup* ini bertujuan untuk menunjukkan bagaimana pengguna akan berinteraksi dengan sistem sebelum sistem benar-benar diimplementasikan. Pada penelitian ini, *mockup* dibuat untuk menggambarkan tiga

kondisi utama sistem, yaitu saat URL dinyatakan *benign* (aman), saat sistem sedang melakukan proses analisis, dan saat URL terdeteksi sebagai *phishing*.

4.7.1 Mockup Sistem Ketika URL Benign

Pada Gambar 4. 13, sistem menampilkan hasil analisis bahwa URL yang dimasukkan oleh *user* termasuk kategori aman (*benign*). Setelah *user* memasukkan URL pada kolom *input* dan menekan tombol Check URL, sistem akan melakukan analisis menggunakan model *machine learning* yang telah dibangun.

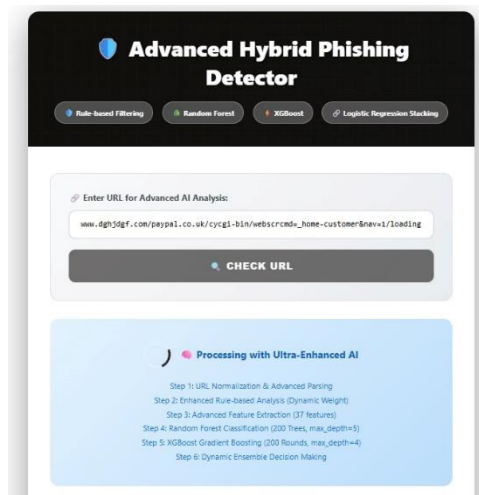


Gambar 4. 13 Mockup Desain Sistem ketika Benign

Jika hasil analisis menunjukkan bahwa URL tersebut aman, maka sistem akan menampilkan notifikasi berwarna hijau dengan informasi SAFE beserta nilai probabilitas akhir hasil prediksi model. Selain itu, sistem juga menampilkan beberapa informasi tambahan seperti tingkat *AI Confidence*, algoritma yang digunakan dalam proses analisis, serta URL yang dianalisis. Tampilan ini memberikan kejelasan kepada *user* bahwa tautan yang diperiksa tidak terindikasi sebagai *phishing*.

4.7.2 Mockup Sistem Saat Proses Analisis

Mockup Gambar 4. 14 menunjukkan kondisi ketika sistem sedang melakukan proses analisis terhadap URL yang dimasukkan oleh *user*. Pada tahap ini, sistem menampilkan indikator *processing* yang menunjukkan bahwa URL sedang diproses oleh algoritma *machine learning*.

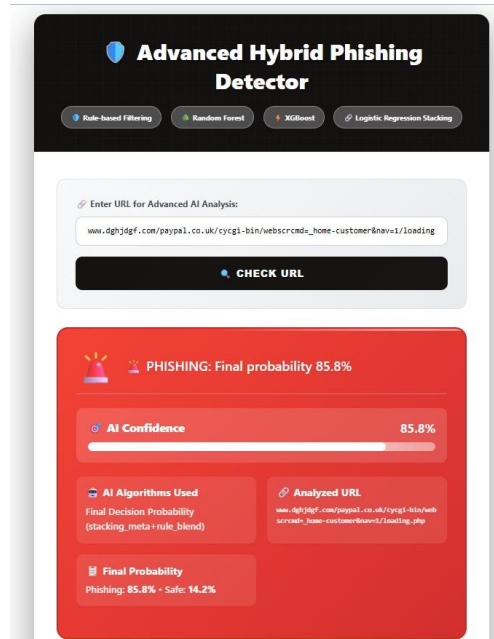


Gambar 4. 14 Mockup Desain Sistem saat Proses

Pada bagian ini ditampilkan tahapan analisis yang dilakukan oleh sistem, seperti ekstraksi fitur dari URL, pemrosesan menggunakan algoritma. Tampilan ini bertujuan memberikan proses kepada *user* bahwa sistem sedang melakukan analisis sebelum menghasilkan keputusan akhir.

4.7.3 Mockup Sistem Ketika URL Phishing

Mockup Gambar 4. 15 menunjukkan kondisi ketika sistem mendeteksi bahwa URL yang dimasukkan termasuk kategori *phishing*. Jika hasil analisis menunjukkan adanya indikasi *phishing*, maka sistem akan menampilkan notifikasi berwarna merah dengan label PHISHING beserta nilai probabilitas akhir dari hasil prediksi model.



Gambar 4. 15 Mockup Desain Sistem ketika Phishing

Selain itu, sistem juga menampilkan informasi tambahan seperti tingkat *AI Confidence*, algoritma yang digunakan dalam proses analisis, serta URL yang dianalisis. Tampilan ini dirancang untuk memberikan peringatan yang jelas kepada *user* agar tidak mengakses atau mempercayai tautan tersebut karena berpotensi membahayakan keamanan data *user*.

BAB V

IMPLEMENTASI

Pada bab ini dijelaskan bentuk fisik dari hasil analisis yang telah dijelaskan pada bab analisis dan bab desain. Hal-hal yang akan dijelaskan pada bab ini adalah lingkungan implementasi, batasan implementasi, implementasi eksperimen dan implementasi evaluasi.

5.1 Lingkungan Implementasi

Lingkungan implementasi terdiri dari perangkat keras (*hardware*) dan perangkat lunak (*software*) yang digunakan dalam proses implementasi. Berikut ini sub bab spesifikasi dari perangkat keras dan perangkat lunak yang digunakan dalam proses implementasi penelitian ini.

5.1.1 Perangkat Keras

Perangkat keras yang digunakan untuk melakukan proses implementasi memiliki spesifikasi pada Tabel 5. 1 sebagai berikut.

Tabel 5. 1 Perangkat keras (Hardware)

No	Hardware	Specifications
1.	Processor	Intel core i5
2.	RAM (Random Access Memory)	8 GB

5.1.2 Perangkat Lunak

Berikut ini spesifikasi dari *software* yang diperlukan dalam melakukan penelitian ini dapat dilihat pada Tabel 5. 2 berikut.

Tabel 5. 2 Perangkat Lunak (Software)

No	Software	Specifications
1.	Operating System	Windows 11
2.	Programming Language	Python
3.	Browser	Google Chrome
4.	Text Editor	Visual Studio Code

5.2 Batasan Implementasi

Adapun batasan yang ditentukan dalam implementasi penelitian ini adalah:

- a. Implementasi yang dilakukan menggunakan bahasa pemrograman Python.
- b. Implementasi setiap tahapan penelitian dilakukan di Visual Studio Code.

5.3 Implementasi Analisis Dataset

Pada subbab implementasi ini, dataset Kaggle berformat CSV (*Comma-Separated Values*) yang telah diperoleh dari repositori publik dimuat lalu dianalisis menggunakan bahasa pemrograman Python melalui *Visual Studio Code*.

5.3.1 Implementasi *Data Preparation*

Implementasi *data preparation* dilakukan untuk mempersiapkan dataset agar siap digunakan dalam proses pemodelan melalui beberapa langkah seperti pembacaan data, pemeriksaan kualitas data, pengecekan duplikat, dan analisis distribusi label.

5.3.1.1 Pembacaan Dataset

Implementasi awal implementasi dimulai dengan membaca dataset agar dapat dikenali sehingga memudahkan proses eksplorasi, pemeriksaan kualitas data, dan persiapan sebelum memasuki tahap pemodelan yang lebih kompleks.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
from urllib.parse import urlparse
from collections import Counter

file_dataset = r"D:\TUGAS AKHIR\phishing
detector\dataset\dataset_phishing.csv"

df = pd.read_csv(file_dataset)
```

Kode 5. 1 Membaca dataset

Pada Kode 5. 1 dilakukan *import library*. Selanjutnya, *path* file dataset didefinisikan dan fungsi *pd.read_csv()* digunakan untuk membaca file CSV.

5.3.1.2 Jumlah baris dan kolom

Implementasi analisis dataset bertujuan untuk memahami struktur data serta mengidentifikasi jumlah baris dan kolom yang terdapat dalam dataset.

```
print(f"Dataset memiliki {df.shape[0]} baris dan {df.shape[1]}  
kolom.\n")
```

Kode 5. 2 Menampilkan jumlah baris dan kolom

Pada Kode 5. 2 menganalisis tipe data pada setiap kolom dalam dataset untuk mengetahui struktur data yang digunakan. Fungsi *df.shape* digunakan untuk menampilkan dimensi dataset yaitu jumlah baris dan kolom.

5.3.1.3 Missing Values

Implementasi *missing value* digunakan untuk menangani data yang kosong atau tidak terisi pada dataset agar data dapat digunakan dengan baik dalam proses analisis atau pemodelan.

```
missing = df.isnull().sum()  
if missing.sum() == 0:  
    print("Tidak ada data yang kosong di seluruh 89 kolom.")  
else:  
    print("Kolom dengan data kosong:")  
    print(missing[missing > 0])
```

Kode 5. 3 Implementasi Missing Values

Pada Kode 5. 3 fungsi *df.isnull().sum()* digunakan untuk menghitung jumlah nilai yang hilang pada setiap kolom. Jika tidak ditemukan *missing values*, maka akan menampilkan pesan konfirmasi. Sebaliknya, jika terdapat *missing values*, maka kolom-kolom yang mengandung nilai kosong akan ditampilkan beserta jumlahnya.

5.3.1.4 URL Duplikat

Pengecekan data duplikat dilakukan untuk memastikan tidak terdapat URL yang muncul lebih dari satu kali dalam dataset karena dapat mempengaruhi proses *training* model.

```
nama_kolom = 'url'
df_duplikat_url = df[df[nama_kolom].duplicated(keep=False)]

if len(df_duplikat_url) > 0:
    print(df_duplikat_url[nama_kolom].value_counts())

    display(df_duplikat_url.head(5))
else:
    print("\nSelamat! Tidak ada baris duplikat pada kolom URL.")
```

Kode 5. 4 Implementasi URL Data Duplikat

Pada Kode 5. 4 dilakukan pengecekan data duplikat pada kolom URL menggunakan *fungsi duplicated()* dengan parameter *keep=False* yang berarti semua baris duplikat akan ditandai. Jika ditemukan data duplikat, maka akan menampilkan daftar URL yang duplikat beserta jumlah kemunculannya dan menampilkan pratinjau data tersebut.

5.3.1.5 Distribusi Label Status

Distribusi label status dianalisis untuk melihat keseimbangan data antara kelas *phishing* dan *legitimate* dalam dataset sebelum proses pemodelan.

```
target_col = 'status'
if target_col in df.columns:
    print(df[target_col].value_counts())
    plt.show()
else:
    print(f"\nKolom '{target_col}' tidak ditemukan. Cek kembali nama kolom target.")
```

Kode 5. 5 Implementasi Analisis Distribusi Kelas Target

Pada Kode 5. 5 dilakukan analisis terhadap kolom target yang bernama status. Fungsi `value_counts()` digunakan untuk menghitung jumlah sampel pada setiap kelas. Hasil analisis divisualisasikan menggunakan `countplot` dari untuk memberikan gambaran visual yang lebih jelas. Visualisasi ini membantu dalam mengidentifikasi apakah terdapat *class imbalance* yang perlu ditangani.

5.3.1.6 Korelasi Antar Fitur

Implementasi korelasi antar fitur dilakukan untuk mengetahui hubungan antar variabel numerik dalam dataset. Korelasi ini membantu dalam memahami keterkaitan antar fitur serta mengidentifikasi fitur yang memiliki hubungan kuat dengan fitur lainnya.

```
numeric_df = df.select_dtypes(include=['number'])  
plt.title('Korelasi Antar Fitur')
```

Kode 5. 6 Implementasi korelasi antar fitur

Pada Kode 5. 6 dataset difilter untuk mengambil kolom yang memiliki tipe data numerik menggunakan fungsi `select_dtypes(include=['number'])`, karena perhitungan korelasi hanya dapat dilakukan pada data numerik.

5.3.2 Implementasi Data Preprocessing

Implementasi *preprocessing* merupakan proses transformasi data mentah menjadi data yang siap digunakan untuk pelatihan model *machine learning*. Tahap ini mencakup beberapa sub-tahapan yaitu *data selection*, *feature engineering*, *data transformation*, dan *data cleaning*.

5.3.2.1 Data Selection

Implementasi *data selection* bertujuan untuk memilih fitur yang relevan, sehingga beberapa kolom dihapus berdasarkan analisis korelasi dan relevansi fitur karena tidak memberikan kontribusi signifikan. Berikut implementasi kode pada tahap *data selection*.

```
kolom_yang_dihapus = [
```

```

    'nb_or',
    'ratio_nullHyperlinks',
    'ratio_intRedirection',
    'ratio_intErrors',
    'submit_email',
    'sfh'
]

df_selected = df.drop(columns=kolom_yang_dihapus,
errors='ignore')

output_path = r'D:\TUGAS AKHIR\phishing
detector\processed\data_selection.csv'
df_selected.to_csv(output_path, index=False)

```

Kode 5. 7 Implementasi data selection

Pada Kode 5. 7 dilakukan penghapusan 6 kolom yang dianggap tidak relevan atau memiliki korelasi rendah dengan target klasifikasi, yaitu *nb_or*, *ratio_nullHyperlinks*, *ratio_intRedirection*, *ratio_intErrors*, *submit_email*, dan *sfh*, karena tidak memiliki nilai pembeda yang berguna bagi model dalam mengenali pola antara kelas *phishing* dan *non-phishing*. Fungsi *drop()* dengan parameter *errors='ignore'* digunakan untuk menghapus kolom tersebut, dan hasil seleksi kemudian disimpan dalam file CSV baru untuk tahap selanjutnya.

5.3.2.2 Data Transformation

Tahap *data transformation* bertujuan untuk mengubah format data agar sesuai dengan kebutuhan proses pemodelan *machine learning*. Pada tahap ini dilakukan transformasi terhadap label kelas yang sebelumnya masih dalam bentuk kategorikal menjadi bentuk numerik.

```

if 'status' in df_final.columns:
    df_final['label'] = df_final['status'].map({'phishing': 1,
'legitimate': 0})
    df_final = df_final.drop(columns=['status'])

```

Kode 5. 8 Implementasi data transformation

Pada Kode 5. 8, dilakukan transformasi label target menggunakan fungsi *map()* yang memetakan nilai *phishing* menjadi 1 dan *legitimate* menjadi 0. Setelah label baru dibuat, kolom 'status' yang lama dihapus menggunakan *.drop(columns=['status'])* untuk menghindari *redundansi* dan mengubah menjadi kolom 'label'. Setelah proses *transformation* selesai, dataset kemudian disimpan dalam file CSV untuk tahap *cleaning*.

5.3.2.3 Data Cleaning

Implementasi *data cleaning* merupakan tahap akhir dari *preprocessing* ini yang bertujuan untuk membersihkan data dari duplikasi. Dalam penelitian ini, pembersihan data difokuskan pada penghapusan URL duplikat. Berikut adalah implementasi kode untuk *data cleaning*.

```
if jumlah_duplikat > 0:
    df_final = df_final.drop_duplicates(subset=['url'],
    keep='first').reset_index(drop=True)
    print("--- Duplikat berhasil dihapus ---")
else:
    print("--- Tidak ada duplikat yang perlu dihapus ---")
```

Kode 5. 9 Implementasi korelasi antar fitur

Pada Kode 5. 9, dilakukan penghapusan data duplikat berdasarkan kolom URL menggunakan fungsi *drop_duplicates()* dengan parameter *subset=['url']* untuk menentukan kolom yang dijadikan acuan pengecekan duplikasi. Parameter *keep='first'* digunakan untuk mempertahankan kemunculan pertama dan menghapus duplikasi selanjutnya. Fungsi *reset_index(drop=True)* digunakan untuk mengatur ulang indeks setelah penghapusan. Dataset yang telah dibersihkan kemudian disimpan dalam file CSV baru untuk tahap pemodelan.

5.4 Implementasi Data Split

Implementasi data split dilakukan setelah seluruh proses *preprocessing* selesai. Pada tahap ini, dataset dibagi menjadi *data training* dan *data test* sebelum model *machine learning* dilatih.

```

X_train, X_test, y_train, y_test = train_test_split(
    X_all,
    y_all,
    test_size=0.2,
    stratify=y_all,
    random_state=12
)

```

Kode 5. 10 Implementasi data split

Pada Kode 5. 10, *train_test_split()* digunakan untuk membagi dataset menjadi dua bagian yang terpisah dengan tujuan melatih dan menguji model. *test_size=0.2* diambil 20% untuk uji, sisanya 80% untuk latih. *stratify=y_all* untuk menjaga keseimbangan jenis URL di kedua bagian. *random_state=12* agar pembagian selalu sama setiap kali dijalankan.

5.5 Implementasi Model

Pada implementasi ini menggunakan tiga algoritma yaitu Random Forest, XGBoost, dan Stacking Ensemble (Logistic Regression sebagai *meta-learner*). Sebelum model mulai dilatih, *Genetic Algorithm* digunakan untuk mencari *hyperparameter* terbaik bagi Random Forest dan XGBoost, agar pengaturan yang digunakan benar-benar optimal dan sesuai.

5.5.1 Implementasi Persiapan Data dan Fitur

Pada tahap ini dilakukan pembacaan dataset dari *data_cleaning.csv*, serta implementasi proses persiapan data dan fitur sebelum digunakan dalam *training* model.

```

def load_and_prepare_data():
    data = pd.read_csv(data_file_path)
    data.columns = data.columns.str.strip()

    y_all = pd.to_numeric(data[TARGET_COL], errors="coerce")
    valid_mask = y_all.notna()
    data = data.loc[valid_mask].copy()
    y_all = y_all.loc[valid_mask].astype("int64")

```

```

    id_cols = [c for c in ["url"] if c in data.columns]
    all_features = [c for c in data.columns if c not in
[TARGET_COL] + id_cols]

    webcontent_features_44 = [c for c in all_features if c not
in url_features_37]
    hybrid_features_81 = url_features_37 +
webcontent_features_44
    feature_candidates = hybrid_features_81 if
ACTIVE_TRAIN_FEATURES == "hybrid81" else url_features_37

    train_idx, test_idx = train_test_split(
        data.index, test_size=TEST_SIZE, stratify=y_all,
random_state=RANDOM_STATE
    )

    X_train = prep_X(data, feature_candidates, train_idx)
    X_test = prep_X(data, feature_candidates, test_idx)

    selected_features =
X_train.columns[X_train.notna().any()].tolist()
    X_train = X_train[selected_features]
    X_test = X_test[selected_features]

    train_medians = X_train.median(numeric_only=True)
    X_train = X_train.fillna(train_medians)
    X_test = X_test.fillna(train_medians)

    y_train = y_all.loc[train_idx]
    y_test = y_all.loc[test_idx]
    return X_train, X_test, y_train, y_test, selected_features,
train_medians

```

Kode 5. 11 Implementasi load dan prepare data

Pada Kode 5. 11, fungsi *load_and_prepare_data()* digunakan untuk menyiapkan data dengan membaca dataset, membersihkan kolom, memisahkan fitur dan target, serta memilih fitur yang digunakan. Data kemudian dibagi menjadi data latih dan

data uji menggunakan *stratified split*. Hasil akhirnya berupa data yang siap digunakan untuk.

5.5.2 Implementasi Training Model Random Forest

Random Forest dilakukan menggunakan *Genetic Algorithm* (GA) adalah cara otomatis menemukan *setting* model terbaik. Dengan membuat banyak kombinasi parameter secara acak, kemudian memilih yang paling akurat untuk dikombinasi agar menghasilkan kombinasi baru. Proses ini diulang berkali-kali hingga menemukan parameter terbaik.

```
def tune_rf_ga(X_train, y_train):
    cv = StratifiedKFold(n_splits=GA_CV_SPLITS, shuffle=True,
        random_state=RANDOM_STATE)

    def decode(ind):
        max_feat = {0: "sqrt", 1: "log2", 2:
            None}[int(clamp(round(ind[4]), 0, 2))]
        return {
            "n_estimators": int(clamp(round(ind[0]), 100,
                500)),
            "max_depth": int(clamp(round(ind[1]), 3, 16)),
            "min_samples_split": int(clamp(round(ind[2]), 2,
                20)),
            "min_samples_leaf": int(clamp(round(ind[3]), 1,
                10)),
            "max_features": max_feat,
            "random_state": RANDOM_STATE,
            "n_jobs": -1,
        }

    def evaluate(ind):
        params = decode(ind)
        model = RandomForestClassifier(**params)
        score = cross_val_score(model, X_train, y_train, cv=cv,
            scoring="f1", n_jobs=-1).mean()
        return (float(score),)
```

Kode 5. 12 Tuning Random Forest dengan Genetic Algorithm

Pada Kode 5. 12, fungsi *decode()* ini mengubah angka-angka dari GA menjadi *setting* parameter model. Parameter Random Forest terdiri dari *n_estimators* yang menentukan jumlah *tree*, *max_depth* yang mengontrol kedalaman maksimal setiap *tree*, *min_samples_split* yang menetapkan jumlah sampel minimum di node, *min_samples_leaf* yang menentukan sampel minimum di *leaf node*, dan *max_features* yang mengontrol strategi pemilihan fitur saat membangun *tree*. Fungsi *evaluate()* untuk menghitung *fitness* (nilai kualitas) setiap kombinasi parameter.

5.5.3 Implementasi Training Model XGBoost

Pada model XGBoost dilakukan menggunakan *Genetic Algorithm* (GA) adalah cara otomatis menemukan *setting* model terbaik. Dengan membangun *trees* secara bertahap, *tree* pertama membuat prediksi pertama, *tree* kedua memperbaiki kesalahan *tree* pertama, dan seterusnya.

```
def tune_xgb_ga(X_train, y_train):
    cv = StratifiedKFold(n_splits=GA_CV_SPLITS, shuffle=True,
        random_state=RANDOM_STATE)

    def decode(ind):
        return {
            "n_estimators": int(clamp(round(ind[0]), 150, 500)),
            "learning_rate": float(clamp(ind[1], 0.01, 0.30)),
            "max_depth": int(clamp(round(ind[2]), 3, 10)),
            "subsample": float(clamp(ind[3], 0.60, 1.00)),
            "colsample_bytree": float(clamp(ind[4], 0.60,
                1.00)),
            "min_child_weight": float(clamp(ind[5], 1.0, 10.0)),
            "gamma": float(clamp(ind[6], 0.0, 5.0)),
            "reg_alpha": float(clamp(ind[7], 0.0, 5.0)),
            "reg_lambda": float(clamp(ind[8], 0.1, 10.0)),
            "eval_metric": "logloss",
            "random_state": RANDOM_STATE,
            "n_jobs": -1,
        }
```

```
def evaluate(ind):
    params = decode(ind)
    model = XGBClassifier(**params)
    score = cross_val_score(model, X_train, y_train, cv=cv,
                             scoring="f1", n_jobs=-1).mean()
    return (float(score),)
```

Kode 5. 13 Tuning XGBoost dengan Genetic Algorithm

Pada Kode 5. 13, fungsi *decode()* ini mengubah angka-angka dari GA menjadi *setting* parameter model. Parameter XGBoost terdiri dari *n_estimators* yang menentukan jumlah *boosting rounds*, *learning_rate* yang mengontrol kecepatan pembelajaran, *max_depth* yang mengontrol kedalaman *tree*, *subsample* yang menentukan fraksi data per iterasi, *colsample_bytree* yang menentukan fraksi fitur per *tree*, *min_child_weight* yang menetapkan bobot minimum *leaf*, *gamma* yang menentukan minimum *loss reduction*, *reg_alpha* untuk regularisasi L1, dan *reg_lambda* untuk regularisasi L2.

5.5.4 Implementasi Training Stacking Ensemble menggunakan Logistic Regression

Implementasi metode Stacking Ensemble untuk menghasilkan prediksi yang lebih optimal dengan cara menggabungkan Random Forest dan XGBoost sebagai *base learner* melalui Logistic Regression yang bertindak sebagai *meta-learner*.

```
stack_model = StackingClassifier(
    estimators=[("rf", rf_model), ("xgb", xgb_model)],
    final_estimator=LogisticRegression(max_iter=1000,
    class_weight="balanced", random_state=RANDOM_STATE),
    stack_method="predict_proba",
    n_jobs=-1
)
stack_model.fit(X_train, y_train)
stack_pred = stack_model.predict(X_test)
stack_prob = stack_model.predict_proba(X_test)[: , 1]
print_metrics("Stacking (RF+XGB+LR)", y_test, stack_pred,
stack_prob)
```

Kode 5. 14 Implementasi Stacking dengan Logistic Regression

Pada Kode 5. 14, parameter *estimators* berisi daftar dua model *base learner* yang telah dilatih sebelumnya. Hasil prediksi dari kedua model dasar ini kemudian digabungkan oleh Logistic Regression yang bertindak sebagai *meta-learner* melalui pengaturan *final_estimator*. Untuk cara penggabungannya, digunakan *stack_method="predict_proba"* agar model akhir belajar dari nilai probabilitas yang dihasilkan oleh model-model dasar tersebut. Terakhir, parameter *class_weight="balanced"* diterapkan untuk menangani masalah ketidakseimbangan jumlah *class imbalance*.

5.5.5 Implementasi Evaluasi Model Final

Pada tahap ini dilakukan evaluasi terhadap model final yang telah dioptimasi, lalu ditunjukkan proses evaluasi model setelah diperoleh parameter terbaik. Model Random Forest dan XGBoost terlebih dahulu dilatih ulang menggunakan data latih, kemudian digunakan untuk melakukan prediksi pada data uji serta menghitung probabilitas kelas.

```
rf_model = RandomForestClassifier(**best_rf_params)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
rf_prob = rf_model.predict_proba(X_test)[:, 1]
print_metrics("Random Forest (GA Tuned)", y_test, rf_pred,
rf_prob)

xgb_model = XGBClassifier(**best_xgb_params)
xgb_model.fit(X_train, y_train)
xgb_pred = xgb_model.predict(X_test)
xgb_prob = xgb_model.predict_proba(X_test)[:, 1]
print_metrics("XGBoost (GA Tuned)", y_test, xgb_pred, xgb_prob)

stack_model = StackingClassifier(
    estimators=[("rf", rf_model), ("xgb", xgb_model)],
    final_estimator=LogisticRegression(max_iter=1000,
    class_weight="balanced", random_state=RANDOM_STATE),
    stack_method="predict_proba",
    n_jobs=-1
```

```

)
stack_model.fit(X_train, y_train)
stack_pred = stack_model.predict(X_test)
stack_prob = stack_model.predict_proba(X_test)[:, 1]
print_metrics("Stacking (RF+XGB+LR)", y_test, stack_pred,
stack_prob)

```

Kode 5. 15 Implementasi Evaluasi Model Final

Pada Kode 5. 15, hasil prediksi tersebut selanjutnya dievaluasi menggunakan fungsi *print_metrics()* untuk memperoleh nilai *Accuracy*, *Precision*, *Recall*, F1-score, dan AUC. Selain itu, dibangun juga model stacking ensemble yang menggabungkan Random Forest dan XGBoost sebagai *base learners* dengan Logistic Regression sebagai *meta-learner*.

5.5.6 Penyimpanan Model dan Metadata

Pada tahap ini dilakukan penyimpanan model dan *metadata* sebagai bagian dari proses *deployment*. Model yang telah dilatih disimpan bersama informasi pendukung untuk memastikan konsistensi antara proses pelatihan dan penggunaan model di tahap selanjutnya.

```

with open(os.path.join(current_dir, "random_forest_model.pkl"),
"wb") as f:
    pickle.dump(rf_model, f)

with open(os.path.join(current_dir, "xgboost_model.pkl"), "wb")
as f:
    pickle.dump(xgb_model, f)

with open(os.path.join(current_dir, "rule_lr.pkl"), "wb") as f:
    pickle.dump(stack_model.final_estimator_, f)

with open(os.path.join(current_dir, "feature_columns.txt"), "w",
encoding="utf-8") as f:
    f.write("\n".join(selected_features))

with open(os.path.join(current_dir, "feature_medians.pkl"),
"wb") as f:

```



```

pickle.dump(train_medians.to_dict(), f)

with open(os.path.join(current_dir, "ga_tuning_report.json"),
"w", encoding="utf-8") as f:
    json.dump(tuning_report, f, indent=2)

```

Kode 5. 16 Penyimpanan Model dan Metadata

Pada Kode 5. 16, ditunjukkan proses penyimpanan model dan *metadata* ke dalam beberapa berkas. Model Random Forest, XGBoost, dan komponen *meta-learner* dari *stacking* disimpan dalam format *.pkl* menggunakan *pickle*. Selain itu, disimpan juga daftar fitur yang digunakan, nilai median untuk *imputasi*, serta laporan hasil *tuning Genetic Algorithm* dalam format JSON. Penyimpanan ini bertujuan untuk menjaga konsistensi *pipeline* serta memudahkan penggunaan kembali model tanpa perlu melakukan pelatihan ulang.

5.6 Implementasi Rule based Filtering

Rule-Based Filtering adalah lapisan *pre-filter* untuk deteksi *phishing* cepat berdasarkan pola URL yang mencurigakan.

```

very_important = {
    "suspicious_tld": (feats.get("suspicious_tld", 0) == 1),
    "nb_at": (feats.get("nb_at", 0) >= 1),
    "ip": (feats.get("ip", 0) == 1),
    "nb_underscore": (feats.get("nb_underscore", 0) > 3),
}

important = {
    "ratio_digits_url": (feats.get("ratio_digits_url", 0) >
0.3),
    "nb_subdomains": (feats.get("nb_subdomains", 0) > 3),
    "nb_percent": (feats.get("nb_percent", 0) > 5),
    "nb_tilde": (feats.get("nb_tilde", 0) >= 1),
    "nb_semicolumn": (feats.get("nb_semicolumn", 0) >= 1),
    "nb_star": (feats.get("nb_star", 0) >= 1),
    "nb_comma": (feats.get("nb_comma", 0) >= 1),
    "random_domain": (feats.get("random_domain", 0) == 1),
}

```

```

less_important = {
    "length_hostname": (feats.get("length_hostname", 0) > 30),
    "nb_dollar": (feats.get("nb_dollar", 0) >= 1),
    "nb_qm": (feats.get("nb_qm", 0) > 2),
    "nb_colon": (feats.get("nb_colon", 0) > 1),
    "nb_eq": (feats.get("nb_eq", 0) > 8),
    "nb_dots": (feats.get("nb_dots", 0) > 4),
    "nb_slash": (feats.get("nb_slash", 0) > 7),
    "nb_and": (feats.get("nb_and", 0) > 3),
    "nb_hyphens": (feats.get("nb_hyphens", 0) > 3),
    "http_in_path": (feats.get("http_in_path", 0) == 1),
    "https_token": (feats.get("https_token", 0) == 1),
    "port": (feats.get("port", 0) == 1),
    "shortening_service": (feats.get("shortening_service", 0) ==
1),
}

vi_count = sum(very_important.values())
imp_count = sum(important.values())
less_count = sum(less_important.values())

risk_score = (3 * vi_count) + (2 * imp_count) + less_count
rule_flag = int((vi_count >= 1) or (risk_score >=
PREFILTER_HARD_PHISHING_SCORE))

```

Kode 5. 17 Implementasi Rule based Filtering

Pada Kode 5. 17, target *variable* dipisahkan dari dataset untuk *rule-based evaluation*. Fungsi `_col()` dibuat untuk *safely* mengakses kolom dengan penanganan untuk kolom yang tidak ada. Fitur-fitur dikategorikan menjadi tiga *dictionary*: *very_important*, *important*, dan *less_important* masing-masing berisi kondisi yang menentukan apakah fitur tersebut menunjukkan indikasi *phishing*. Setiap kategori dikonversi menjadi *DataFrame* dan dijumlahkan untuk mendapatkan jumlah indikator pada setiap kategori (*vi_count*, *imp_count*, *less_count*). *Risk score* dihitung dengan formula *weighted sum*: 3 kali *very important count* ditambah 2 kali *important count* ditambah 1 kali *less important count*. *Flag phishing* ditentukan jika

very_important count ≥ 1 ATAU *risk_score* ≥ 7 , dengan hasil dikonversi ke integer (0 atau 1).

5.7 Implementasi Evaluasi

Confusion Matrix adalah yang menunjukkan performa model klasifikasi dengan membandingkan prediksi model terhadap label sebenarnya, yang memungkinkan perhitungan berbagai metrik evaluasi seperti *accuracy*, *precision*, *recall*, F1-score, dan AUC untuk memberikan penilaian menyeluruh terhadap kinerja model.

```
def evaluate_model(name, model, X_eval, y_eval):
    y_pred = model.predict(X_eval)
    y_prob = model.predict_proba(X_eval)[:, 1] if hasattr(model,
"predict_proba") else y_pred

    tn, fp, fn, tp = confusion_matrix(y_eval, y_pred, labels=[0,
1]).ravel()
    try:
        auc = roc_auc_score(y_eval, y_prob)
    except Exception:
        auc = np.nan

    return {
        "ML Model": name,
        "Accuracy": round(accuracy_score(y_eval, y_pred), 3),
        "Precision": round(precision_score(y_eval, y_pred,
zero_division=0), 3),
        "Recall": round(recall_score(y_eval, y_pred,
zero_division=0), 3),
        "F1 Score": round(f1_score(y_eval, y_pred,
zero_division=0), 3),
        "AUC": round(auc, 3) if not np.isnan(auc) else np.nan,
        "TN": int(tn), "FP": int(fp), "FN": int(fn), "TP":
int(tp),
    }

detail_rows = []
if "forest" in globals():
```

```

        detail_rows.append(evaluate_model("Random Forest", forest,
X_test, y_test))
    if "xgb" in globals():
        detail_rows.append(evaluate_model("XGBoost", xgb, X_test,
y_test))
    if "stack_model" in globals():
        detail_rows.append(evaluate_model("Stacking (RF+XGB+LR)",
stack_model, X_test, y_test))

detail_df = pd.DataFrame(detail_rows).sort_values(
    by=["Accuracy", "F1 Score"], ascending=False
).reset_index(drop=True)

detail_df

```

Kode 5. 18 Implementasi Evaluasi

Pada Kode 5. 18, Fungsi *evaluate_model()* berfungsi mengevaluasi performa model dengan menghasilkan prediksi kelas melalui *model.predict()* dan probabilitas prediksi melalui *model.predict_proba()*. *Confusion matrix* dibuat dengan *confusion_matrix()* untuk menghasilkan 4 komponen (TN, FP, FN, TP), kemudian 6 metrik evaluasi dihitung: *accuracy_score()* untuk akurasi, *precision_score()* untuk presisi, *recall_score()* untuk *recall*, *f1_score()* untuk F1-score, dan *roc_auc_score()* untuk AUC. Semua metrik dibulatkan 3 desimal dan disimpan dalam *dictionary*, lalu dikembalikan sebagai hasil. Setelah itu, fungsi *evaluate_model()* dipanggil 3 kali untuk masing-masing model Random Forest, XGBoost, Stacking dan hasil disimpan di *list detail_rows*, kemudian dikonversi menjadi *DataFrame* dan diurutkan berdasarkan *Accuracy* dan F1-Score tertinggi sehingga model terbaik muncul di baris pertama untuk perbandingan mudah.

5.8 Implementasi Feature Extraction

Implementasi ekstraksi fitur pada sistem ini dilakukan melalui dua jenis pendekatan.

1. Fitur berbasis URL (statis), yaitu fitur yang dihitung langsung dari teks atau struktur URL tanpa perlu membuka atau mengakses halaman web tersebut.

2. Fitur berbasis konten web (dinamis), yaitu fitur yang diperoleh dengan melakukan permintaan HTTP ke halaman web, kemudian memproses atau membaca struktur HTML dari halaman tersebut.

Dengan menggunakan dua pendekatan ini, sistem dapat bekerja secara fleksibel. Jika hanya menggunakan fitur berbasis URL, sistem dapat berjalan lebih cepat dan efisien. Namun, ketika fitur berbasis konten web juga digunakan, tingkat akurasi deteksi dapat meningkat karena sistem memiliki informasi yang lebih lengkap dari halaman web yang dianalisis.

5.8.1 Normalisasi URL dan utilitas awal

```
def parse_url(url: str):
    u = (url or "").strip()
    if not u.startswith(("http://", "https://")):
        u = "http://" + u
    parsed = urlparse(u)
    hostname = (parsed.hostname or "").lower()
    path = parsed.path or ""
    return u, parsed, hostname, path

def is_ip(hostname: str) -> int:
    try:
        ipaddress.ip_address(hostname)
        return 1
    except Exception:
        return 0

def entropy(s: str) -> float:
    if not s:
        return 0.0
    probs = [s.count(c) / len(s) for c in set(s)]
    return -sum(p * math.log2(p) for p in probs)
```

Kode 5. 19 Normalisasi URL dan utilitas awal

Pada Kode 5. 19, *fungsi parse_url()* memastikan bahwa URL yang diberikan memiliki skema yang valid (http:// atau https://), kemudian memecah URL tersebut menjadi komponen-komponen terpisah. Fungsi *is_ip()* digunakan untuk mendeteksi

apakah *host* dalam URL tersebut berupa alamat IP. Sedangkan, fungsi *entropy()* mengukur tingkat keacakan dari domain, dimana semakin tinggi nilai entropinya, semakin mencurigakan domain tersebut. Ketiga fungsi ini bekerja secara bersama-sama untuk menganalisis dan menilai validitas serta potensi risiko pada URL yang dianalisis.

5.8.2 Ekstraksi feature URL

```
def extract_url_features(url: str) -> dict:
    full, parsed, hostname, path = parse_url(url)

    digits_url = sum(c.isdigit() for c in full)
    digits_host = sum(c.isdigit() for c in hostname)

    random_domain = 1 if entropy(domain) > 3.5 else 0
    shortening_service = 1 if any(s in hostname for s in
    SHORTENERS) else 0

    feats = {
        "length_url": len(full),
        "length_hostname": len(hostname),
        "nb_dots": full.count("."),
        "nb_hyphens": full.count("-"),
        "ratio_digits_url": digits_url / max(len(full), 1),
        "ratio_digits_host": digits_host / max(len(hostname),
        1),
        "random_domain": random_domain,
        "shortening_service": shortening_service,
    }

    return feats
```

Kode 5. 20 Ekstraksi feature URL

Pada Kode 5. 20, fitur-fitur ini dihitung berdasarkan berbagai karakteristik URL, seperti panjang URL, jumlah simbol khusus (seperti @, -, /, dan .), rasio digit, jumlah *subdomain*, serta indikasi domain yang acak. Beberapa indikator *phishing* yang kuat meliputi keberadaan *host* yang berupa alamat IP, banyaknya simbol khusus yang digunakan, penggunaan *subdomain* yang berlebihan, domain yang

terkesan acak, atau penggunaan layanan pemendek URL/*URL shortener*. Karakteristik-karakteristik ini dapat memberikan petunjuk tentang potensi risiko dan validitas sebuah URL.

5.8.3 Ekstraksi fitur konten web

```
def extract_web_content_features(url: str) -> dict:
    out = {}
    try:
        resp = requests.get(
            url,
            headers={"User-Agent": USER_AGENT},
            timeout=WEB_FETCH_TIMEOUT,
            allow_redirects=True
        )
        html = resp.text or ""
        final_url = resp.url or url
    except Exception:
        return out

    base_host = (urlparse(final_url).hostname or "").lower()
    out["nb_external_redirection"] = sum(
        1 for r in (resp.history or [])
        if not _same_or_subdomain((urlparse(r.url).hostname or
        "").lower(), base_host)
    )

    if BeautifulSoup is None:
        return out

    soup = BeautifulSoup(html, "html.parser")
    anchors = [a.get("href", "") for a in soup.find_all("a")]
    total_a = len(anchors)
    ext_a = sum(1 for h in anchors if _is_external_href(h,
    final_url, base_host))
    int_a = max(total_a - ext_a, 0)

    out["nb_hyperlinks"] = total_a
```

```

    out["ratio_intHyperlinks"] = (int_a / total_a) if total_a
else 0.0
    out["ratio_extHyperlinks"] = (ext_a / total_a) if total_a
else 0.0
    out["iframe"] = 1 if soup.find("iframe") else 0
    out["popup_window"] = 1 if "window.open(" in html.lower()
else 0

    return out

```

Kode 5. 21 Ekstraksi fitur konten web

Pada Kode 5. 21, melakukan pengambilan *fetch* halaman untuk memperoleh struktur HTML-nya. Fitur konten yang diekstraksi mencakup jumlah *hyperlink*, rasio antara link internal dan eksternal, adanya redirect eksternal, serta elemen berisiko seperti *iframe* dan *popup*. Jika halaman tidak dapat diakses, fungsi ini akan mengembalikan fitur kosong sebagai langkah *fallback* yang aman.

5.8.4 Penggabungan fitur dan pembentukan vektor model

```

def extract_features(url: str, include_web_content: bool) ->
dict:
    full_url, _, _, _ = parse_url(url)
    feats = extract_url_features(full_url)
    if include_web_content:
        feats.update(extract_web_content_features(full_url))
    return feats

def build_ml_vector(url: str, cols: list, medians: dict,
include_web_content: bool):
    feats = extract_features(url,
include_web_content=include_web_content)
    vector = []
    for c in cols:
        default_v = medians.get(c, 0.0)
        v = feats.get(c, default_v)
        vector.append(to_float(v, default_v))
    return np.array([vector], dtype=float)

```

Kode 5. 22 Penggabungan fitur dan pembentukan vektor model

Pada Kode 5. 22, fungsi *extract_features()* menggabungkan fitur URL dan fitur konten sesuai dengan mode yang dipilih. Fungsi *build_ml_vector()* menyusun urutan fitur berdasarkan *feature_columns.txt* untuk memastikan konsistensi dengan data saat pelatihan. Apabila ada fitur yang tidak tersedia, sistem akan menggantinya dengan nilai median yang disimpan dalam file *feature_medians.pkl* untuk menjaga stabilitas *input* yang diberikan ke model.

5.9 Implementasi Eksperimen

Implementasi eksperimen dilakukan untuk menguji model yang telah dirancang. Pada tahap ini, dilakukan beberapa skenario eksperimen menggunakan algoritma Random Forest, XGBoost, metode Stacking, serta hybrid Rule-Based dan Stacking untuk melihat perbedaan hasil yang diperoleh.

5.9.1 Implementasi Eksperimen Random Forest

Eksperimen pertama menggunakan model Random Forest. Pada tahap ini dilakukan pengujian kinerja model Random Forest sebagai *base learner* yang dilatih untuk mengklasifikasi URL ke dalam dua kategori, yaitu *phishing* dan *safe* berdasarkan fitur-fitur yang telah diekstraksi.

```
if decision_mode == "rf_only":
    preds = predict_models(
        url=url,
        cols=cols,
        medians=medians,
        include_web_content=include_web_content,
        precomputed_rule_score=risk_score,
        precomputed_rule_flag=rule_flag
    )
    p = clamp01(float(preds.get("rf_prob") or 0.0))
    return build_response(
        final_phishing_prob=p,
        decision_source="rf_only",
        model_name="Random Forest Only",
        rf_prob=preds.get("rf_prob"),
        xgb_prob=None,
```

```

        stack_prob=None
    )

```

Kode 5. 23 Implementasi Random Forest

Pada Kode 5. 23, decision yang dipilih adalah *rf_only*, sistem memanggil fungsi *predict_models()* yang mengolah URL dan mengembalikan prediksi dari semua model yang tersedia. Dari hasil prediksi tersebut, hanya probabilitas dari Random Forest (*rf_prob*) yang diambil dan digunakan sebagai probabilitas akhir phishing. Nilai ini di-clamp menggunakan fungsi *clamp01()* untuk memastikan nilai probabilitas tetap dalam rentang 0 hingga 1. Selanjutnya, fungsi *build_response()* dipanggil untuk membuat response JSON yang mengirimkan hasil prediksi beserta metadata, di mana *decision_source* ditandai sebagai *rf_only* dan *model_name* ditampilkan sebagai *Random Forest Only*. Nilai probabilitas dari Random Forest diset sebagai *None* karena mode ini fokus hanya pada hasil Random Forest.

5.9.2 Implementasi Eksperimen XGBoost

Eksperimen kedua menggunakan model XGBoost. Pada tahap ini dilakukan pengujian kinerja model XGBoost sebagai *base learner* yang dilatih untuk mengklasifikasi URL ke dalam dua kategori, yaitu *phishing* dan *safe* berdasarkan fitur-fitur yang telah diekstraksi.

```

if decision_mode == "xgb_only":
    preds = predict_models(
        url=url,
        cols=cols,
        medians=medians,
        include_web_content=include_web_content,
        precomputed_rule_score=risk_score,
        precomputed_rule_flag=rule_flag
    )
    p = clamp01(float(preds.get("xgb_prob") or 0.0))
    return build_response(
        final_phishing_prob=p,
        decision_source="xgb_only",
        model_name="XGBoost Only",
    )

```

```

        rf_prob=None,
        xgb_prob=preds.get("xgb_prob"),
        stack_prob=None
    )

```

Kode 5. 24 Implementasi XGBoost

Pada Kode 5. 24, implementasi ini sangat mirip dengan mode Random Forest, namun fokus diberikan pada probabilitas dari model XGBoost. Fungsi *predict_models()* dipanggil dengan parameter yang sama untuk mendapatkan prediksi dari semua model. Namun, dari hasil tersebut, probabilitas phishing diambil dari XGBoost (*xgb_prob*) dan di-clamp ke dalam rentang 0-1 untuk memastikan validitas nilai probabilitas. Response dibangun melalui *build_response()* dengan *decision_source* yang ditandai sebagai *xgb_only* dan *model_name* sebagai *XGBoost Only*. Dalam mode ini, *rf_prob* dan *stack_prob* diset sebagai *None* karena hanya hasil XGBoost yang menjadi dasar pengambilan keputusan.

5.9.3 Implementasi Eksperimen Stacking

Eksperimen ketiga mengimplementasikan pendekatan *stacking* dengan menggabungkan model Random Forest dan XGBoost sebagai *base learner*, serta menggunakan Logistic Regression sebagai *meta-learner*.

```

if decision_mode == "ml_stacking_only":
    preds = predict_models(
        url=url,
        cols=cols,
        medians=medians,
        include_web_content=include_web_content,
        precomputed_rule_score=risk_score,
        precomputed_rule_flag=rule_flag
    )

    if preds.get("stack_prob") is not None:
        p = clamp01(float(preds["stack_prob"]))
        source = "ml_stacking_only"

```

```

        model_name = "RF + XGB + Logistic Regression
Stacking"
    else:
        rf_p = float(preds.get("rf_prob") or 0.0)
        xgb_p = float(preds.get("xgb_prob") or 0.0)
        p = clamp01(0.5 * rf_p + 0.5 * xgb_p)
        source = "ml_rf_xgb_average_only"
        model_name = "RF + XGB Average (Meta model
unavailable)"

    return build_response(
        final_phishing_prob=p,
        decision_source=source,
        model_name=model_name,
        rf_prob=preds.get("rf_prob"),
        xgb_prob=preds.get("xgb_prob"),
        stack_prob=preds.get("stack_prob")
    )

```

Kode 5. 25 Implementasi Stacking

Pada Kode 5. 25, mengimplementasikan *meta-learner* dengan menggunakan prediksi dari Random Forest dan XGBoost sebagai input untuk *meta-learner* Logistic Regression. Setelah memanggil *predict_models()*, sistem melakukan pengecekan apakah meta-model tersedia dan mampu menghasilkan prediksi *stacking* (*stack_prob*). Jika meta-model tersedia, probabilitas akhir diambil dari prediksi *stacking* meta-model, *decision_source* diset sebagai *ml_stacking_only*, dan *model_name* menampilkan stack architecture sebagai *RF + XGB + Logistic Regression Stacking*. Namun, jika *meta-learner* tidak tersedia atau terjadi *error*, sistem menggunakan *fallback mechanism* dengan menghitung rata-rata tertimbang dari probabilitas Random Forest dan XGBoost (masing-masing dengan bobot 0.5), *decision_source* diubah menjadi *ml_rf_xgb_average_only*, dan *model name* menunjukkan bahwa meta model tidak tersedia.

5.9.4 Implementasi Eksperimen Rule based Filtering dengan Stacking

Eksperimen keempat mengimplementasikan pendekatan *hybrid* yang menggabungkan *rule-based filtering* dengan model *stacking*. Pada tahap ini, aturan

Rule digunakan untuk terlebih dahulu mengidentifikasi URL yang memiliki indikasi kuat sebagai *phishing* atau *safe*, kemudian untuk kasus yang masih ambigu diproses lebih lanjut menggunakan model *stacking* yang terdiri dari Random Forest, XGBoost, dan Logistic Regression.

```
if use_prefilter:
    hard_phishing = (risk_score >=
    PREFILTER_HARD_PHISHING_SCORE) or (
        PREFILTER_BLOCK_ON_VI_HIT and
    rule_detail.get("vi_count", 0) >= 1
    )
    hard_safe = (risk_score <= PREFILTER_HARD_SAFE_SCORE)

    if PREFILTER_FORCE_ML_ON_ZERO_SCORE and risk_score == 0:
        hard_safe = False

    if hard_phishing:
        p = clamp01(max(PREFILTER_PHISHING_MIN_CONF,
    rule_prob))
        return build_response(
            final_phishing_prob=p,
            decision_source="rule_based_prefilter_phishing",
            model_name="Rule-Based Prefilter",
            web_used=False
        )

    if hard_safe:
        final_safe_prob = clamp01(PREFILTER_SAFE_CONF)
        p = clamp01(1.0 - final_safe_prob)
        return build_response(
            final_phishing_prob=p,
            decision_source="rule_based_prefilter_safe",
            model_name="Rule-Based Prefilter",
            web_used=False
        )

preds = predict_models(
    url=url,
```

```

        cols=cols,
        medians=medians,
        include_web_content=include_web_content,
        precomputed_rule_score=risk_score,
        precomputed_rule_flag=rule_flag
    )

    if preds.get("stack_prob") is not None:
        p = clamp01(float(preds["stack_prob"]))
        source = "ml_stacking_ambiguous_case"
        model_name = "RF + XGB + Logistic Regression Stacking"
    else:
        rf_p = float(preds.get("rf_prob") or 0.0)
        xgb_p = float(preds.get("xgb_prob") or 0.0)
        p = clamp01(0.5 * rf_p + 0.5 * xgb_p)
        source = "ml_rf_xgb_ambiguous_case"
        model_name = "RF + XGB Average (Meta model unavailable)"

    return build_response(
        final_phishing_prob=p,
        decision_source=source,
        model_name=model_name,
        rf_prob=preds.get("rf_prob"),
        xgb_prob=preds.get("xgb_prob"),
        stack_prob=preds.get("stack_prob")
    )

```

Kode 5. 26 Implementasi Rule based Filtering dengan Stacking

Pada Kode 5. 26, implementasi hybrid rule-based dan *stacking* dimulai dengan pengecekan flag *use_prefilter* untuk menentukan apakah prefilter rule-based harus diterapkan. Sistem pertama-tama mengevaluasi dua kondisi *decision*: *hard_phishing* yang terpenuhi ketika *risk score* mencapai *threshold* PREFILTER_HARD_PHISHING_SCORE atau ketika terdapat minimal satu *very important indicator* (*vi_count* ≥ 1), serta *hard_safe* yang terpenuhi ketika *risk score* tidak melebihi PREFILTER_HARD_SAFE_SCORE. Terdapat juga *logic* khusus yang memaksa sistem untuk tidak menganggap URL sebagai *hard_safe* ketika *risk score* adalah 0 jika PREFILTER_FORCE_ML_ON_ZERO_SCORE

diaktifkan, karena ini mengindikasikan bahwa URL tidak memicu *rule-based indicators* dan perlu dievaluasi lebih lanjut oleh *machine learning*.

Jika kondisi *hard_phishing* terpenuhi, sistem langsung mengembalikan *response* dengan probabilitas phishing yang ditetapkan minimal sebesar *PREFILTER_PHISHING_MIN_CONF* dan maksimal sebesar *rule_prob* dari evaluasi *rule-based*. *Decision source* ditandai sebagai *rule_based_prefilter_phishing* dan model *name* ditampilkan sebagai *Rule-Based Prefilter*, dengan *web_used* diset *false* karena *content web* tidak perlu dianalisis untuk kasus ini. Sebaliknya, jika kondisi *hard_safe* terpenuhi, sistem mengembalikan *response* dengan probabilitas *phishing* yang rendah dihitung dari $1.0 - \text{PREFILTER_SAFE_CONF}$, *decision source* diberi label *rule_based_prefilter_safe*, dan proses selesai tanpa melibatkan *machine learning models*.

Untuk yang tidak memenuhi kondisi *hard_phishing* maupun *hard_safe* (*ambiguous cases*), sistem melanjutkan dengan memanggil *predict_models()* untuk mendapatkan prediksi dari random forest, XGBoost, dan *meta-learner stacking*. Jika meta-model tersedia dan berhasil menghasilkan prediksi, probabilitas akhir diambil dari *stack_prob*, *decision source* diset sebagai *ml_stacking_ambiguous_case*, dan model *name* menunjukkan *architecture stacking*. Jika meta-model tidak tersedia, sistem menggunakan fallback dengan menghitung rata-rata tertimbang dari RF dan XGB probabilities, *decision source* menjadi *ml_rf_xgb_ambiguous_case*. Response yang dibangun mencakup semua informasi probabilitas serta *decision_source* yang detailed untuk memungkinkan audit trail yang jelas mengenai bagaimana keputusan akhir diambil.

BAB VI

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil yang diperoleh dari seluruh tahapan penelitian yang telah dilakukan, serta pembahasan terhadap hasil tersebut. Hasil yang ditampilkan mencakup analisis dataset, proses *preprocessing*, implementasi model *Machine Learning*, serta evaluasi kinerja model dalam mendeteksi website phishing. Pembahasan dilakukan untuk menginterpretasikan hasil eksperimen serta menjawab tujuan penelitian yang telah dirumuskan.

6.1 Hasil

Pada subbab ini disajikan hasil dari setiap tahapan yang telah diimplementasikan pada Bab V, mulai dari analisis dataset, *preprocessing* data, hingga implementasi dan pengujian model.

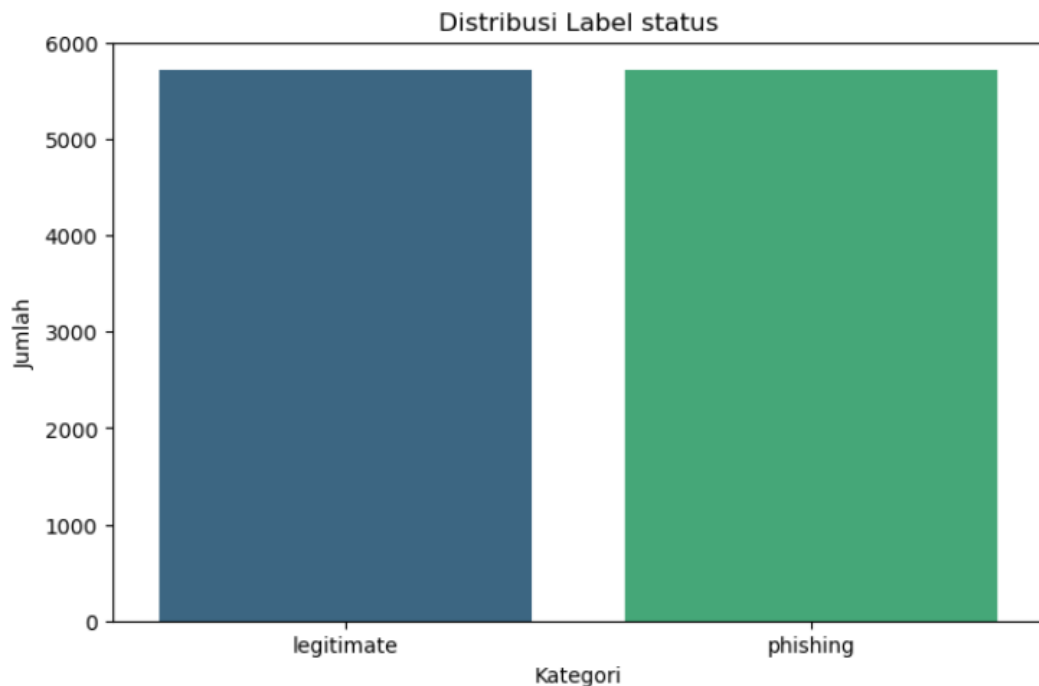
6.1.1 Hasil Analisis Dataset

Pada Tabel 6. 1 dituliskan hasil analisis dataset yang telah dilakukan untuk memahami karakteristik data yang digunakan serta menentukan fitur-fitur yang relevan dalam proses deteksi *website phishing*. Pada tahapan yang sudah dilakukan beberapa proses, antara lain identifikasi jumlah data, distribusi label (*phishing* dan *benign*), pengecekan data duplikat, proses seleksi fitur untuk menentukan atribut yang digunakan dalam pemodelan.

Tabel 6. 1 Hasil Analisis Dataset

No	Karakteristik	Deskripsi
1.	Nama Dataset	dataset_phishing
2.	Sumber	Kaggle
3.	Jumlah Sampel	11.430 baris data
4.	Jumlah Fitur	89 fitur (88 fitur independen + 1 label target)
5.	Label Target	status (0 = Non-phishing, 1 = Phishing)

Selanjutnya dilakukan analisis atribut label untuk mengetahui keseimbangan data antara kelas *phishing* dan *legimate*. Hasil distribusi disajikan pada Gambar berikut.



Gambar 6. 1 Distribusi Kelas

Berdasarkan hasil tersebut, dapat diketahui bahwa dataset memiliki distribusi yang seimbang (*balanced*) antara kelas *phishing* dan *legimate*. Kondisi ini menunjukkan bahwa dataset tidak mengalami ketimpangan kelas (*class imbalance*), sehingga model yang dibangun tidak akan cenderung bias terhadap salah satu kelas. Selain itu, hasil analisis juga menunjukkan adanya 1 data duplikat berdasarkan atribut URL. Meskipun jumlahnya relatif kecil, data duplikat tersebut tetap perlu ditangani pada tahap *preprocessing* untuk menjaga kualitas data dan memastikan setiap sampel bersifat unik.

6.1.2 Hasil Analisis Data Preprocessing

Pada tahapan ini disajikan hasil dari setiap proses data *preprocessing* yang telah dilakukan, meliputi data *selection*, data *transformation*, dan data *cleaning*

6.1.2.1 Hasil Data Selection

Dari total fitur yang tersedia pada dataset dilakukan *data selection*, hanya fitur yang relevan yang dipertahankan, sedangkan fitur yang tidak relevan atau tidak dapat

digunakan dalam skenario deteksi URL dihapus dari dataset. Hasil dari proses ini berupa dataset yang telah disederhanakan dan siap digunakan pada tahap *preprocessing* dan pemodelan. Tabel 6. 2 menunjukkan hasil implementasi dari proses data selection yang telah dilakukan.

Tabel 6. 2 Hasil implementasi data selection

No	Keterangan	Jumlah
1.	Jumlah data (baris)	11.430
2.	Jumlah fitur awal	89 fitur
3.	Jumlah fitur digunakan	83 fitur
4.	Jumlah fitur dihapus	6 fitur
5.	Metode seleksi	Drop column (<i>zero variance & irrelevant features</i>)

6.1.2.2 Hasil Data Transformasi

Pada penelitian ini, proses transformasi difokuskan pada konversi label kelas dari bentuk kategorikal menjadi numerik. Berdasarkan hasil implementasi, kolom status yang sebelumnya berisi nilai kategorikal berupa *phishing* dan *legitimate* telah berhasil dikonversi menjadi bentuk numerik. Nilai *phishing* direpresentasikan sebagai 1, sedangkan *legitimate* direpresentasikan sebagai 0. Selain itu, nama kolom status diubah menjadi label untuk menyesuaikan dengan kebutuhan proses pemodelan.

Perubahan ini dapat dilihat pada Tabel 6. 3 dilakukan agar model *Machine Learning* dapat memproses data secara numerik serta mempermudah interpretasi hasil klasifikasi.

Tabel 6. 3 Hasil Data Transformasi

No	Komponen	Sebelum Transformasi	Setelah Transformasi
1.	Label	phishing legitimate	1 0
2.	Nama kolom	status	label
3.	Tipe data	Kategorikal (<i>string</i>)	Numerik (<i>integer</i>)

6.1.2.3 Hasil Data Cleaning

Pada tahap *data cleaning*, dilakukan proses pembersihan data untuk memastikan kualitas dan integritas dataset sebelum digunakan dalam pemodelan. Fokus utama pada tahap ini adalah penanganan data duplikat yang berpotensi menyebabkan bias dalam proses pembelajaran model. Berdasarkan hasil implementasi, dilakukan identifikasi terhadap baris data yang memiliki nilai URL yang sama. Data yang terdeteksi duplikat kemudian dihapus untuk memastikan bahwa setiap sampel data bersifat unik. Berikut hasil dari proses *data cleaning* yang telah dilakukan, khususnya terkait penghapusan data duplikat, disajikan pada Tabel 6. 4 berikut.

Tabel 6. 4 Hasil Data Cleaning

No	Keterangan	Jumlah
1.	Jumlah data (baris awal)	11.430 data
2.	Jumlah data duplikat	2 data
3.	Jumlah data setelah cleaning	11.429 data
4.	Label Phishing	5715
5.	Label Legitimate	5714

6.1.3 Hasil Model Meachine Learning

Pada tahap ini dituliskan hasil implementasi model *Machine Learning* untuk mendeteksi *website phishing* menggunakan dataset hasil *preprocessing*. Model yang digunakan dalam penelitian ini terdiri dari Random Forest, XGBoost, dan *Stacking Ensemble* yang menggabungkan kedua model tersebut dengan Logistic Regression sebagai *meta-learner*.

6.1.3.1 Hasil Random Forest

Model *Random Forest* diimplementasikan menggunakan *hyperparameter* hasil optimasi *Genetic Algorithm* (GA), sebagai *baseline* model dalam penelitian ini. Berdasarkan hasil implementasi, proses optimasi dilakukan untuk menemukan kombinasi *hyperparameter* yang menghasilkan performa klasifikasi terbaik pada dataset deteksi *phishing* yang digunakan. Model dilatih menggunakan data *train* sebanyak 9.143 sampel dengan 81 fitur *hybrid* yang mencakup fitur berbasis URL dan konten web, kemudian dievaluasi pada data test sebanyak 2.286 sampel.

Konfigurasi *hyperparameter* model Random Forest yang dihasilkan dari proses optimasi GA ditampilkan pada Tabel 6. 5. Model dibangun menggunakan 428 pohon keputusan dengan kedalaman maksimum 16, nilai *min_samples_split* sebesar 4, *min_samples_leaf* sebesar 1, serta pemilihan fitur menggunakan fungsi logaritma basis 2 ($\log_2$).

Tabel 6. 5 Hyperparameter Random Forest

Hyperparameter	Nilai
n_estimators	428
max_depth	16
min_samples_split	4
min_samples_leaf	1
max_features	Log2
random_state	12

Setelah fase pelatihan model selesai dilaksanakan, langkah selanjutnya adalah melakukan evaluasi performa model menggunakan dataset pengujian (*testing data*). Hasil evaluasi tersebut dipresentasikan melalui *confusion matrix* pada Tabel 6. 6, yang mengilustrasikan distribusi frekuensi prediksi model secara mendetail. Melalui tabel tersebut, dapat diobservasi perbandingan antara nilai aktual dengan hasil klasifikasi model, baik untuk kategori *benign* maupun *phishing*, guna mengidentifikasi efektivitas model dalam membedakan kedua kelas tersebut.

Tabel 6. 6 Confusion Matrix Random Forest

	Predicted Benign (0)	Predicted Phishing (1)
Actual Benign (0)	1.111	32
Actual Phishing (1)	32	1.111

Berdasarkan hasil *confusion matrix* pada Tabel 6. 6, model Random Forest menghasilkan 1.111 True Positive (TP) dan 1.111 True Negative (TN), dengan masing-masing 32 False Positive (FP) dan 32 False Negative (FN). Nilai FP menunjukkan jumlah URL benign yang salah diklasifikasikan sebagai *phishing*, sedangkan FN menunjukkan jumlah URL *phishing* yang tidak berhasil terdeteksi

oleh model. Dari nilai-nilai tersebut, diperoleh metrik evaluasi sebagaimana ditampilkan pada Tabel 6. 7.

Tabel 6. 7 Nilai Evaluasi Random Forest

Metrik	Nilai
Accuracy	0.972
Precision	0.972
Recall	0.972
F1-Score	0.972
AUC	0.995

Model Random Forest berhasil mencapai nilai *Accuracy* sebesar 97,20%, yang berarti model mampu mengklasifikasikan 97,20% dari seluruh URL *data test* dengan benar. Nilai *Precision* sebesar 97,20% menunjukkan bahwa dari seluruh URL yang diprediksi sebagai *phishing*, 97,20% di antaranya memang merupakan URL *phishing* yang sebenarnya. Nilai *Recall* sebesar 97,20% menunjukkan bahwa model mampu mendeteksi 97,20% dari seluruh URL *phishing* yang ada dalam *data test*. Nilai F1-Score sebesar 97,20% merupakan rata-rata harmonik dari *Precision* dan *Recall* yang mencerminkan keseimbangan antara keduanya. Keseimbangan sempurna antara seluruh metrik evaluasi ini mengindikasikan bahwa model tidak memiliki kecenderungan bias terhadap salah satu kelas.

Jumlah *false negative* yang relatif lebih tinggi dibandingkan model lain mengindikasikan bahwa Random Forest memiliki keterbatasan dalam menangkap pola *phishing* yang kompleks. Hal ini disebabkan oleh karakteristik algoritma Random Forest yang berbasis *bagging*, di mana setiap *tree* dibangun secara independen sehingga kurang optimal dalam memperbaiki kesalahan klasifikasi secara iteratif. Selain itu, Random Forest cenderung lebih baik dalam menangkap pola umum (*global patterns*), tetapi kurang sensitif terhadap pola-pola minor atau *edge cases* yang sering muncul pada URL *phishing* modern.

6.1.3.2 Hasil XGBoost

Model XGBoost diimplementasikan menggunakan *hyperparameter* hasil optimasi *Genetic Algorithm* (GA). Berbeda dari Random Forest yang membangun pohon secara paralel dan independen, XGBoost menggunakan pendekatan *gradient boosting* di mana setiap pohon baru dibangun secara sekuensial untuk memperbaiki kesalahan prediksi pohon sebelumnya. Koreksi bertahap ini memungkinkan model secara adaptif berfokus pada sampel yang sulit diklasifikasikan. Model dilatih pada *data train* sebanyak 9.143 sampel dengan 81 fitur *hybrid* yang sama, kemudian dievaluasi pada *data test* sebanyak 2.286 sampel.

Konfigurasi *hyperparameter* model XGBoost hasil optimasi GA ditampilkan pada Tabel 6. 8. Model dibangun dengan 356 *estimator* dan *learning rate* sebesar 0,2447, yang mengatur seberapa besar kontribusi setiap pohon baru terhadap prediksi akhir. Nilai *learning rate* yang tidak terlalu kecil ini dikombinasikan dengan jumlah *estimator* yang moderat, menciptakan keseimbangan antara kecepatan konvergensi dan kualitas prediksi. Kedalaman maksimum pohon ditetapkan sebesar 5, jauh lebih dangkal dibandingkan *Random Forest* dengan kedalaman 16, namun karena mekanisme *boosting* memungkinkan kesalahan diperbaiki oleh pohon berikutnya, kedalaman yang lebih dangkal justru membantu model menghindari *overfitting*. Parameter *subsample* sebesar 0,7991 dan *colsample_bytree* sebesar 0,7933 berarti setiap pohon hanya menggunakan sekitar 80% sampel dan 79% fitur secara acak, yang berfungsi memperkuat kemampuan generalisasi model. Selain itu, kombinasi regularisasi L1 (*reg_alpha* = 0,8880) dan L2 (*reg_lambda* = 2,2702) memberikan penalti ganda terhadap kompleksitas model, sementara gamma sebesar 1,2957 menetapkan ambang batas minimum gain yang harus dicapai sebelum sebuah *node* diperbolehkan untuk dipecah lebih lanjut.

Tabel 6. 8 Hyperparameter XGBoost

Hyperparameter	Nilai
n_estimators	365
learning_rate	0,2447
max_depth	5
subsamples	0,7991

Hyperparameter	Nilai
colsample_bytree	0.7933
min_child_weight	6.657
gamma	1.2957
reg_alpha	0.8880
reg_lambda	2.2702
Random_state	12

Setelah proses pelatihan selesai, model dievaluasi terhadap data test. Hasil prediksi model ditampilkan dalam bentuk *confusion matrix* pada Tabel 6. 9.

Tabel 6. 9 Confusion Matrix XGBoost

	Predicted Benign (0)	Predicted Phishing (1)
Actual Benign (0)	1.115	28
Actual Phishing (1)	29	1.114

Berdasarkan *confusion matrix* pada Tabel 6. 9, model XGBoost menghasilkan 1.114 True Positive (TP) dan 1.115 True Negative (TN), dengan 28 False Positive (FP) dan 29 False Negative (FN). Dibandingkan Random Forest yang mencatatkan 32 FP dan 32 FN, XGBoost berhasil menekan kesalahan klasifikasi secara signifikan menjadi hanya 57 total kesalahan dari 64 kesalahan Random Forest, atau berkurang sebesar 10,9%. Nilai FP sebesar 28 yang sedikit lebih kecil dari FN sebesar 29 menunjukkan bahwa model sedikit lebih berhati-hati dalam mengklasifikasikan URL sebagai *phishing*, sehingga lebih sedikit URL benign yang salah dituduh, meskipun konsekuensinya terdapat satu URL *phishing* tambahan yang lolos dari deteksi. Dari nilai-nilai tersebut, diperoleh metrik evaluasi sebagaimana ditampilkan pada Tabel 6. 10.

Tabel 6. 10 Nilai Evaluasi XGBoost

Metrik	Nilai
Accuracy	0.975
Precision	0.975
Recall	0.974
F1-Score	0.975
AUC	0.997

Model XGBoost mencapai nilai *Accuracy* sebesar 97,5%, *Precision* sebesar 97,55%, *Recall* sebesar 97,4%, dan F1-Score sebesar 97,5%. Nilai *Precision* yang sedikit lebih tinggi dari *Recall* mencerminkan kecenderungan model yang lebih konservatif dalam melabeli URL sebagai *phishing*, sejalan dengan distribusi FP dan FN yang diperoleh. Dibandingkan seluruh model yang diimplementasikan, XGBoost mencatatkan nilai tertinggi pada metrik *Accuracy*, *Precision*, F1-Score, dan AUC. Pencapaian ini menunjukkan bahwa mekanisme *gradient boosting* yang dikombinasikan dengan regularisasi berlapis hasil optimasi GA mampu menghasilkan batas keputusan yang lebih presisi dibandingkan pendekatan *bagging* pada Random Forest, meskipun menggunakan jumlah *estimator* yang lebih sedikit.

Dengan demikian, XGBoost lebih sensitif terhadap pola kompleks dan anomali pada data. Selain itu, nilai AUC yang sangat tinggi (0.997) menunjukkan bahwa model memiliki kemampuan diskriminasi yang sangat baik dalam membedakan antara URL *phishing* dan *benign*, bahkan pada berbagai *threshold* klasifikasi. Namun, kompleksitas model yang lebih tinggi juga berimplikasi pada waktu pelatihan yang lebih lama serta kebutuhan *tuning hyperparameter* yang lebih optimal.

6.1.3.3 Hasil Stacking

Model *Stacking Ensemble* diimplementasikan dengan menggabungkan *Random Forest* dan *XGBoost* sebagai *base learner*, serta *Logistic Regression* sebagai *meta-learner*. Berbeda dari voting ensemble yang menggabungkan prediksi *base learner* secara langsung melalui rata-rata atau mayoritas, stacking melatih sebuah model tambahan (*meta-learner*) untuk mempelajari cara terbaik mengombinasikan *output* dari masing-masing *base learner*. Dengan demikian, *meta-learner* diharapkan dapat mengeksplorasi kekuatan komplementer kedua model dasar sekaligus mengompensasi kelemahan masing-masing. Model dilatih pada data train sebanyak 9.143 sampel dan dievaluasi pada data test sebanyak 2.286 sampel.

Konfigurasi arsitektur *Stacking Ensemble* yang digunakan ditampilkan pada Tabel 6. 11. *Stack method* yang digunakan adalah *predict_proba*, yang berarti *meta-learner* tidak menerima label biner (0 atau 1) dari *base learner*, melainkan menerima nilai probabilitas prediksi masing-masing kelas. Pendekatan ini lebih informatif karena probabilitas mengandung tingkat kepercayaan model, sehingga *meta-learner* dapat membedakan prediksi yang diberikan dengan keyakinan tinggi dari prediksi yang masih ragu-ragu di ambang batas. *Logistic Regression* dipilih sebagai *meta-learner* karena sifatnya yang sederhana dan tidak rentan terhadap *overfitting* pada fitur input yang terbatas, serta mampu menghasilkan kombinasi linear optimal dari probabilitas kedua *base learner*. Konfigurasi *class_weight* bernilai *balanced* memastikan *meta-learner* memberikan perhatian proporsional terhadap kedua kelas meskipun terdapat ketidakseimbangan distribusi, sementara *max_iter* sebesar 1.000 memastikan proses optimasi parameter konvergen sepenuhnya.

Tabel 6. 11 Konfigurasi Stacking Ensemble

Komponen	Keterangan
Base Learner 1	Random Forest (GA-tuned)
Base Learner 2	XGBoost(GA-tuned)
Meta-Learner	Logistic Regresion
Stack Method	predict_proba
max_iter(LR)	1000
Class_weight(LR)	balanced

Setelah proses pelatihan selesai, model dievaluasi terhadap data test. Hasil prediksi model ditampilkan dalam bentuk *confusion matrix* pada Tabel 6. 12.

Tabel 6. 12 Confusion Matrix Stacking Ensemble

	Predicted Benign (0)	Predicted Phishing (1)
Actual Benign (0)	1.112	31
Actual Phishing (1)	30	1.113

Berdasarkan *confusion matrix* pada Tabel 6. 12, model Stacking Ensemble menghasilkan 1.113 *True Positive* (TP) dan 1.112 *True Negative* (TN), dengan 31

False Positive (FP) dan 30 False Negative (FN). Total kesalahan klasifikasi sebanyak 61 dari 2.286 URL menempatkan Stacking di antara Random Forest (64 kesalahan) dan XGBoost (57 kesalahan). Hal ini menunjukkan bahwa proses stacking berhasil meningkatkan performa di atas Random Forest, namun tidak berhasil melampaui XGBoost yang sudah sangat teroptimasi. Kondisi ini umum terjadi ketika salah satu *base learner* sudah mencapai tingkat performa yang sangat tinggi, sehingga *meta-learner* tidak mendapatkan sinyal komplementer yang cukup berbeda untuk menghasilkan prediksi yang lebih unggul. Dari nilai-nilai tersebut, diperoleh metrik evaluasi sebagaimana ditampilkan pada Tabel 6. 13.

Tabel 6. 13 Nilai Evaluasi Stacking Ensemble

Metrik	Nilai
Accuracy	0.973
Precision	0.972
Recall	0.973
F1-Score	0.973
AUC	0.996

Model *Stacking Ensemble* mencapai nilai *Accuracy* sebesar 97,33%, *Precision* sebesar 97,29%, *Recall* sebesar 97,38%, dan *F1-Score* sebesar 97,33%. Nilai *Recall* yang sedikit lebih tinggi dari *Precision* menunjukkan bahwa *meta-learner* cenderung sedikit lebih sensitif dalam mendeteksi phishing, yang tercermin dari jumlah FN (30) yang sedikit lebih kecil dari FP (31). Tingginya nilai recall menunjukkan bahwa model ini lebih unggul dalam mendeteksi URL phishing, sehingga mampu meminimalkan jumlah false negative.

Secara keseluruhan, Stacking Ensemble berhasil mengungguli *Random Forest* pada seluruh metrik dan menjadi model dengan performa tertinggi kedua setelah XGBoost dalam penelitian ini. Keunggulan utama *stacking* terletak pada kemampuannya dalam menggabungkan karakteristik dua model yang berbeda yaitu *Random Forest* berfungsi menangkap pola umum dan stabil serta *XGBoost* berfungsi menangkap pola kompleks dan sulit. *Logistic Regression* sebagai *meta-learner* berperan dalam mempelajari kombinasi optimal dari kedua model tersebut,

sehingga mampu memperbaiki kesalahan yang tidak dapat ditangani oleh masing-masing model secara individual. Dengan demikian, *stacking* tidak hanya meningkatkan akurasi, tetapi juga meningkatkan konsistensi model terhadap berbagai jenis data.

6.1.4 Hasil Rule Based Filtering

Pada tahap ini, implementasi metode rule-based filtering dilakukan sebagai lapisan awal dalam sistem deteksi *phishing*. Berdasarkan hasil implementasi, rule-based filtering berhasil mengidentifikasi sebagian URL *phishing* secara langsung tanpa perlu melalui proses klasifikasi *machine learning*. Model *hybrid* diterapkan dengan mengintegrasikan sistem rule-based sebagai lapisan penyaringan awal dan model Stacking Ensemble sebagai lapisan klasifikasi lanjutan. Arsitektur dua lapisan ini berangkat dari pemikiran bahwa tidak semua URL memerlukan analisis *machine learning* yang kompleks. URL yang sudah menunjukkan indikator *phishing* yang sangat jelas secara *heuristik* dapat langsung diklasifikasikan tanpa harus melalui proses komputasi model, sehingga sumber daya komputasi bisa diprioritaskan hanya untuk URL yang ambigu dan membutuhkan analisis lebih mendalam.

Pada lapisan pertama, setiap URL dievaluasi menggunakan sistem penilaian risiko berbasis aturan *heuristik*. Rule-based filtering dalam penelitian ini mengklasifikasikan URL ke dalam tiga kategori: *Phishing*, *Suspicious*, dan *Benign*, berdasarkan jumlah indikator mencurigakan yang terdeteksi pada URL. Sistem ini mengklasifikasikan indikator *phishing* ke dalam tiga tingkat kepentingan, yaitu indikator sangat penting dengan bobot 3 poin, indikator cukup penting dengan bobot 2 poin, dan indikator penting dengan bobot 1 poin. URL yang memperoleh total skor risiko lebih dari atau sama dengan 7, atau memiliki setidaknya satu indikator sangat penting yang terpenuhi, langsung diklasifikasikan sebagai *phishing* dengan nilai *confidence* 95% tanpa melalui model *machine learning*. URL yang tidak memenuhi ambang tersebut diteruskan ke lapisan kedua untuk diproses oleh model *Stacking Ensemble*.

Konfigurasi sistem Hybrid yang digunakan ditampilkan pada Tabel 6. 14. Dari total 2.286 URL data test, sebanyak 418 URL (18,3%) berhasil disaring pada lapisan rule-based dan langsung diklasifikasikan sebagai *phishing*, sementara 1.868 URL (81,7%) sisanya diteruskan ke model *Stacking Ensemble*.

Tabel 6. 14 Konfigurasi Sistem Hybrid

Komponen	Keterangan
Lapisan 1	Rule-Based Filtering
Amambang Skor Phishing	≥ 7
Confidence Rule-Based	95%
Lapisan 2	Stacking Ensemble
URL diproses Rule-Based	418 URL (18,3%)
URL diproses Stacking	1.868 URL (81,7%)

Berdasarkan tabel di atas, terlihat bahwa sebagian besar data berada pada kategori *Suspicious*, yaitu sebanyak 1.868 data. Hal ini menunjukkan bahwa banyak URL memiliki indikasi mencurigakan, namun belum cukup kuat untuk langsung dikategorikan sebagai *phishing*. Kategori *Phishing* sebanyak 418 data menunjukkan jumlah URL yang dapat langsung terdeteksi sebagai berbahaya oleh rule-based tanpa perlu melalui model *Machine Learning*. Hal ini menunjukkan bahwa rule-based filtering mampu menangkap pola *phishing* yang jelas secara efektif.

Setelah proses klasifikasi selesai pada kedua lapisan, hasil prediksi digabungkan dan dievaluasi terhadap label sebenarnya. Hasil prediksi model ditampilkan dalam bentuk confusion matrix pada Tabel 6. 15.

Tabel 6. 15 Confusion Matrix Hybrid

	Predicted Benign (0)	Predicted Phishing (1)
Actual Benign (0)	1.061	82
Actual Phishing (1)	29	1.114

Berdasarkan *confusion matrix* pada Tabel 6. 15, model *Hybrid* menghasilkan 1.114 True Positive (TP) dan 1.061 True Negative (TN), dengan 82 False Positive (FP) dan 29 False Negative (FN). Pola distribusi kesalahan model *Hybrid* berbeda secara mencolok dibandingkan ketiga model sebelumnya. Nilai FN sebesar 29 merupakan yang terendah bersama XGBoost, artinya model *Hybrid* sangat jarang membiarkan URL *phishing* lolos tanpa terdeteksi. Namun di sisi lain, nilai FP sebesar 82 merupakan yang tertinggi di antara seluruh model, hampir tiga kali lipat dari model lainnya. Lonjakan FP ini bersumber dari lapisan rule-based yang bersifat konservatif aturan heuristik tidak dapat membedakan URL benign yang memiliki karakteristik teknis mencurigakan (misalnya domain dengan entropi tinggi atau hostname yang panjang) dari URL *phishing* yang sesungguhnya. Begitu lapisan rule-based memutuskan sebuah URL sebagai *phishing*, keputusan tersebut bersifat final dan tidak dapat dikoreksi oleh *Stacking Ensemble*.

Evaluasi rule-based dilakukan dengan menggunakan evaluasi metrik untuk melihat kemampuan rule dalam mendeteksi *phishing*. Hasil dari evaluasi metrik model *Hybrid* dengan penerapan *Rule-Based* dapat dilihat pada Tabel 6. 16.

Tabel 6. 16 Nilai Evaluasi Hybrid

Metrik	Nilai
Accuracy	0.951
Precision	0.931
Recall	0.974
F1-Score	0.952
AUC	0.983

Model *Hybrid* (Rule-Based + *Stacking*) menghasilkan nilai *Accuracy* sebesar 0.951, *Precision* sebesar 0.931, *Recall* sebesar 0.974, F1-Score sebesar 0.952, dan AUC sebesar 0.983. Nilai *recall* yang tinggi (0.974) menunjukkan bahwa model memiliki kemampuan yang sangat baik dalam mendeteksi URL *phishing*, sehingga hanya sedikit kasus *phishing* yang tidak teridentifikasi (*false negative* rendah). Hal ini disebabkan oleh adanya lapisan rule-based yang secara eksplisit dirancang untuk

menangkap pola-pola *phishing* yang sudah diketahui, sehingga meningkatkan sensitivitas sistem terhadap berbagai ancaman yang bersifat eksplisit.

Di sisi lain, nilai *precision* yang lebih rendah (0.931) mengindikasikan adanya peningkatan jumlah *false positive*. Namun, dalam konteks arsitektur *hybrid*, fenomena ini tidak sepenuhnya dapat dianggap sebagai kelemahan. *False positive* yang muncul pada tahap awal (*rule-based*) pada dasarnya berperan sebagai mekanisme *Suspicious flag*, yaitu menandai URL yang mencurigakan untuk kemudian diperiksa kembali pada lapisan kedua menggunakan model *machine learning* (*Stacking Ensemble*). Dengan demikian, URL yang terdeteksi mencurigakan tidak langsung dianggap sebagai kesalahan akhir, melainkan melalui proses verifikasi ulang yang lebih kompleks. Pendekatan ini mencerminkan konsep *defense-in-depth*, di mana sistem melakukan validasi berlapis untuk meningkatkan keandalan deteksi.

Akibat dari mekanisme ini, sistem menjadi lebih sensitif terhadap potensi ancaman dan cenderung “*over-detect*” pada tahap awal. Hal ini memang berdampak pada penurunan nilai *accuracy* menjadi 0.951, karena meningkatnya jumlah *false positive* memengaruhi perhitungan akurasi secara keseluruhan. Namun demikian, nilai F1-Score sebesar 0.952 menunjukkan bahwa keseimbangan antara *precision* dan *recall* masih terjaga dengan baik. Selain itu, nilai AUC sebesar 0.983 mengindikasikan bahwa secara umum model tetap memiliki kemampuan yang sangat baik dalam membedakan antara URL *phishing* dan non-*phishing*.

Secara keseluruhan, pendekatan *Hybrid* tidak hanya berfokus pada hasil klasifikasi akhir, tetapi juga pada proses deteksi yang lebih aman dan berlapis. *False positive* dalam konteks ini dapat dipahami sebagai bagian dari strategi mitigasi risiko, di mana sistem lebih memilih untuk memeriksa ulang URL yang mencurigakan daripada melewati potensi ancaman *phishing*. Oleh karena itu, meskipun secara metrik tradisional terlihat mengalami penurunan *precision* dan *accuracy*, model Hybrid justru menawarkan keunggulan dalam aspek keamanan dan keandalan

deteksi, sehingga lebih sesuai untuk sistem yang memprioritaskan perlindungan maksimal terhadap serangan *phishing*.

6.1.5 Hasil Perbandingan Kinerja Model

Berdasarkan hasil evaluasi kinerja model yang telah dilakukan terhadap seluruh data uji sebanyak 2286 URL, terlihat adanya perbedaan karakteristik performa yang signifikan pada masing-masing model, yaitu Random Forest, XGBoost, Stacking Ensemble, dan Hybrid (Rule-Based + Stacking). Perbedaan ini tidak hanya terlihat dari nilai metrik utama seperti *accuracy*, *precision*, *recall*, F1-score, dan AUC, tetapi juga mencerminkan perbedaan mendasar dalam pendekatan pembelajaran yang digunakan oleh masing-masing model dalam mendeteksi pola *phishing*.

Model Random Forest menunjukkan performa yang stabil dengan nilai *accuracy* sebesar 0.972003 serta distribusi kesalahan yang seimbang antara *false positive* (32) dan *false negative* (32). Keseimbangan ini menunjukkan bahwa model memiliki kemampuan klasifikasi yang konsisten tanpa kecenderungan bias terhadap salah satu kelas. Hal ini disebabkan oleh pendekatan *bagging* yang digunakan, di mana banyak pohon keputusan dibangun secara independen menggunakan *subset* data yang berbeda, kemudian hasilnya digabungkan melalui mekanisme voting. Pendekatan ini efektif dalam menangkap pola umum dan meningkatkan generalisasi model, serta mengurangi risiko *overfitting*. Namun, karena setiap pohon bekerja secara independen tanpa mekanisme pembelajaran berurutan, Random Forest memiliki keterbatasan dalam menangkap pola *phishing* yang kompleks dan tidak linear. Akibatnya, masih terdapat sejumlah *false negative* yang menunjukkan bahwa beberapa URL *phishing* belum berhasil terdeteksi.

Selanjutnya, model XGBoost menunjukkan performa terbaik secara keseluruhan dengan nilai *accuracy* tertinggi sebesar 0.975066, serta jumlah kesalahan klasifikasi yang paling rendah (*false positive* = 28 dan *false negative* = 29). Keunggulan ini disebabkan oleh pendekatan *boosting*, di mana model dibangun secara bertahap dengan fokus utama memperbaiki kesalahan dari model sebelumnya. Dengan demikian, XGBoost mampu secara adaptif mempelajari pola-pola sulit, termasuk

karakteristik *phishing* yang kompleks dan tersembunyi. Selain itu, penerapan teknik optimasi seperti *regularization* dan *gradient-based learning* membuat model tidak hanya lebih akurat, tetapi juga lebih efisien dan tahan terhadap *overfitting*. Hal ini menjadikan XGBoost sebagai model yang unggul baik dari sisi performa klasifikasi maupun efisiensi pembelajaran.

Pada tahap selanjutnya, model Stacking Ensemble menggabungkan beberapa model dasar, yaitu Random Forest dan XGBoost, untuk menghasilkan prediksi akhir yang lebih *robust* melalui *meta-learner*. Pendekatan ini memungkinkan model untuk memanfaatkan keunggulan masing-masing algoritma, di mana Random Forest berkontribusi dalam menangkap pola umum, sementara XGBoost berperan dalam mendeteksi pola yang lebih kompleks. Hasilnya, model Stacking menunjukkan kemampuan generalisasi yang lebih baik, terutama dalam meningkatkan sensitivitas terhadap URL *phishing*. Hal ini terlihat dari nilai *recall* yang tinggi (0.973753), yang menunjukkan bahwa model mampu meminimalkan jumlah *false negative*. Selain itu, distribusi kesalahan yang relatif seimbang menunjukkan bahwa model memiliki stabilitas yang baik. Namun, peningkatan performa ini diperoleh dengan konsekuensi meningkatnya kompleksitas komputasi, karena proses prediksi harus melalui beberapa model sekaligus sebelum menghasilkan keputusan akhir.

Model *Hybrid (Rule-Based + Stacking)* menghadirkan pendekatan yang berbeda dengan menggabungkan metode berbasis aturan (*rule-based*) dan *machine learning*. Pada tahap awal, sistem menggunakan rule-based untuk mendeteksi indikator-indikator *phishing* yang bersifat eksplisit dan telah diketahui sebelumnya. Pendekatan ini memungkinkan deteksi cepat terhadap pola-pola yang jelas berbahaya, sehingga meningkatkan nilai *recall* menjadi 0.974628 dan menurunkan jumlah *false negative*. URL yang terindikasi mencurigakan pada tahap ini tidak langsung dianggap sebagai kesalahan akhir, melainkan diperlakukan sebagai *Suspicious flag* yang kemudian diperiksa kembali oleh model Stacking pada lapisan kedua. Mekanisme ini menciptakan proses deteksi berlapis (*multi-layer detection*) yang meningkatkan keamanan sistem.

Namun, pendekatan ini juga menyebabkan peningkatan jumlah *false positive* (82) yang berdampak pada penurunan *precision* menjadi 0.931 dan *accuracy* menjadi 0.951. Meskipun secara metrik terlihat sebagai kelemahan, dalam konteks sistem keamanan hal ini justru dapat dipahami sebagai strategi defensif, di mana sistem lebih memilih untuk melakukan pemeriksaan ulang terhadap URL yang mencurigakan daripada melewatkan potensi ancaman. Dengan kata lain, *false positive* dalam model *Hybrid* bukan semata kesalahan, melainkan bagian dari mekanisme penyaringan awal sebelum dilakukan validasi lebih lanjut oleh *machine learning*. Konsekuensinya, sistem menjadi lebih sensitif dan aman, meskipun harus mengorbankan sebagian akurasi dan meningkatkan beban komputasi.

Tabel 6. 17 Hasil Perbandingan Kinerja Model

Model	Accur acy	Precis ion	Recall	F1- Score	AUC	TP	TN	FP	FN
Random Forest	0.972	0.972	0.972	0.972	0.995	1111	1111	32	32
XGBoost	0.975	0.974	0.974	0.975	0.996	1114	1115	28	29
Stacking	0.973	0.973	0.973	0.973	0.996	1113	1112	31	30
Hybrid	0.951	0.931	0.974	0.952	0.983	1114	1061	82	29

Secara keseluruhan, perbedaan performa antar model dipengaruhi oleh cara masing-masing model dalam melakukan pembelajaran. Random Forest cenderung stabil karena menggabungkan banyak model independen, XGBoost menunjukkan hasil terbaik karena mampu memperbaiki kesalahan secara bertahap, sedangkan Stacking menggabungkan beberapa model untuk meningkatkan hasil prediksi. Sementara itu, pendekatan *Hybrid* menambahkan lapisan deteksi awal yang membuat sistem lebih sensitif terhadap potensi *phishing*.

6.1.5 Hasil Eksperimen

Pada subbab ini disajikan hasil eksperimen yang dilakukan untuk mengevaluasi kinerja model dalam mendeteksi *website phishing*. Eksperimen dilakukan dengan membandingkan beberapa model yang telah diimplementasikan, yaitu *Random Forest*, *XGBoost*, *Stacking Ensemble*, serta kombinasi dengan metode *rule-based*

filtering. Pada subbab ini disajikan hasil eksperimen menggunakan masing masing model terhadap data uji yang terdiri dari 40 URL, yaitu 20 URL *benign* dan 20 URL *phishing*. Pengujian dilakukan secara manual dengan melihat probabilitas hasil prediksi model terhadap masing-masing URL.

Eksperimen ini mengevaluasi kinerja algoritma *Random Forest* dalam mendeteksi tautan (URL) *phishing*. Aturan keputusan (*decision rule*) yang ditetapkan dalam eksperimen ini adalah penggunaan nilai ambang batas (*threshold*) sebesar 0.60. URL dengan probabilitas prediksi <0.60 diklasifikasikan sebagai *benign* (aman), sedangkan URL dengan probabilitas >0.60 dikategorikan sebagai *phishing* (berbahaya).

6.1.5.1 Hasil Eksperimen Random Forest

Pada subbab ini disajikan hasil eksperimen model Random Forest pada data uji sebanyak 40 URL, terdiri dari 20 URL *benign* dan 20 URL *phishing*. Penentuan kelas dilakukan berdasarkan probabilitas kelas tertinggi pada setiap URL.

Berdasarkan pengujian, Random Forest menunjukkan performa baik dengan berhasil mengklasifikasikan seluruh sampel data *benign* secara konsisten dengan rentang probabilitas yang aman (rata rata 0.31 hingga 0.51). Seluruh URL *benign* dikenali sebagai *benign*. Keberhasilan ini dipengaruhi oleh arsitektur *Random Forest* yang mampu mengenali pola terstruktur dan fitur leksikal standar dari domain resmi yang sudah umum, seperti *python.org*, *kubernetes.io*, dan *ubuntu.com*. Fitur-fitur tersebut dikenali oleh pohon keputusan (*decision trees*) di dalam model sebagai karakteristik dengan risiko yang sangat rendah, sehingga tingkat False Positive atau pemblokiran terhadap URL sah dapat ditekan secara maksimal. Di sisi lain, untuk kasus *phishing* dengan struktur domain yang sangat acak, model dapat mendeteksinya dengan mudah. Tingginya entropi atau keacakan karakter serta panjang URL yang tidak wajar menjadi penyebab utama model memberikan skor probabilitas tinggi yang berhasil melewati ambang batas 0.60.

Tabel 6. 18 Hasil Eksperimen Random Forest

URL	Label	Persentase
https://www.ietf.org/	benign	0.34
https://www.python.org/downloads/	benign	0.32
https://pytorch.org/	benign	0.51
https://kubernetes.io/docs/home/	benign	0.50
https://www.r-project.org/	benign	0.39
https://julialang.org/	benign	0.51
https://www.rust-lang.org/learn	benign	0.34
https://www.postgresql.org/docs/	benign	0.34
https://www.sqlite.org/index.html	benign	0.33
https://git-scm.com/docs	benign	0.50
https://www.apache.org/licenses/	benign	0.32
https://www.debian.org/releases/stable/	benign	0.31
https://ubuntu.com/download	benign	0.49
https://www.mozilla.org/firefox/new/	benign	0.32
https://www.kernel.org/	benign	0.35
https://www.freebsd.org/	benign	0.34
https://go.dev/doc/	benign	0.58
https://numpy.org/doc/	benign	0.51
https://pandas.pydata.org/docs/	benign	0.51
https://scikit-learn.org/stable/	benign	0.46
https://verify-session-paypal.net/webscr/login.php	phising	0.53
https://account-security-microsoft365.com/validate/index.html	phising	0.62
https://appleid-confirmation.support-case.net/signin	phising	0.64
https://drive-shared-docs.verify-user.co/open/view.php	phising	0.62
https://secure-banking-alert.com/auth/update	phising	0.62
https://office365-protect-mail.com/owa/auth	phising	0.57
https://docusign-review-file.com/secure/document	phising	0.54
https://dropbox-file-check.net/login/verify	phising	0.58
https://webmail-security-check.org/account/recover	phising	0.54
https://confirm-amazon-billing.com/a-to-z/login	phising	0.50
https://banking-notice-center.com/customer/verify	phising	0.57
https://outlook-auth-session.net/portal/login	phising	0.55
http://nmhokdypni.sum4iit.club/vnafvra97w/?q=3717065149&id=100	phising	0.65
https://crypto-wallet-verification.org/restore	phising	0.47

URL	Label	Persentase
http://www.kueronekayacotn-co-jp.kueronekayacotn.rvxxkq.top/ai/?authenticated=true&openid/gp/signin/x&i=a&oauth=m&i?ie=utf8&ref_=rhf_custrec_signin6a906d6920fa43de76f847daeb0940fff77e45ae	phising	0.80
https://linkedin-notification-center.com/auth	phising	0.46
https://faturarenner.lojavirtual.com.br/?gclid=eaiaiqobchmi9ukc2pmr_qivcmqgch0ugg5_eaayasaagjbb_d_bwe	phising	0.64
https://icloud-security-update.net/account/verify	phising	0.58
https://payment-confirmation-gateway.com/signin	phising	0.62
https://account-recovery-helpdesk.org/validate	phising	0.57

Pada Tabel 6. 18 menjelaskan hasil eksperimen ini adalah tingginya angka *False Negatives* pada kelas *phishing* terstruktur. Terdapat anomali di mana URL yang memiliki label asli *phishing* justru diprediksi sebagai *benign* karena probabilitasnya berada di area ambiguitas, yakni antara 0.46 hingga 0.58, sehingga gagal menembus *threshold* 0,60. Kegagalan prediksi ini bukan disebabkan oleh kelemahan logik algoritma, melainkan akibat dari teknik rekayasa sosial atau kamuflase leksikal (*lexical deception*) yang diterapkan oleh penyerang. Penyerang menyisipkan kata-kata kunci tepercaya ke dalam URL, seperti *verify*, *secure*, *support*, *protect*, serta nama merek seperti *paypal* atau *office365*, yang secara statistik sering muncul pada data latih di kelas *benign*.

Secara keseluruhan, hasil eksperimen menunjukkan bahwa model memiliki performa yang baik dalam mendeteksi URL dengan pola yang jelas, baik *benign* maupun *phishing* ekstrem, namun memiliki keterbatasan dalam mendeteksi *phishing* yang dirancang secara lebih halus. Oleh karena itu, diperlukan pendekatan yang lebih adaptif, seperti penyesuaian *threshold* atau penggunaan model *hybrid*, untuk meningkatkan kemampuan deteksi terutama dalam menghadapi teknik *phishing* modern yang semakin kompleks.

6.1.5.2 Hasil Eksperimen XGBoost

Pada subbab ini disajikan hasil eksperimen model XGBoost pada data uji sebanyak 40 URL, terdiri dari 20 URL *benign* dan 20 URL *phishing*. Penentuan kelas dilakukan berdasarkan probabilitas kelas tertinggi pada setiap URL.

Berdasarkan hasil pengujian, model XGBoost menunjukkan performa yang sangat baik dalam mengklasifikasikan URL. Sebagian besar URL *phishing* berhasil dikenali dengan benar dengan tingkat keyakinan yang tinggi. Model mampu mengidentifikasi pola-pola mencurigakan pada URL *phishing* secara efektif.

Tabel 6. 19 Hasil Eksperimen XGBoost

URL	Label	Persentase
https://www.ietf.org/	benign	0,07
https://www.python.org/downloads/	benign	0,34
https://pytorch.org/	benign	0,69
https://kubernetes.io/docs/home/	benign	0,92
https://www.r-project.org/	benign	0,10
https://julialang.org/	benign	0,75
https://www.rust-lang.org/learn	benign	0,31
https://www.postgresql.org/docs/	benign	0,34
https://www.sqlite.org/index.html	benign	0,33
https://git-scm.com/docs	benign	0,84
https://www.apache.org/licenses/	benign	0,32
https://www.debian.org/releases/stable/	benign	0,55
https://ubuntu.com/download	benign	0,80
https://www.mozilla.org/firefox/new/	benign	0,52
https://www.kernel.org/	benign	0,06
https://www.freebsd.org/	benign	0,07
https://go.dev/doc/	benign	0,93
https://numpy.org/doc/	benign	0,80
https://pandas.pydata.org/docs/	benign	0,69
https://scikit-learn.org/stable/	benign	0,60
https://verify-session-paypal.net/webscr/login.php	phising	0,87
https://account-security-microsoft365.com/validate/index.html	phising	0,83
https://appleid-confirmation.support-case.net/signin	phising	0,94

URL	Label	Persentase
https://drive-shared-docs.verify-user.co/open/view.php	phising	0,85
https://secure-banking-alert.com/auth/update	phising	0,93
https://office365-protect-mail.com/owa/auth	phising	0,95
https://docusign-review-file.com/secure/document	phising	0,65
https://dropbox-file-check.net/login/verify	phising	0,87
https://webmail-security-check.org/account/recover	phising	0,69
https://confirm-amazon-billing.com/a-to-z/login	phising	0,72
https://banking-notice-center.com/customer/verify	phising	0,83
https://outlook-auth-session.net/portal/login	phising	0,85
http://nmhokdypni.sum4iit.club/vnafvra97w/?q=3717065149&id=100	phising	0,93
https://crypto-wallet-verification.org/restore	phising	0,26
http://www.kueronekayacotn-co-jp.kueronekayacotn.rvxxkq.top/ai/?authenticated=true&openid/gp/signin/x&i=a&oauth=m&i?ie=utf8&ref_rhf_custrec_signin6a906d6920fa43de76f847daeb0940fff77e45ae	phising	0,94
https://linkedin-notification-center.com/auth	phising	0,35
https://faturarenner.lojavirtual.com.br/?gclid=caiaiobchmi9uke2pmr_qivcmqgch0ugg5_eaayasaagjbb_d_bwe	phising	0,76
https://icloud-security-update.net/account/verify	phising	0,81
https://payment-confirmation-gateway.com/signin	phising	0,91
https://account-recovery-helpdesk.org/validate	phising	0,69

Pada Tabel 6. 19, model XGBoost menunjukkan tingkat akurasi yang tinggi, namun tidak mencapai 100% karena masih terdapat beberapa kesalahan klasifikasi. Kesalahan ini ditandai dengan adanya URL yang seharusnya termasuk dalam kategori *benign* tetapi diprediksi sebagai *phishing*, maupun sebaliknya.

Pada URL *phishing*, skor keyakinan prediksi berada pada rentang sekitar 35% hingga 94%, sedangkan pada URL *benign* terdapat beberapa data yang tidak terklasifikasi dengan tepat oleh model. Meskipun demikian, sebagian besar URL tetap berhasil diklasifikasikan dengan benar.

6.1.5.3 Hasil Eksperimen Stacking Ensemble

Pada subbab ini dilakukan pengujian model Stacking Ensemble pada data uji berjumlah 40 URL, terdiri atas 20 URL *benign* dan 20 URL phishing. Nilai persentase pada tabel merepresentasikan skor probabilitas phishing yang dihasilkan model untuk setiap URL. Berdasarkan ambang keputusan 0,60, model mengklasifikasikan 18 dari 20 URL phishing dengan benar, sedangkan pada kelas *benign* model mengklasifikasikan 12 dari 20 URL dengan benar. Hasil ini menunjukkan bahwa Stacking Ensemble memiliki kemampuan deteksi *phishing* yang kuat, meskipun masih terdapat kesalahan klasifikasi pada sebagian URL *benign*.

Tabel 6. 20 Hasil Eksperimen Stacking Ensemble

URL	Label	Persentase
https://www.ietf.org/	benign	0,08
https://www.python.org/downloads/	benign	0,46
https://pytorch.org/	benign	0,64
https://kubernetes.io/docs/home/	benign	0,77
https://www.r-project.org/	benign	0,12
https://julialang.org/	benign	0,68
https://www.rust-lang.org/learn	benign	0,16
https://www.postgresql.org/docs/	benign	0,17
https://www.sqlite.org/index.html	benign	0,15
https://git-scm.com/docs	benign	0,73
https://www.apache.org/licenses/	benign	0,14
https://www.debian.org/releases/stable/	benign	0,24
https://ubuntu.com/download	benign	0,69
https://www.mozilla.org/firefox/new/	benign	0,23
https://www.kernel.org/	benign	0,08
https://www.freebsd.org/	benign	0,08
https://go.dev/doc/	benign	0,86
https://numpy.org/doc/	benign	0,72
https://pandas.pydata.org/docs/	benign	0,64
https://scikit-learn.org/stable/	benign	0,50
https://verify-session-paypal.net/webscr/login.php	phising	0,79
https://account-security-microsoft365.com/validate/index.html	phising	0,86

URL	Label	Persentase
https://appleid-confirmation.support-case.net/signin	phising	0,91
https://drive-shared-docs.verify-user.co/open/view.php	phising	0,86
https://secure-banking-alert.com/auth/update	phising	0,89
https://office365-protect-mail.com/owa/auth	phising	0,86
https://docusign-review-file.com/secure/document	phising	0,67
https://dropbox-file-check.net/login/verify	phising	0,84
https://webmail-security-check.org/account/recover	phising	0,69
https://confirm-amazon-billing.com/a-to-z/login	phising	0,65
https://banking-notice-center.com/customer/verify	phising	0,81
https://outlook-auth-session.net/portal/login	phising	0,80
http://nmhokdypni.sum4iit.club/vnafvra97w/?q=3717065149 &id=100	phising	0,91
https://crypto-wallet-verification.org/restore	phising	0,28
http://www.kueronekayacotn-co- jp.kueronekayacotn.rvxxkq.top/ai/?authenticated=true& amp;openid/gp/signin/x&i=a&oauth=m &i?ie=utf8&ref_=rhf_custrec_signin6a9 06d6920fa43de76f847daeb0940fff77e45ae	phising	0,97
https://linkedin-notification-center.com/auth	phising	0,32
https://faturarenner.lojavirtual.com.br/?gclid=eaiaiqobchmi9 uke2pmr_qivcmqgch0ugg5_eaayasaagjbb_d_bwe	phising	0,85
https://icloud-security-update.net/account/verify	phising	0,81
https://payment-confirmation-gateway.com/signin	phising	0,88
https://account-recovery-helpdesk.org/validate	phising	0,73

Pada Tabel 6. 20, model Stacking Ensemble memperoleh akurasi 75,0% dengan 30 prediksi benar dari total 40 data uji, serta tingkat kesalahan 25,0% atau 10 prediksi salah. Pada kelas *phishing*, model menghasilkan 18 prediksi benar dan 2 prediksi salah. Pada kelas *benign*, model menghasilkan 12 prediksi benar dan 8 prediksi salah. Temuan ini menunjukkan bahwa performa model sudah baik untuk mendeteksi *phishing*, namun masih perlu peningkatan agar kesalahan pada URL *benign* dapat ditekan.

6.1.5.4 Hasil Eksperimen Rule based Filtering dengan Stacking

Pada subbab ini disajikan hasil eksperimen metode Rule-Based Filtering yang dikombinasikan dengan Stacking pada data uji sebanyak 40 URL, terdiri dari 20 URL benign dan 20 URL *phishing*. Metode ini menggabungkan aturan berbasis karakteristik URL dengan hasil prediksi dari beberapa model untuk menghasilkan keputusan akhir. Penentuan kelas dilakukan berdasarkan hasil akhir dari proses *stacking* yang telah difilter menggunakan rule-based.

Tabel 6. 21 Hasil Eksperimen Rule based Filtering dengan Stacking

URL	Label	Persentase
https://www.ietf.org/	benign	0,08
https://www.python.org/downloads/	benign	0,46
https://pytorch.org/	benign	0,64
https://kubernetes.io/docs/home/	benign	0,77
https://www.r-project.org/	benign	0,12
https://julialang.org/	benign	0,68
https://www.rust-lang.org/learn	benign	0,16
https://www.postgresql.org/docs/	benign	0,17
https://www.sqlite.org/index.html	benign	0,15
https://git-scm.com/docs	benign	0,73
https://www.apache.org/licenses/	benign	0,14
https://www.debian.org/releases/stable/	benign	0,24
https://ubuntu.com/download	benign	0,69
https://www.mozilla.org/firefox/new/	benign	0,23
https://www.kernel.org/	benign	0,08
https://www.freebsd.org/	benign	0,08
https://go.dev/doc/	benign	0,86
https://numpy.org/doc/	benign	0,72
https://pandas.pydata.org/docs/	benign	0,64
https://scikit-learn.org/stable/	benign	0,50
https://verify-session-paypal.net/webscr/login.php	phising	0,79
https://account-security-microsoft365.com/validate/index.html	phising	0,86
https://appleid-confirmation.support-case.net/signin	phising	0,91
https://drive-shared-docs.verify-user.co/open/view.php	phising	0,86
https://secure-banking-alert.com/auth/update	phising	0,89

URL	Label	Persentase
https://office365-protect-mail.com/owa/auth	phising	0,86
https://docuSign-review-file.com/secure/document	phising	0,67
https://dropbox-file-check.net/login/verify	phising	0,84
https://webmail-security-check.org/account/recover	phising	0,69
https://confirm-amazon-billing.com/a-to-z/login	phising	0,65
https://banking-notice-center.com/customer/verify	phising	0,81
https://outlook-auth-session.net/portal/login	phising	0,80
http://nmhokdypni.sum4iit.club/vnafvra97w/?q=3717065149&id=100	phising	0,95
https://crypto-wallet-verification.org/restore	phising	0,28
http://www.kueronekayacotn-co-jp.kueronekayacotn.rvxxkq.top/ai/?authenticated=true&openid/gp/signin/x&i=a&oauth=m&i?ie=utf8&ref_rhf_custrec_signin6a906d6920fa43de76f847daeb0940fff77e45ae	phising	0,95
https://linkedin-notification-center.com/auth	phising	0,32
https://faturarenner.lojavirtual.com.br/?gclid=caiaiqobchmi9ukc2pmr_qivcmqgch0ugg5_eaayasaagjbb_d_bwe	phising	0,95
https://icloud-security-update.net/account/verify	phising	0,81
https://payment-confirmation-gateway.com/signin	phising	0,88
https://account-recovery-helpdesk.org/validate	phising	0,73

Pada Tabel 6. 21, metode Rule-Based Filtering dengan Stacking memperoleh akurasi sebesar 75,0% dengan 30 prediksi benar dari total 40 data uji, serta tingkat kesalahan sebesar 25,0% atau sebanyak 10 prediksi salah. Pada kelas *phishing*, model menghasilkan 18 prediksi benar dan 2 prediksi salah, sedangkan pada kelas *benign* terdapat 12 prediksi benar dan 8 prediksi salah. Data yang ditandai dengan warna kuning menunjukkan kesalahan klasifikasi, yaitu ketika label hasil prediksi tidak sesuai dengan label asli, sementara warna hijau menunjukkan data yang diklasifikasikan langsung menggunakan *rule-based* tanpa melalui proses *stacking*.

Meskipun jumlah prediksi benar dan salah pada pendekatan *Stacking Ensemble* dan *Rule-Based Stacking* sama persis pada data uji ini, terdapat perbedaan penting pada mekanisme klasifikasi, di mana metode *Rule-Based Stacking* mampu

mengklasifikasikan sebagian data secara langsung melalui aturan *rule*, sehingga proses menjadi lebih efisien meskipun tingkat akurasi tetap sama.

6.2 Pembahasan

Subbab ini membahas hasil eksperimen yang telah dilakukan terhadap model deteksi yang sudah dibangun menggunakan dataset terbaru. Analisis difokuskan pada perbandingan kinerja masing-masing model dalam mengklasifikasikan URL *phishing* dan *benign*, serta mengidentifikasi model yang menghasilkan performa terbaik berdasarkan metrik evaluasi yang telah ditetapkan.

6.2.1 Pembahasan Eksperimen

Subbab ini membahas hasil eksperimen berdasarkan dua pertanyaan penelitian yang telah dirumuskan sebelumnya. Eksperimen dilakukan terhadap empat pendekatan model, yaitu Random Forest, XGBoost, Stacking Ensemble, serta kombinasi Rule-Based Filtering dengan *Stacking*, menggunakan 40 URL data uji yang terdiri dari 20 URL *benign* dan 20 URL *phishing*.

6.2.1.1 Pembahasan Kinerja Model Tunggal Random Forest.

Berdasarkan hasil eksperimen pada subbab 6.1.5.1, hasil eksperimen menunjukkan bahwa model Random Forest secara umum mampu mengklasifikasikan URL *benign* dengan probabilitas prediksi yang konsisten rendah, berkisar antara 0,31 hingga 0,58. Namun, apabila dibandingkan dengan hasil evaluasi pada dataset penuh yang hanya mencapai akurasi sekitar 97,20% dengan jumlah *false negative* sebanyak 32, maka terlihat adanya perbedaan performa yang cukup signifikan. Kesenjangan ini mengindikasikan bahwa pengujian dengan sampel kecil tidak mampu merepresentasikan kompleksitas pola URL secara keseluruhan. Data uji manual cenderung terdiri dari URL dengan pola yang lebih mudah dikenali, sehingga tidak mencerminkan tantangan nyata dalam deteksi *phishing* modern.

Dari sisi *confidence score*, model *Random Forest* menunjukkan rentang probabilitas, yaitu sekitar 0,31–0,58 untuk URL *benign* dan 0,46–0,80 untuk URL *phishing*. Rentang ini menunjukkan bahwa model sering berada pada kondisi

keputusan yang mendekati batas, sehingga tingkat keyakinan prediksinya tidak terlalu kuat.

Dari perspektif pertanyaan penelitian pertama, *Random Forest* sebagai model tunggal memiliki keunggulan dalam stabilitas dan kemampuan generalisasi pada pola umum. Namun, kelemahannya terletak pada rendahnya tingkat kepercayaan prediksi serta keterbatasan dalam menangkap pola *phishing* yang kompleks. Hal ini terlihat dari masih adanya *false negative*, yang dalam konteks keamanan siber merupakan kesalahan paling berbahaya karena dapat menyebabkan URL *phishing* tidak terdeteksi.

Tabel 6. 22 Kinerja Model Tunggal Random Forest

	Predicted Benign (0)	Predicted Phishing (1)
Actual Benign (0)	TN = 20	FP = 0
Actual Phishing (1)	FN = 12	TP = 8

- **Accuracy**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Accuracy = \frac{8 + 20}{8 + 20 + 0 + 12} = \frac{28}{40} = 0.70$$

$$Accuracy = 70\%$$

- **Precision**

$$Precision = \frac{TP}{TP + FP}$$

$$Precision = \frac{8}{8 + 0} = \frac{8}{8} = 1$$

$$Precision = 100\%$$

- **Recall**

$$Recall = \frac{TP}{TP + FN}$$

$$Recall = \frac{8}{8 + 12} = \frac{8}{20} = 0.40$$

$$Recall = 40\%$$

- **F1-Score**

$$F1 - Score = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

$$F1 - Score = 2 \frac{1.00 \times 0.40}{1.00 + 0.40}$$

$$F1 - Score = \frac{0.80}{1.40} = 0.57$$

$$F1 - Score = 57\%$$

Berdasarkan hasil perhitungan *confusion matrix* dan *evaluation metrics*, model Random Forest menunjukkan performa yang cukup baik dalam mengklasifikasikan URL, dengan nilai *accuracy* sebesar 70%. Nilai ini menunjukkan bahwa sebagian besar data uji berhasil diklasifikasikan dengan benar oleh model. Namun, jika dianalisis lebih dalam, model ini memiliki *precision* yang sangat tinggi (100%), yang berarti seluruh URL yang diprediksi sebagai *phishing* benar-benar merupakan *phishing* (tidak terdapat *false positive*). Hal ini menunjukkan bahwa model sangat hati-hati (konservatif) dalam memberikan label *phishing*. Nilai *recall* yang relatif rendah (40%) mengindikasikan bahwa masih terdapat sejumlah URL phishing yang tidak berhasil terdeteksi (*false negative*). Artinya, model masih melewatkan beberapa serangan *phishing*, terutama yang memiliki karakteristik menyerupai URL *benign*. Nilai F1-score sebesar 57% menunjukkan bahwa terdapat ketidakseimbangan antara *precision* dan *recall*, di mana model lebih unggul dalam menghindari kesalahan deteksi terhadap URL *benign* dibandingkan dalam mendeteksi seluruh kasus *phishing*.

Model Random Forest cenderung memiliki sifat konservatif, yaitu lebih memprioritaskan penghindaran kesalahan dalam mengklasifikasikan URL *benign* sebagai phishing (*false positive*), namun dengan konsekuensi meningkatnya jumlah phishing yang tidak terdeteksi (*false negative*).

6.2.1.2 Pembahasan Kinerja Model Tunggal XGBoost

Berdasarkan hasil eksperimen pada subbab 6.1.5.2, model XGBoost menunjukkan kemampuan deteksi *phishing* yang kuat, tetapi masih menghadapi tantangan pada kelas *benign*. Pada pengujian manual 40 URL (20 *benign* dan 20 *phishing*) dengan ambang keputusan 0,60, model menghasilkan 29 prediksi benar dan 11 prediksi salah, sehingga akurasi mencapai 72,5%. Rinciannya, pada kelas *benign* model mengklasifikasikan benar 11 URL dan salah 9 URL, sedangkan pada kelas *phishing* model mengklasifikasikan benar 18 URL dan salah 2 URL. Pola ini menegaskan bahwa XGBoost lebih sensitif mendeteksi *phishing* dibanding menjaga konsistensi klasifikasi *benign*.

Dari sisi distribusi skor, pemisahan kelas *phishing* terlihat cukup baik. Mayoritas URL *phishing* berada pada rentang skor tinggi, misalnya 0,65 sampai 0,95, yang menunjukkan keyakinan model terhadap indikasi serangan. Namun, masih terdapat dua URL *phishing* dengan skor rendah, yaitu 0,26 dan 0,35, sehingga diprediksi sebagai *benign*.

Pada kelas *benign*, rentang skor model sangat lebar, dari 0,06 hingga 0,93. Variasi ini menunjukkan bahwa beberapa URL *benign* memperoleh skor *phishing* cukup tinggi dan akhirnya salah diprediksi sebagai *phishing*. Hasil tersebut mengindikasikan adanya kemiripan pola fitur antara sebagian URL *benign* dan pola yang biasa ditemukan pada URL mencurigakan, atau kombinasi karakter domain yang memicu bobot risiko pada model *boosting*. Akibatnya, *false positive* pada kelas *benign* masih relatif.

Tabel 6. 23 Kinerja Model Tunggal XGBoost

	Predicted Benign (0)	Predicted Phishing (1)
Actual Benign (0)	TN = 11	FP = 9
Actual Phishing (1)	FN = 2	TP = 18

- **Accuracy**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Accuracy = \frac{18 + 11}{18 + 11 + 9 + 2} = \frac{29}{40} = 0.725$$

$$Accuracy = 72.5\%$$

- **Precision**

$$Precision = \frac{TP}{TP + FP}$$

$$Precision = \frac{18}{18 + 9} = \frac{18}{27} = 0.67$$

$$Precision = 67\%$$

- **Recall**

$$Recall = \frac{TP}{TP + FN}$$

$$Recall = \frac{18}{18 + 2} = \frac{18}{20} = 0.90$$

$$Recall = 90\%$$

- **F1-Score**

$$F1 - Score = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

$$F1 - Score = 2 \frac{0.69 \times 0.90}{0.69 + 0.90}$$

$$F1 - Score = \frac{1.242}{1.59} = 0.78$$

$$F1 - Score = 78\%$$

Berdasarkan hasil perhitungan, model XGBoost menghasilkan precision sebesar 66,7%, recall sebesar 90,0%, dan F1-score sebesar 78%. Nilai recall yang tinggi menunjukkan bahwa model sangat baik dalam mendeteksi phishing, namun precision yang lebih rendah mengindikasikan masih adanya kesalahan klasifikasi pada URL benign.

Dari perspektif penelitian, XGBoost memiliki keunggulan dalam menangkap pola phishing yang kompleks dan meminimalkan *false negative*. Namun, model ini masih perlu ditingkatkan dalam mengurangi *false positive* agar performa lebih seimbang. Oleh karena itu, XGBoost layak digunakan sebagai model utama, dengan kemungkinan penyesuaian ambang keputusan atau penambahan mekanisme validasi untuk meningkatkan akurasi pada kelas benign.

6.2.1.3 Pembahasan Kinerja Model Stacking Ensemble

Berdasarkan hasil eksperimen pada subbab 6.1.5.3, model *Stacking Ensemble* menunjukkan performa yang cukup baik pada data uji dengan akurasi sebesar 75,0%, yaitu 30 prediksi benar dari total 40 URL. Secara rinci, model mampu mendeteksi 18 dari 20 URL *phishing* dengan benar, namun hanya mampu mengklasifikasikan 12 dari 20 URL *benign* dengan benar. Hasil ini menunjukkan bahwa model memiliki kemampuan yang kuat dalam mendeteksi *phishing*, tetapi masih mengalami kelemahan pada identifikasi URL *benign*.

Jika dibandingkan dengan hasil evaluasi pada dataset penuh yang mencapai akurasi sekitar 97,33%, terlihat adanya kesenjangan performa yang cukup signifikan. Hal ini mengindikasikan bahwa pengujian pada sampel kecil sangat dipengaruhi oleh karakteristik URL yang diuji, khususnya URL benign yang memiliki struktur kompleks seperti dokumentasi teknis (misalnya *kubernetes*, *go.dev*, dan *git-scm*),

yang cenderung menghasilkan skor *phishing* tinggi dan menyebabkan *false positive*.

Dari sisi *confidence score*, model *Stacking* menunjukkan distribusi nilai yang relatif beragam. Pada kelas *phishing*, sebagian besar URL memiliki skor tinggi (di atas 0,65), yang menunjukkan kemampuan model dalam mengenali pola serangan dengan cukup baik. Efektivitas model sedikit terhambat oleh adanya distribusi skor yang rendah pada kategori *phishing*, yang memicu potensi *false negative*. Fenomena ini diikuti dengan munculnya skor kepercayaan yang tinggi pada kategori *benign* yang menyebabkan terjadinya misklasifikasi dalam bentuk *false positive*. Hal tersebut mengisyaratkan bahwa model *Stacking* masih mengalami ambiguitas dalam membedakan fitur spesifik antara kedua kelas tersebut pada kasus-kasus tertentu.

Tabel 6. 24 Kinerja Model Stacking Ensemble

	Predicted Benign (0)	Predicted Phishing (1)
Actual Benign (0)	TN = 12	FP = 8
Actual Phishing (1)	FN = 2	TP = 18

- **Accuracy**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Accuracy = \frac{18 + 12}{18 + 12 + 8 + 2} = \frac{30}{40} = 0.75$$

$$Accuracy = 75\%$$

- **Precision**

$$Precision = \frac{TP}{TP + FP}$$

$$Precision = \frac{18}{18 + 8} = \frac{18}{26} = 0.69$$

$$Precision = 69\%$$

- **Recall**

$$Recall = \frac{TP}{TP + FN}$$

$$Recall = \frac{18}{18 + 2} = \frac{18}{20} = 0.90$$

$$Recall = 90\%$$

- **F1-Score**

$$F1 - Score = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

$$F1 - Score = 2 \frac{0.69 \times 0.90}{0.69 + 0.90}$$

$$F1 - Score = \frac{1.242}{1.59} = 0.78$$

$$F1 - Score = 78\%$$

Ini menunjukkan bahwa meskipun *Stacking Ensemble* mampu menggabungkan keunggulan *Random Forest* dan *XGBoost*, model ini masih menghadapi keterbatasan dalam membedakan URL phishing yang tersamarkan dengan URL *benign* yang kompleks. Hal ini disebabkan oleh adanya korelasi kesalahan antara kedua model dasar, sehingga *meta-learner* tidak sepenuhnya mampu memperbaiki kesalahan prediksi.

Dari perspektif pertanyaan penelitian pertama, *Stacking Ensemble* memiliki keunggulan dalam meningkatkan stabilitas prediksi dibandingkan model tunggal, namun belum mampu memberikan peningkatan signifikan dalam akurasi pada data

uji nyata. Kelemahan utama terletak pada tingginya *false positive* pada URL benign dan masih adanya *false negative* pada *phishing* tertentu.

6.2.1.4 Pembahasan Integrasi Rule-Based Filtering dengan Stacking

Berdasarkan hasil eksperimen pada subbab 6.1.5.4, integrasi *Rule-Based Filtering* dengan *Stacking* menghasilkan akurasi sebesar 75,0%, dengan jumlah prediksi benar dan salah yang identik dengan model *Stacking Ensemble*, yaitu 30 prediksi benar dan 10 prediksi salah. Pada kelas *phishing*, model mampu mendeteksi 18 dari 20 URL dengan benar, sedangkan pada kelas benign hanya 12 dari 20 URL yang berhasil diklasifikasikan dengan tepat. Hasil ini menunjukkan bahwa secara kuantitatif, pendekatan *hybrid* belum memberikan peningkatan performa dibandingkan model *Stacking* murni.

Meskipun demikian, terdapat perbedaan mendasar dari sisi mekanisme klasifikasi. Pada pendekatan *hybrid*, sebagian URL diklasifikasikan secara langsung melalui *rule-based filtering* tanpa melalui proses *machine learning*. Berdasarkan hasil eksperimen pada 40 URL, terdapat 3 URL yang berhasil diproses langsung oleh mekanisme *rule-based*, yang umumnya memiliki karakteristik mencurigakan yang sangat eksplisit, seperti struktur URL yang panjang, penggunaan domain tidak lazim, serta kombinasi karakter acak.

Tabel 6. 25 Integrasi Rule-Based Filtering dengan Stacking

	Predicted Benign (0)	Predicted Phishing (1)
Actual Benign (0)	TN = 12	FP = 8
Actual Phishing (1)	FN = 2	TP = 18

- **Accuracy**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Accuracy = \frac{18 + 12}{18 + 12 + 8 + 2} = \frac{30}{40} = 0.75$$

$$Accuracy = 75\%$$

- **Precision**

$$Precision = \frac{TP}{TP + FP}$$

$$Precision = \frac{18}{18 + 8} = \frac{18}{26} = 0.69$$

$$Precision = 69\%$$

- **Recall**

$$Recall = \frac{TP}{TP + FN}$$

$$Recall = \frac{18}{18 + 2} = \frac{18}{20} = 0.90$$

$$Recall = 90\%$$

- **F1-Score**

$$F1 - Score = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

$$F1 - Score = 2 \frac{0.69 \times 0.90}{0.69 + 0.90}$$

$$F1 - Score = \frac{1.242}{1.59} = 0.78$$

$$F1 - Score = 78\%$$

Keberadaan mekanisme ini menunjukkan bahwa *rule-based filtering* berfungsi sebagai lapisan awal (*pre-filtering*) yang mampu menangani kasus-kasus yang bersifat jelas tanpa memerlukan proses komputasi yang lebih kompleks. Dengan demikian, meskipun jumlahnya masih terbatas pada eksperimen skala kecil, pendekatan ini secara konseptual mampu mengurangi beban komputasi pada model *machine learning*, karena sebagian data tidak perlu diproses melalui tahapan *Stacking*. Dalam konteks sistem berskala besar, kontribusi ini berpotensi memberikan dampak efisiensi yang lebih signifikan, terutama pada skenario *real-time processing* dengan volume data yang tinggi.

Namun demikian, pendekatan ini masih menghadapi permasalahan yang sama dengan model *Stacking*, yaitu tingginya *false positive* pada URL benign yang memiliki karakteristik kompleks. Hal ini menunjukkan bahwa aturan yang digunakan dalam *rule-based filtering* belum mampu membedakan secara efektif antara URL *phishing* dan URL benign yang memiliki struktur serupa. Selain itu, beberapa URL *phishing* yang tersamarkan juga tetap tidak terdeteksi, seperti pada kasus skor rendah yang berada di bawah ambang klasifikasi. Temuan ini menunjukkan bahwa efektivitas aturan masih terbatas dalam menangkap pola *phishing* yang semakin adaptif.

Dari sisi *confidence score*, pendekatan *hybrid* menghasilkan kombinasi antara keputusan deterministik dan probabilistik. URL yang diproses melalui aturan menghasilkan keputusan yang tegas, sedangkan URL yang diproses melalui *Stacking* tetap menunjukkan variasi skor probabilitas. Kombinasi ini menciptakan dua karakteristik keputusan dalam satu sistem, yang tidak secara langsung meningkatkan akurasi, namun memberikan fleksibilitas dalam mekanisme deteksi.

Dari perspektif pertanyaan penelitian kedua, integrasi *rule-based filtering* tidak memberikan peningkatan performa akurasi pada data uji, namun memberikan kontribusi pada efisiensi pemrosesan sistem. Kemampuan untuk memproses sebagian URL secara langsung tanpa melalui model *machine learning* menunjukkan potensi pengurangan beban komputasi, meskipun pada eksperimen ini masih dalam skala terbatas. Oleh karena itu, pendekatan *hybrid* lebih tepat diposisikan sebagai strategi peningkatan efisiensi, bukan sebagai pendekatan untuk meningkatkan akurasi model.

Meskipun dalam eksperimen ini *rule-based filtering* berkontribusi terhadap peningkatan *false positive*. Penelitian ini memperkuat bahwa pendekatan deteksi phishing yang efektif tidak bergantung pada satu metode tunggal, melainkan pada integrasi berbagai pendekatan yang saling melengkapi, yaitu kecepatan dan determinisme *rule-based filtering*, kemampuan generalisasi *Random Forest*, ketajaman batas keputusan *XGBoost*, serta fleksibilitas adaptif dari *stacking ensemble* dalam suatu sistem yang terintegrasi.

6.2.2 Pembahasan Efisiensi dan Konsistensi Kinerja Sistem

Dalam penelitian ini, analisis efisiensi sistem tidak didasarkan pada hasil eksperimen dengan data uji berjumlah 40 URL, karena ukuran dataset yang terbatas serta proses input yang dilakukan secara manual dinilai belum mampu merepresentasikan efisiensi pemrosesan implementasi sistem. Oleh karena itu, evaluasi efisiensi dilakukan menggunakan keseluruhan *data testing* yang berjumlah 2.286 URL, yang diperoleh dari proses pembagian dataset sebesar 80:20. Pendekatan ini dipilih karena lebih mencerminkan skenario operasional sistem secara realistis, di mana model diuji pada data yang belum pernah dilihat sebelumnya.

6.2.2.1 Pembahasan Efisiensi Pemrosesan Rule-Based Filtering

Berdasarkan hasil eksperimen, penerapan *rule-based filtering* memungkinkan sebagian URL diklasifikasikan secara langsung tanpa melalui model Stacking. Hal ini menunjukkan bahwa pendekatan tersebut efektif dalam meningkatkan efisiensi

pemrosesan, karena mampu mengurangi beban komputasi pada model utama. Ringkasan hasil efisiensi pemrosesan ditunjukkan pada Tabel 6. berikut.

Tabel 6. 26 Perbandingan Efisiensi Pemrosesan Model

Aspek	Stacking	Hybrid (Rule + Stacking)	Analisis
Total URL	2.286	2.286	Sama
URL diproses Rule-Based	0 (0%)	418 (18,3%)	<i>Hybrid</i> mengurangi beban ML
URL diproses ML	2.286 (100%)	1.868 (81,7%)	Ada reduksi beban komputasi
Batch Time (detik)	0.303080	0.294937	Lebih cepat 2,7%
Penghematan Waktu	-	0.008143 detik	Efisiensi kecil tapi nyata
Throughput (URL/s)	7.542	7.750	<i>Hybrid</i> sedikit lebih tinggi

Berdasarkan Tabel 6. 26 sebanyak 418 URL atau 18,3% dari total data uji berhasil disaring pada tahap awal menggunakan *rule-based filtering*, sehingga hanya 81,7% data yang perlu diproses oleh model *machine learning*. Hal ini menunjukkan adanya pengurangan beban komputasi yang cukup signifikan. Hasil pengukuran menunjukkan penurunan *batch time* dari 0,303080 detik pada model *Stacking* murni menjadi 0,294937 detik pada model *Hybrid*, yang setara dengan penghematan waktu sebesar 0,008143 detik atau sekitar 2,7%. Nilai *throughput* turut meningkat dari 7.542 URL/detik menjadi 7.750 URL/detik.

Meskipun reduksi *batch time* sebesar 2,7% terlihat moderat pada skala eksperimen ini, perlu dipahami bahwa peningkatan efisiensi tersebut tidak linier dengan proporsi URL yang disaring (18,3%). Hal ini disebabkan oleh fakta bahwa operasi *rule-based filtering* sendiri memiliki biaya komputasi tersendiri berupa evaluasi *risk score* per URL, sehingga penghematan bersih lebih kecil dari yang secara teoritis diharapkan. Hasil peningkatan efisiensi ini terlihat relatif kecil dalam skala eksperimen, secara teoritis dampaknya akan menjadi lebih signifikan pada sistem dengan volume data yang besar atau dalam skenario *real-time processing*. Hal ini

karena proses penyaringan awal dapat mengurangi jumlah data yang harus diproses oleh model yang lebih kompleks, sehingga meningkatkan efisiensi secara kumulatif.

6.2.2.2 Analisis Latency Prediksi per URL dan Konsistensi Kinerja

Untuk melihat efisiensi pada tingkat individu, dilakukan analisis terhadap *latency* prediksi per URL yang ditunjukkan pada Tabel 6. berikut.

Tabel 6. 27 Latency Prediksi per URL

Model	Latency(ms)	Min(ms)	Max(ms)	Std(ms)
Random Forest	165.708	120.112	263.782	42.488
XGBoost	31.400	28.996	34.000	1.686
Stacking	226.340	215.371	244.218	6.985
Hybrid	231.175	221.200	243.215	4.577

Berdasarkan tabel Tabel 6. 27, hasil pengukuran menunjukkan pertama XGBoost merupakan model paling efisien dengan latency 31,400 ms, jauh lebih rendah dibandingkan *Random Forest* (165,708 ms). Hal ini dapat dijelaskan oleh arsitektur boosting dengan *shallow trees* (*max_depth* = 5) yang secara inheren lebih cepat dalam inferensi tunggal dibandingkan ensemble pohon dalam (*deep trees*, *max_depth* = 16) pada *Random Forest*.

Kedua, model *Hybrid* memiliki *latency* per URL yang sedikit lebih tinggi (231,175 ms) dibandingkan *Stacking* (226,340 ms). Hal ini secara teknis disebabkan oleh adanya *overhead* dari proses evaluasi *risk score* rule-based sebelum keputusan routing ke model ML dibuat. Dalam konteks pemrosesan tunggal, *overhead* tersebut berdampak lebih besar secara proporsional dibanding pada pemrosesan batch.

Ketiga, dan paling relevan secara operasional, model *Hybrid* menunjukkan standar deviasi *latency* yang lebih rendah (Std = 4,577 ms) dibandingkan *Stacking* (6,985 ms) maupun *Random Forest* (42,488 ms). Temuan ini memiliki implikasi penting yang tidak boleh diabaikan: meskipun rata-rata *latency Hybrid* sedikit lebih tinggi,

variabilitas yang lebih kecil mengindikasikan konsistensi prediksi yang lebih baik. Dalam konteks sistem deteksi keamanan siber yang beroperasi secara *real-time*, konsistensi waktu respons sering kali lebih bernilai dari sekadar kecepatan rata-rata semata, karena lonjakan latency (*latency spike*) yang tidak terduga dapat mengganggu stabilitas sistem secara keseluruhan.

Berdasarkan keseluruhan analisis, penerapan *rule-based filtering* sebagai lapisan awal memberikan pengaruh terhadap dua aspek utama. Dari dimensi efisiensi pemrosesan, lapisan ini berhasil mereduksi beban komputasi model ML sebesar 18,3% (418 dari 2.286 URL data test), menghasilkan peningkatan *throughput* sebesar 2,8% (dari 7.542 menjadi 7.750 URL/detik) dan penghematan *batch time* sebesar 2,7% (dari 0,303080 detik menjadi 0,294937 detik). Meskipun peningkatan absolut relatif kecil pada skala eksperimen yang terbatas, efisiensi kumulatif ini akan semakin signifikan pada sistem produksi berskala besar.

Dari dimensi konsistensi kinerja prediksi, model *Hybrid* menunjukkan standar deviasi *latency* yang lebih rendah (4,577 ms berbanding 6,985 ms pada *Stacking*), yang mengimplikasikan perilaku sistem yang lebih stabil dan dapat diprediksi dalam skenario operasional. Meskipun rata-rata *latency* per URL *Hybrid* sedikit lebih tinggi (231,175 ms vs. 226,340 ms), nilai Std yang lebih kecil menunjukkan bahwa sistem lebih stabil dalam menghadapi perubahan variasi beban. Oleh karena itu, integrasi *rule-based filtering* paling optimal diterapkan pada skenario yang memprioritaskan stabilitas respons dan skalabilitas komputasi, dengan tetap memastikan bahwa ambang batas aturan disesuaikan secara berkala untuk menyesuaikan terhadap evolusi pola serangan *phishing* yang baru.

Berdasarkan seluruh hasil analisis, dapat disimpulkan bahwa penerapan *rule-based filtering* sebagai lapisan awal memberikan pengaruh positif terhadap efisiensi pemrosesan dan konsistensi kinerja sistem deteksi *phishing*. Pendekatan ini mampu mengurangi beban komputasi model *machine learning* sekaligus meningkatkan stabilitas waktu prediksi. Oleh karena itu, metode ini lebih optimal diterapkan pada sistem dengan kebutuhan respons yang konsisten dan beban pemrosesan yang

tinggi, dengan tetap melakukan peninjauan dan penyesuaian aturan secara berkala untuk mengikuti perkembangan pola serangan *phishing*.

DAFTAR PUSTAKA

- [1] L. A. Febrika Ardy, I. Istiqomah, A. E. Ezer, and S. N. Neyman, "Phishing di Era Media Sosial: Identifikasi dan Pencegahan Ancaman di Platform Sosial," *J. Internet Softw. Eng.*, vol. 1, no. 4, p. 11, 2024, doi: 10.47134/pjise.v1i4.2753.
- [2] E. Mouncey and S. Ciobotaru, "Phishing scams on social media: An evaluation of cyber awareness education on impact and effectiveness," *J. Econ. Criminol.*, vol. 7, no. December 2024, p. 100125, 2025, doi: 10.1016/j.jeconc.2025.100125.
- [3] R. Zieni, L. Massari, and M. C. Calzarossa, "Phishing or Not Phishing? A Survey on the Detection of Phishing Websites," *IEEE Access*, vol. 11, no. January, pp. 18499–18519, 2023, doi: 10.1109/ACCESS.2023.3247135.
- [4] F. P. E. Putra, U. Ubaidi, A. Zulfikri, G. Arifin, and R. M. Ilhamsyah, "Analysis of Phishing Attack Trends, Impacts and Prevention Methods: Literature Study," *Brill. Res. Artif. Intell.*, vol. 4, no. 1, pp. 413–421, 2024, doi: 10.47709/brilliance.v4i1.4357.
- [5] A. January-march, "1 Quarter," no. July, pp. 1–13, 2025.
- [6] A. Khan, D. M. Ahmed, and A. Fathima, "Enhanced Phishing Detection Using Machine Learning Algorithms: A Comparative Study of Random Forest, SVM, and Logistic Regression Models," *SSRN Electron. J.*, 2025, doi: 10.2139/ssrn.5191566.
- [7] V. A. Windarni, A. F. Nugraha, S. T. A. Ramadhani, D. A. Istiqomah, F. M. Puri, and A. Setiawan, "Deteksi Website Phishing Menggunakan Teknik Filter Pada Model Machine Learning," *Inf. Syst. J.*, vol. 6, no. 01, pp. 69–80, 2023, doi: 10.24076/infosjournal.2023v6i01.1268.
- [8] A. B. Nassif, M. A. Talib, Q. Nasir, and F. M. Dakalbab, "Machine Learning for Anomaly Detection: A Systematic Review," *IEEE Access*, vol. 9, pp. 78658–78700, 2021, doi: 10.1109/ACCESS.2021.3083060.
- [9] A. Almomani *et al.*, "Phishing Website Detection With Semantic Features Based on Machine Learning Classifiers: A Comparative Study," *International Journal on Semantic Web and Information Systems*, vol. 18,

- no. 1. 2022. doi: 10.4018/IJSWIS.297032.
- [10] J. Tanimu and S. Shiaeles, “Phishing Detection Using Machine Learning Algorithm,” *Proc. 2022 IEEE Int. Conf. Cyber Secur. Resilience, CSR 2022*, pp. 317–322, 2022, doi: 10.1109/CSR54599.2022.9850316.
 - [11] V. Adeyemi Onih, “Phishing Detection Using Machine Learning: A Model Development and Integration,” *Int. J. Sci. Manag. Res.*, vol. 07, no. 04, pp. 27–63, 2024, doi: 10.37502/ijsmr.2024.7403.
 - [12] M. S. I. Ovi, M. H. Rahman, and M. A. Hossain, “PhishGuard: A Multi-Layered Ensemble Model for Optimal Phishing Website Detection,” *2024 6th Int. Conf. Sustain. Technol. Ind. 5.0, STI 2024*, 2024, doi: 10.1109/STI64222.2024.10951075.
 - [13] Sohaib Latif and Saher Pervaiz, “Detecting Phishing Attacks in Cybersecurity Using Machine Learning With Data Preprocessing and Feature Engineering,” *Kashf J. Multidiscip. Res.*, vol. 2, no. 03, pp. 89–99, 2025, doi: 10.71146/kjmr335.
 - [14] M. Dharani, S. Badkul, K. Gharat, A. Vidhate, and D. Bhosale, “Detection of Phishing Websites Using Ensemble Machine Learning Approach,” vol. 03012, pp. 1–5, 2021.
 - [15] M. A. Tamal, M. K. Islam, T. Bhuiyan, A. Sattar, and N. U. Prince, “Unveiling suspicious phishing attacks: enhancing detection with an optimal feature vectorization algorithm and supervised machine learning,” *Front. Comput. Sci.*, vol. 6, no. January, 2024, doi: 10.3389/fcomp.2024.1428013.
 - [16] R. Kumar, R. Kumar, R. K. Sahu, R. Patra, and A. Ghosh, “Detection of Phishing Websites Using Machine Learning,” *Smart Innov. Syst. Technol.*, vol. 313, pp. 317–330, 2023, doi: 10.1007/978-981-19-8669-7_29.
 - [17] I. D. Mienye and Y. Sun, “A Survey of Ensemble Learning: Concepts, Algorithms, Applications, and Prospects,” *IEEE Access*, vol. 10, no. September, pp. 99129–99149, 2022, doi: 10.1109/ACCESS.2022.3207287.
 - [18] C. Giraud-Carrier, *Combining Base-Learners into Ensembles*. Springer International Publishing, 2022. doi: 10.1007/978-3-030-67024-5_9.
 - [19] M. Al-Sarem *et al.*, “An optimized stacking ensemble model for phishing websites detection,” *Electron.*, vol. 10, no. 11, pp. 1–18, 2021, doi:

10.3390/electronics10111285.

- [20] S. Shafieian and M. Zulkernine, “Multi-layer stacking ensemble learners for low footprint network intrusion detection,” *Complex Intell. Syst.*, vol. 9, no. 4, pp. 3787–3799, 2023, doi: 10.1007/s40747-022-00809-3.
- [21] Y. Mourtaji, M. Bouhorma, D. Alghazzawi, G. Aldabbagh, and A. Alghamdi, “Hybrid Rule-Based Solution for Phishing URL Detection Using Convolutional Neural Network,” *Wirel. Commun. Mob. Comput.*, vol. 2021, 2021, doi: 10.1155/2021/8241104.
- [22] C. Zhao, X. Yuan, J. Long, L. Jin, and B. Guan, “Chinese stock market Preprint not peer reviewed,” pp. 1–20, 2025.
- [23] A. Safi and S. Singh, “A systematic literature review on phishing website detection techniques,” *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 35, no. 2, pp. 590–611, 2023, doi: 10.1016/j.jksuci.2023.01.004.
- [24] E. S. Aung, C. T. Zhan, and H. Yamana, “A Survey of URL-based Phishing Detection,” pp. 1–8, 2019, [Online]. Available: <http://quadrodefertas.com.br/www1>.
- [25] R. Basnet and A. H. Sung, “Rule-Based Phishing Attack Detection Rule-Based Phishing Attack Detection,” no. October, 2016.
- [26] N. S. Zaini *et al.*, “Phishing detection system using machine learning classifiers,” *Indones. J. Electr. Eng. Comput. Sci.*, vol. 17, no. 3, pp. 1165–1171, 2020, doi: 10.11591/ijeecs.v17.i3.pp1165-1171.
- [27] M. Elsadig *et al.*, “Intelligent Deep Machine Learning Cyber Phishing URL Detection Based on BERT Features Extraction,” *Electron.*, vol. 11, no. 22, 2022, doi: 10.3390/electronics11223647.
- [28] Y. Wang, “Malicious URL Detection An Evaluation of Feature Extraction and Machine Learning Algorithm,” *Highlights Sci. Eng. Technol.*, vol. 23, pp. 117–123, 2022, doi: 10.54097/hset.v23i.3209.
- [29] D. Menaga, S. Vijay, V. Vignesh, and S. Ramalakshmi, “An Efficient Detection of Phishing Website using Machine Learning,” *Proc. 5th Int. Conf. Smart Electron. Commun. ICOSEC 2024*, vol. 11, no. 5, pp. 1189–1194, 2024, doi: 10.1109/ICOSEC61587.2024.10722540.
- [30] H. Ghuge, D. Ghuge, A. Rangneniwar, P. Samudra, and A. V Markad, “Real-

Time Detection of Malicious URLs Using Feature-Based Machine Learning Approaches,” vol. 13, no. 2, pp. 1–9, 2025.

- [31] R. Alazaidah *et al.*, “Website Phishing Detection Using Machine Learning Techniques,” *J. Stat. Appl. Probab.*, vol. 13, no. 1, pp. 119–129, 2024, doi: 10.18576/jsap/130108.
- [32] H. Gao *et al.*, “Machine learning in business and finance: a literature review and research opportunities,” *Financ. Innov.*, vol. 10, no. 1, 2024, doi: 10.1186/s40854-024-00629-z.
- [33] S. Alrefaai, G. Ozdemir, and A. Mohamed, “Detecting Phishing Websites Using Machine Learning,” *HORA 2022 - 4th Int. Congr. Human-Computer Interact. Optim. Robot. Appl. Proc.*, pp. 1–17, 2022, doi: 10.1109/HORA55278.2022.9799917.
- [34] A. U. Rehman, I. Imtiaz, S. Javaid, and M. Muslih, “Real-Time Phishing URL Detection Using Machine Learning †,” pp. 1–11, 2025.
- [35] J. Julio, M. Lamas, and R. W. Portillo, “Web architecture for URL-based phishing detection based on Random Forest , Classification Trees , and Support Vector Machine,” vol. 25, no. 69, pp. 107–121, 1988, doi: 10.4114/intartif.vol25iss69pp107-121.
- [36] F. Nthurima, A. Mutua, and W. S. Titus, “Detecting Phishing Emails Using Random Forest and AdaBoost Classifier Model,” vol. 6, no. 2, pp. 123–136, 2023.
- [37] P. M. Dinesh, M. Mukesh, B. Navaneethan, R. S. Sabeenian, M. E. Paramasivam, and A. Manjunathan, “Identification of Phishing Machine Learning Algorithm Attacks using,” vol. 04010, 2023.
- [38] D. Revaldo, “Implementation of Random Forest Classification and Support Vector Machine Algorithms for Phishing Link Detection,” vol. 8106, pp. 127–137, 2024.
- [39] R. Yang, K. Zheng, B. Wu, and C. Wu, “Phishing Website Detection Based on Deep Convolutional Neural Network and Random Forest Ensemble Learning,” pp. 1–18, 2021.
- [40] A. Fajar, S. Yazid, and I. Budi, “Enhancing Phishing Detection through Feature Importance Analysis and Explainable AI : A Comparative Study of

CatBoost , XGBoost , and EBM Models”.

- [41] L. Jovanovic, D. Jovanovic, and M. Antonijevic, “Improving Phishing Website Detection Using a Hybrid Two-level Framework for Feature Selection and XGBoost Tuning,” vol. 22, pp. 543–574, 2023, doi: 10.13052/jwe1540-9589.2237.
- [42] R. Alzubi, “Improving Web Security through Machine Learning : A Feature-Based Methodology for Detecting Phishing URLs,” vol. 15, no. 5, pp. 26845–26851, 2025.
- [43] R. B. Basnet, A. H. Sung, and Q. Liu, “Rule-Based Phishing Attack Detection”.
- [44] F. Salahdine, Z. El Mrabet, and N. Kaabouch, “Phishing Attacks Detection A Machine Learning-Based Approach,” 2021.
- [45] V. Sai and C. Telaprolu, “Analyzing URL Structure for Machine Learning : Feature Engineering and Classification Applications,” vol. 2024, pp. 1–5, 2024.
- [46] H. Dalianis, “Evaluation Metrics and Evaluation,” *Clinical Text Mining*. pp. 45–53, 2018. doi: 10.1007/978-3-319-78503-5_6.
- [47] Ž. Vujović, “Classification Model Evaluation Metrics,” *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 6, pp. 599–606, 2021, doi: 10.14569/IJACSA.2021.0120670.